[TÊN TRƯỜNG / KHOA]

ĐỒ ÁN / LUẬN VĂN

TRỢ LÝ SĂN MỐI ĐE DỌA DỰA TRÊN PEAK ASSISTANT

Thực thi tất định trên telemetry và khuyến nghị hỗ trợ người quyết định

(AI Agent Hunting v7: tích hợp PEAK Assistant của Cisco Talos, kiểm chứng trên bộ dữ liệu Boss of the SOC v1)

Sinh viên thực hiện: [Họ và tên]

Mã số sinh viên: [MSSV]

Giảng viên hướng dẫn: [Học hàm, học vị, họ và tên]

[Địa điểm], [tháng/năm]

LỜI CAM ĐOAN

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi dưới sự hướng dẫn của giảng viên hướng dẫn. Các số liệu và kết quả thực nghiệm trong đồ án là trung thực, được sinh ra từ mã nguồn và dữ liệu đi kèm (kho `AI_Agent_Hunting`, thư mục `results/`); các tài liệu tham khảo đều được trích dẫn rõ nguồn.

Họ tên sinh viên: [Họ và tên] Chữ ký: _____

TÓM TẮT

Săn mối đe dọa (threat hunting) là hoạt động chủ động tìm những xâm nhập mà cơ chế phát hiện tự động bỏ sót. Khung PEAK của Splunk SURGe chia mỗi cuộc săn thành ba pha Prepare, Execute và Act, trong đó pha Prepare (chọn chủ đề, nghiên cứu, giả thuyết, mô hình ABLE, phạm vi, kế hoạch) tốn nhiều công sức nhưng lại phù hợp để trợ lý trí tuệ nhân tạo hỗ trợ. Cisco Talos đã công bố mã nguồn mở PEAK Assistant, một tập tác tử dùng mô hình ngôn ngữ lớn (LLM) để viết báo cáo nghiên cứu, bảng ABLE và kế hoạch săn.

Đồ án này xây dựng AI Agent Hunting v7, một hệ thống ghép PEAK Assistant vào một đường thực thi tất định. Hệ thống nhận một PoC (giả thuyết đã cụ thể hoá thành các vị từ `trường – phép so sánh – giá trị`), nhờ PEAK Assistant viết bảng ABLE và kế hoạch săn, chạy các vị từ trên telemetry (SQLite), kiểm tra độ phủ của nguồn dữ liệu, rồi sinh một khuyến nghị có xếp hạng, kèm lý do, giới hạn, câu hỏi và rủi ro để **người săn quyết định**. Thiết kế giữ ba nguyên tắc: LLM không tạo hay sửa bằng chứng; kết quả rỗng không bao giờ bị đọc thành "sạch" khi nguồn dữ liệu thiếu hoặc chưa chứng minh được; LLM là thành phần tuỳ chọn, hệ thống vẫn chạy và vẫn đưa khuyến nghị theo luật khi không có LLM.

Hệ thống được kiểm chứng trên bộ dữ liệu Boss of the SOC v1 (4.417.543 sự kiện) với bốn PoC có sẵn. PoC khai thác Joomla có ít nhất vài nghìn bản ghi khớp (200 bản ghi được giữ làm bằng chứng) và khuyến nghị chuyển IR ở mức tin cậy trung bình, vì dữ liệu chỉ chứng minh hoạt động quét chứ chưa chứng minh khai thác thành công. Ba PoC còn lại không có bản ghi khớp và được khuyến nghị đóng kèm cảnh báo, không phải kết luận "sạch": cửa sổ chỉ phủ một trong hai mươi tám ngày dữ liệu, nguồn có thể thiếu đúng loại sự kiện cần tìm, và một lượt rà soát cuối phát hiện máy được nêu trong ba PoC này không ghi nguồn tương ứng nên các PoC chưa từng được tìm đúng chỗ (đã sửa bằng cách hiển thị phạm vi và chạy lại không lọc máy). Đồ án cũng mô tả các lỗi gặp phải khi vận hành LLM thật (lỗi 404 thoáng qua của endpoint, đầu ra JSON không ổn định) và cách xử lý, đồng thời cung cấp hướng dẫn thực hành từ cài đặt đến viết PoC mới.

Ở giai đoạn sau, đồ án đổi trọng tâm: hệ thống chỉ đảm nhận pha **Prepare** và không chạm vào hệ thống nội bộ. Đầu vào là PoC công khai (CVE trên NVD, các kho GitHub nhiều sao); đầu ra là một *kế hoạch săn* dạng dữ liệu (`HuntPlan`) gồm các giai đoạn tấn công, dấu vết, truy vấn SPL chỉ đọc viết bằng placeholder, giới hạn và điểm dừng, để một đội khác thực thi (pha Execute). Mọi truy vấn do LLM viết phải qua bộ kiểm tra tĩnh trước khi rời dự án; truy vấn hậu khai thác trên dữ liệu tiến trình bắt buộc lọc theo tiến trình cha; chữ lẫn ngôn ngữ khác bị yêu cầu viết lại. Khi đội kia trả kết quả, lệnh `verify` (không dùng LLM) quyết định chấp nhận (`ACCEPT_NO_EVIDENCE`, không bao giờ là "sạch"), chuyển IR, bổ sung dữ liệu, chạy lại, hoặc sinh vòng pivot

kế tiếp. Hướng này được thử trên Log4Shell (CVE-2021-44228) và Spring4Shell (CVE-2022-22965) bằng mô hình miễn phí; vòng xác minh mới chỉ được thử bằng kết quả giả lập, và các truy vấn chưa chạy trên Splunk thật.

Từ khoá: săn mối đe dọa, PEAK, PEAK Assistant, ABLE, mô hình ngôn ngữ lớn, hệ thống đa tác tử, hỗ trợ ra quyết định, Boss of the SOC.

ABSTRACT

Threat hunting looks for intrusions that automated detection misses. The PEAK framework (Splunk SURGe) splits a hunt into Prepare, Execute and Act. Prepare is labour-intensive and well suited to an AI assistant; Cisco Talos released PEAK Assistant, an open-source set of LLM agents that write research reports, ABLE tables and hunt plans.

This project builds AI Agent Hunting v7, which couples PEAK Assistant with a deterministic execution path. Given a PoC (a hypothesis expressed as `field – operator – value` predicates), the system asks PEAK Assistant for the ABLE table and hunt plan, runs the predicates against telemetry (SQLite), checks data-source coverage, and produces a ranked recommendation with reasons, caveats, questions and risks so that a **human hunter decides**. Three principles hold throughout: the LLM never creates or edits evidence; an empty result is never read as "clean" when data is missing or unproven; the LLM is optional and the system still produces rule-based recommendations without it.

The system was evaluated on Boss of the SOC v1 (4,417,543 events) with four existing PoCs. The Joomla PoC matched at least thousands of records (200 kept as evidence) and was recommended for IR escalation at medium confidence, because the data proves scanning but not successful exploitation. The other three PoCs matched nothing and were recommended for closure with caveats rather than declared clean: the window covered one of twenty-eight days, the source may lack the required event type, and a final review found that the host named in these three PoCs does not log the relevant source, so the PoCs had never searched the right place (fixed by disclosing the scope and re-running without the host filter). The thesis also documents the failures met when operating a real LLM (transient 404 responses, unstable JSON output) and how they were handled, and includes a practical guide from installation to authoring new PoCs.

In its later stage the project narrows to the **Prepare** phase alone and never touches internal systems. The input is a public PoC (a CVE on NVD, highly starred GitHub repositories); the output is a *hunt plan* as data (`HuntPlan`): attack stages, observables, read-only SPL written with placeholders, limits and stop conditions, to be executed by another team (the Execute phase). Every search written by the LLM must pass a static checker before it leaves the project; post-exploitation searches over process data must filter on the parent process; stray words of other languages trigger a rewrite. When the other team returns results, `verify` (no LLM) decides to accept (`ACCEPT_NO_EVIDENCE`, never "clean"), escalate, collect data, rerun, or generate the next pivot round. The approach was tried on Log4Shell (CVE-2021-44228) and

Spring4Shell (CVE-2022-22965) with a free model; the verification loop has only been exercised with simulated results, and the searches have not run on a real Splunk.

Keywords: threat hunting, PEAK, PEAK Assistant, ABLE, large language models, multi-agent systems, decision support, Boss of the SOC.

MỤC LỤC

TÓM TẮT iii
ABSTRACT v
Chương 1. Mở đầu 1
1.1. Đặt vấn đề 1
1.2. Bối cảnh và động cơ thực hiện 1
1.3. Mục tiêu nghiên cứu 2
1.4. Đối tượng và phạm vi 2
1.5. Phương pháp nghiên cứu 3
1.6. Đóng góp chính 3
1.7. Bố cục luận văn 4
Chương 2. Cơ sở lý thuyết 5
2.1. Săn mối đe dọa 5
2.2. Khung PEAK 6
2.3. MITRE ATT&CK và chỉ báo thực thi 7
2.4. Mô hình ngôn ngữ lớn và hệ thống đa tác tử 8
2.5. Độ tin cậy của bằng chứng và bằng chứng âm tính 8
2.6. Hỗ trợ ra quyết định 9
2.7. Telemetry và bộ dữ liệu Boss of the SOC v1 9
2.8. Tổng hợp: từ lý thuyết đến quyết định thiết kế 9
Chương 3. Phân tích bài toán và yêu cầu hệ thống 11
3.1. Hiện trạng kho mã v6 11
3.2. Vấn đề cần giải quyết 11
3.3. Yêu cầu chức năng 12
3.4. Yêu cầu phi chức năng 12
3.5. Các phương án thiết kế 12
3.6. Các bất biến thiết kế 13
3.7. Định hướng mới: chỉ làm pha Prepare 14
Chương 4. Thiết kế hệ thống 16
4.1. Tổng quan kiến trúc 16
4.2. Luồng xử lý một PoC 16

4.3. Mô hình dữ liệu 18
4.4. Lớp cầu nối PEAK Assistant 19
4.5. Bộ thực thi và adapter 19
4.6. Bộ luật khuyến nghị 21
4.7. Thiết kế bảo mật và quyền riêng tư 23
4.8. Cấu trúc đầu ra 24
4.9. Thiết kế luồng Prepare-only 24
Chương 5. Cài đặt hệ thống 30
5.1. Môi trường và công nghệ 30
5.2. Cấu trúc mã nguồn 30
5.3. Lớp LLM (`llm.py`) 31
5.4. Cầu nối PEAK (`prepare.py`) 33
5.5. Bộ thực thi PoC (`poc/agent.py`) 33
5.6. Adapter 34
5.7. Khuyến nghị và báo cáo 34
5.8. Dòng lệnh 35
5.9. Kiểm thử 35
5.10. Cài đặt luồng Prepare-only 36
Chương 6. Hướng dẫn thực hành 39
6.1. Chuẩn bị môi trường 39
6.2. Cấu hình LLM 39
6.3. Chuẩn bị dữ liệu telemetry 40
6.4. Chạy cuộc săn đầu tiên 41
6.5. Đọc và dùng báo cáo 42
6.6. Viết PoC mới 43
6.7. Dùng với Splunk thật 44
6.8. Xử lý sự cố 44
6.9. Quy trình tái lập kết quả của luận văn 45
6.10. Chạy luồng Prepare-only 45
Chương 7. Thực nghiệm và đánh giá 47
7.1. Thiết lập thực nghiệm 47

7.2. Kết quả chính 48
7.3. So sánh ba cấu hình 51
7.4. Các lỗi đã phát hiện nhờ thực nghiệm 51
7.5. Gọi LLM: miễn phí và độ tin cậy của endpoint 53
7.6. Kiểm thử đơn vị và tính hồi quy 55
7.7. Đánh giá theo yêu cầu 55
7.8. Thời gian chạy 56
7.9. Điều chưa được kiểm chứng 56
7.10. Thực nghiệm luồng Prepare-only 57
Chương 8. Thảo luận 61
8.1. Những gì đồ án đạt được 61
8.2. So sánh với phương án thay thế 61
8.3. Hạn chế 62
8.4. Các mối đe dọa đối với tính hợp lệ 63
8.5. An toàn và trách nhiệm khi dùng 64
8.6. Bài học rút ra 64
8.7. Hạn chế và rủi ro của hướng Prepare-only 64
Chương 9. Kết luận và hướng phát triển 67
9.1. Kết luận 67
9.2. Hạn chế còn lại 68
9.3. Hướng phát triển 68
Tài liệu tham khảo 70
Phụ lục 72
Phụ lục A. Tệp PoC mẫu 72
Phụ lục B. Trích báo cáo khuyến nghị mẫu 73
Phụ lục C. Tham chiếu dòng lệnh và biến môi trường 74
Phụ lục D. Lược đồ `recommendation.json` 76
Phụ lục E. Danh sách kiểm thử 77
Phụ lục F. Kết quả từng lần chạy 78
Phụ lục G. Cấu trúc thư mục kho 78
Phụ lục H. Bảng thuật ngữ 79

Phụ lục I. Trích kế hoạch săn sinh ra (Log4Shell, không dùng PEAK).....80

DANH MỤC HÌNH

Hình 1. *Ba pha PEAK và thành phần hiện thực tương ứng trong hệ thống* 6
Hình 2. *Kiến trúc tổng thể của hệ thống* 16
Hình 3. *Luồng xử lý một PoC* 17
Hình 4. *Xử lý lỗi của lớp LLM*.....19
Hình 5. *Cây quyết định của luật khuyến nghị* 21
Hình 6. *Quy trình Prepare-only: từ PoC công khai đến kế hoạch săn, đội Execute chạy, xác minh và vòng pivot* 25

DANH MỤC BẢNG

Bảng 1. *Ba loại săn trong PEAK và phạm vi của đồ án* 6
Bảng 2. *Bốn thành phần của ABLE* 7
Bảng 3. *Từ cơ sở lý thuyết đến quyết định thiết kế* 9
Bảng 4. *Yêu cầu chức năng* 12
Bảng 5. *Yêu cầu phi chức năng* 12
Bảng 6. *So sánh các phương án tích hợp* 13
Bảng 7. *Yêu cầu của luồng Prepare-only* 14
Bảng 8. *Các trường của Recommendation* 18
Bảng 9. *Luật xác định khuyến nghị và độ tin cậy* 21
Bảng 10. *Các tệp đầu ra của một PoC* 24
Bảng 11. *Các trường chính của* `HuntPlan` 25
Bảng 12. *Thứ tự quyết định của* `verify` 27
Bảng 13. *Ai tin ai trong luồng Prepare-only* 28
Bảng 14. *Công nghệ sử dụng* 30
Bảng 15. *Cấu trúc mã nguồn* 30
Bảng 16. *Các chế độ chạy theo cấu hình LLM* 40
Bảng 17. *Tham chiếu nhanh tuỳ chọn dòng lệnh* 42
Bảng 18. *Từ khuyến nghị đến hành động của người săn* 42
Bảng 19. *Lỗi thường gặp khi viết PoC* 44
Bảng 20. *Sự cố thường gặp và cách xử lý* 44
Bảng 21. *Từ quyết định của* `verify` *đến việc cần làm* 46
Bảng 22. *Thành phần của CSDL đánh giá* 47
Bảng 23. *Bốn PoC thực nghiệm* 47
Bảng 24. *Kết quả bốn PoC với mô hình* `auto` 48
Bảng 25. *Phạm vi truy vấn và độ phủ của ba PoC rỗng (cửa sổ 21/8)* 49
Bảng 26. *Dấu hiệu cần tìm có trong dữ liệu không (toàn CSDL 4.417.543 dòng)* 50
Bảng 27. *Cùng bốn PoC dưới ba cấu hình* 51
Bảng 28. *Số lời gọi và token của judge/advisor* 51
Bảng 29. *Lỗi phát hiện khi chạy thật và cách xử lý* 51
Bảng 30. *Kết quả thử một lần các mô hình miễn phí* 53
Bảng 31. *Hai lần chạy Joomla với cache Prepare và che dữ liệu* 54
Bảng 32. *Đối chiếu yêu cầu và kết quả* 55
Bảng 33. *Các lần lập kế hoạch từ PoC công khai* 57
Bảng 34. *So sánh ba cách tiếp cận* 61
Bảng 35. *Các trường của tệp PoC* 73
Bảng 36. *Biến môi trường* 75
Bảng 37. *Các khoá cấp cao của* `recommendation.json` 76
Bảng 38. *Phân bố 146 bài kiểm thử theo tệp* 77

Bảng 39. *Tổng hợp các lần chạy cuối (sau khi sửa phạm vi truy vấn) đã dùng trong luận văn* 78
Bảng 40. *Thuật ngữ và từ viết tắt* 79

Chương 1. Mở đầu

1.1. Đặt vấn đề

Các hệ thống phát hiện xâm nhập dựa trên luật và chữ ký chỉ bắt được những gì đã được mô tả trước. Kẻ tấn công có kỹ năng thường đi vòng qua các luật đó bằng công cụ hợp pháp có sẵn trên hệ thống, bằng mã hoá lệnh, hoặc bằng hành vi trông giống quản trị thông thường. Săn mối đe dọa (threat hunting) bổ sung cho phát hiện tự động bằng cách đặt một giả thuyết về hoạt động của kẻ tấn công rồi chủ động dùng dữ liệu để xác nhận hoặc bác bỏ giả thuyết đó. Hoạt động này phụ thuộc nhiều vào kinh nghiệm của người săn và tốn thời gian ở giai đoạn chuẩn bị: đọc tài liệu về kỹ thuật tấn công, viết giả thuyết kiểm chứng được, xác định nguồn dữ liệu cần dùng, rồi dịch tất cả thành các truy vấn cụ thể.

Khung PEAK (Prepare, Execute, Act with Knowledge) của Splunk SURGe mô tả quy trình đó một cách có cấu trúc [1]. Giai đoạn Prepare của PEAK, gồm chọn chủ đề, nghiên cứu, phát biểu giả thuyết, lập bảng ABLE (Actor, Behavior, Location, Evidence), xác định phạm vi và lập kế hoạch, là phần mà các mô hình ngôn ngữ lớn (LLM) có thể hỗ trợ tốt vì nó chủ yếu là đọc, tổng hợp và viết. Cisco Talos đã công bố mã nguồn mở **PEAK Assistant** [5], một tập tác tử LLM thực hiện đúng các việc đó và xuất ra báo cáo nghiên cứu, bảng ABLE và kế hoạch săn. Chính nhóm tác giả cũng nói rõ đây là bản chứng minh khái niệm, chưa qua kiểm thử bảo mật, nên chỉ phù hợp chạy cục bộ.

Việc dùng LLM trong săn mối đe dọa đặt ra một vấn đề niềm tin. LLM có thể viết ra những khẳng định nghe hợp lý nhưng không có căn cứ trong dữ liệu [12]. Nếu để LLM vừa lập kế hoạch vừa kết luận, người săn khó phân biệt đâu là điều dữ liệu chứng minh, đâu là điều mô hình suy đoán. Một vấn đề thứ hai tinh tế hơn: khi truy vấn không trả về gì, người dùng và cả mô hình đều dễ diễn giải là "không có tấn công", trong khi nguyên nhân có thể là dữ liệu không được thu thập, cửa sổ thời gian quá hẹp, hoặc vị từ viết sai. Nguyên tắc "không có bằng chứng không đồng nghĩa với bằng chứng không có" [13] vì vậy cần được hiện thực thành quy tắc của hệ thống chứ không chỉ là lời nhắc.

1.2. Bối cảnh và động cơ thực hiện

Kho mã `AI_Agent_Hunting` ban đầu (phiên bản v6) là một engine điều tra tổng quát xây quanh ba đồ thị: đồ thị khẳng định (ClaimGraph), đồ thị năng lực nhà cung cấp (CapabilityGraph) và đồ thị bằng chứng (EvidenceGraph). Engine có hơn ba mươi sáu nghìn dòng mã nguồn, hàng chục mô-đun và khoảng tám mươi tệp kiểm thử, giải bài toán trả lời câu hỏi điều tra tự do. Qua thực tế sử dụng, phần có giá trị nhất của kho lại là đường chạy PoC: nhận một giả thuyết đã cụ thể hoá, chạy các vị từ trên telemetry, rồi báo cáo. Phần engine lớn gây ba khó khăn: khó đọc và khó bảo trì, khó

giải thích cho người dùng cuối, và trùng lặp với phần Prepare mà PEAK Assistant đã làm tốt hơn.

Động cơ của đồ án là tái cấu trúc kho theo hướng: (i) giữ phần thực thi tất định đã được kiểm chứng; (ii) thay phần chuẩn bị tự xây bằng PEAK Assistant; (iii) bổ sung phần còn thiếu nhất là đầu ra có ích cho con người, tức một khuyến nghị có lý do để người săn ra quyết định; (iv) làm gọn kho để người mới đọc được trong vài giờ.

1.3. Mục tiêu nghiên cứu

Mục tiêu tổng quát là xây dựng một trợ lý săn mối đe dọa dùng PEAK Assistant cho pha Prepare, thực thi tất định cho pha Execute, và đưa ra khuyến nghị hỗ trợ quyết định cho pha Act, kiểm chứng trên dữ liệu thật.

Các mục tiêu cụ thể:

- Phân tích và làm gọn kho mã v6, giữ lại đường chạy PoC cùng các adapter telemetry.
- Tích hợp PEAK Assistant qua một lớp cầu nối có khả năng chịu lỗi, dùng chung một cấu hình LLM.
- Thiết kế bộ luật khuyến nghị tất định, có giải thích, không để LLM thay đổi sự thật về bằng chứng.
- Bảo đảm LLM là tuỳ chọn: không cấu hình LLM thì hệ thống vẫn chạy.
- Chạy bốn PoC có sẵn trên bộ dữ liệu Boss of the SOC v1, ghi nhận kết quả, lỗi và hạn chế một cách trung thực.
- Viết tài liệu hướng dẫn thực hành đủ để một người mới cài đặt, chạy, đọc kết quả và tự viết PoC.
- Chuyển trọng tâm sang pha Prepare thuần: từ PoC công khai (NVD, GitHub) sinh kế hoạch săn đã qua kiểm tra an toàn, bàn giao cho đội thực thi và xác minh kết quả họ trả về, không chạm vào hệ thống nội bộ.

1.4. Đối tượng và phạm vi

Đối tượng nghiên cứu là quy trình săn theo giả thuyết (hypothesis-driven hunting) của PEAK, trong đó giả thuyết đã được cụ thể hoá thành PoC. Phạm vi gồm:

- **Trong phạm vi:** pha Prepare qua PEAK Assistant (ABLE, kế hoạch săn); pha Execute trên telemetry là cơ sở dữ liệu SQLite (CDB) (adapter Splunk trực tiếp được gỡ khỏi nhánh chính, mã cũ còn ở commit `4dace56`); pha Act dạng khuyến nghị, bản nháp truy vấn SPL, backlog và ghi chú cho các bên liên quan; kiểm chứng trên Boss of the SOC v1; và luồng Prepare-only: thu thập PoC công khai, lập và kiểm tra kế hoạch săn, xác minh kết quả do đội khác trả về. Luồng này không truy cập hệ thống nội bộ và không chạy PoC.

- **Ngoài phạm vi:** săn theo đường cơ sở (baseline) và săn có hỗ trợ mô hình học máy (M-ATH) của PEAK; huấn luyện mô hình; tự động hoá hành động phản ứng sự cố; đánh giá định lượng độ chính xác trên nhiều bộ dữ liệu.
- **Giới hạn đã biết:** bộ dữ liệu có nhãn tấn công hạn chế; bản nháp SPL và các truy vấn trong kế hoạch Prepare-only chưa được chạy trên một máy chủ Splunk thật trong đồ án này; vòng xác minh mới chỉ thử bằng kết quả giả lập.

1.5. Phương pháp nghiên cứu

Đồ án theo hướng nghiên cứu thiết kế và thực nghiệm. Về lý thuyết, tổng hợp tài liệu về săn mối đe dọa, khung PEAK, MITRE ATT&CK, hệ thống đa tác tử dùng LLM và độ tin cậy của LLM. Về thiết kế, xác lập yêu cầu, chọn kiến trúc, và đặt ra các bất biến mà mọi thành phần phải tôn trọng. Về hiện thực, viết mã Python, kiểm thử đơn vị, và tích hợp thư viện PEAK Assistant ở một phiên bản được ghim theo mã băm commit. Về thực nghiệm, chạy hệ thống trên dữ liệu thật, thu thập đầu ra, phân tích từng kết quả và ghi lại mọi lỗi phát sinh. Mọi số liệu trong luận văn lấy từ kết quả chạy lưu trong thư mục `results/` của kho.

1.6. Đóng góp chính

1. Một kiến trúc ghép PEAK Assistant (không tất định) với bộ thực thi tất định, trong đó ranh giới tin cậy được ghi rõ và được kiểm thử.
2. Bộ luật khuyến nghị có giải thích, xử lý riêng bốn trường hợp dễ nhầm: chuỗi khớp một phần, kết quả khớp nhưng chưa chứng minh thành công, kết quả rỗng do thiếu dữ liệu, và kết quả rỗng do phạm vi tìm kiếm (host) không có dữ liệu.
3. Cơ chế vận hành LLM thực tế: gọi lại có backoff, quay về chế độ không LLM khi lỗi, chấp nhận nhiều dạng đầu ra JSON, và chế độ hoàn toàn không cần LLM.
4. Làm gọn kho từ khoảng 36.400 dòng xuống khoảng 9.400 dòng mã nguồn (gồm khoảng 2.300 dòng của luồng Prepare-only, sau khi gỡ adapter Splunk khoảng 2.300 dòng) mà vẫn giữ khả năng khôi phục engine cũ.
5. Bộ kết quả thực nghiệm và hướng dẫn thực hành có thể tái chạy.
6. Luồng Prepare-only: biến PoC công khai thành `HuntPlan` có schema, trong đó mọi truy vấn do LLM viết bị duyệt tĩnh trước khi rời dự án, dấu vết được đối chiếu với văn bản PoC để không để LLM bịa, và điểm dừng, giới hạn, truy vấn độ phủ do mã sinh chứ không do LLM.
7. Hai quy tắc kiểm tra mới cho phần LLM viết: truy vấn hậu khai thác trên dữ liệu tiến trình phải lọc theo tiến trình cha, và bộ phát hiện từ lẫn ngôn ngữ khác (không cần dữ liệu ngoài) cho chữ tiếng Việt.
8. Bộ xác minh `verify` tất định, trả sáu quyết định và sinh vòng pivot an toàn, trong đó kết quả rỗng không bao giờ được gọi là "sạch".

1.7. Bố cục luận văn

Chương 2 trình bày cơ sở lý thuyết. Chương 3 phân tích bài toán và yêu cầu. Chương 4 mô tả thiết kế kiến trúc và bộ luật khuyến nghị. Chương 5 mô tả cài đặt. Chương 6 là hướng dẫn thực hành. Chương 7 trình bày thực nghiệm và đánh giá. Chương 8 thảo luận hạn chế. Chương 9 kết luận và đề xuất hướng phát triển. Hướng Prepare-only được trình bày ở các mục 3.7, 4.9, 5.10, 6.10, 7.10 và 8.7. Phần phụ lục cung cấp tệp PoC mẫu, báo cáo mẫu, tham chiếu dòng lệnh, cấu hình, danh sách kiểm thử và bảng thuật ngữ.

Chương 2. Cơ sở lý thuyết

Chương này trình bày các khái niệm nền tảng mà hệ thống dựa vào. Mỗi mục kết thúc bằng phần "áp dụng vào đồ án" để người đọc thấy khái niệm đó ảnh hưởng thế nào tới thiết kế ở các chương sau.

2.1. Săn mối đe dọa

2.1.1. Định nghĩa và vị trí trong vận hành an ninh

Săn mối đe dọa là hoạt động chủ động, do con người dẫn dắt và có máy hỗ trợ, nhằm tìm ra những hoạt động độc hại đã lọt qua các lớp phát hiện tự động. Điểm xuất phát là giả định đã bị xâm nhập (assume breach): người săn không đợi cảnh báo mà đặt câu hỏi "nếu kẻ tấn công đã ở trong mạng thì dấu vết của chúng trông như thế nào, và dữ liệu nào có thể chứa dấu vết đó". Kết quả của một cuộc săn vì thế không chỉ là nhị phân "có/không tấn công". Nó có thể là: giả thuyết được xác nhận, giả thuyết bị bác bỏ trong phạm vi dữ liệu đã xem, hoặc không kết luận được do dữ liệu không đủ. Một cuộc săn dù không tìm thấy kẻ tấn công vẫn tạo giá trị: phát hiện khoảng trống dữ liệu, sinh ra luật phát hiện mới, hoặc củng cố hiểu biết về môi trường [1].

Trong chu trình xử lý sự cố của NIST [16], săn mối đe dọa nằm ở phần phát hiện và phân tích, và khi tìm thấy dấu hiệu thật thì chuyển giao cho nhóm ứng cứu sự cố (IR). Điều này giải thích vì sao đầu ra cuối cùng của hệ thống trong đồ án là một khuyến nghị về việc có chuyển giao cho IR hay không, chứ không phải một bản án.

2.1.2. Mô hình trưởng thành của hoạt động săn

Bianco đề xuất Hunting Maturity Model gồm năm mức từ HMM0 đến HMM4: ban đầu (chỉ dựa vào cảnh báo tự động), tối thiểu (có tra cứu chỉ báo), theo quy trình (dùng quy trình do người khác viết), đổi mới (tự tạo quy trình mới) và dẫn đầu (tự động hoá quy trình hiệu quả thành phát hiện) [8]. Mô hình cho thấy một hướng phát triển tự nhiên: các cuộc săn thủ công thành công nên được chuyển thành luật phát hiện tự động. Trong khung PEAK, bước đó gắn với pha Act.

2.1.3. Kim tự tháp đau đớn và hành vi

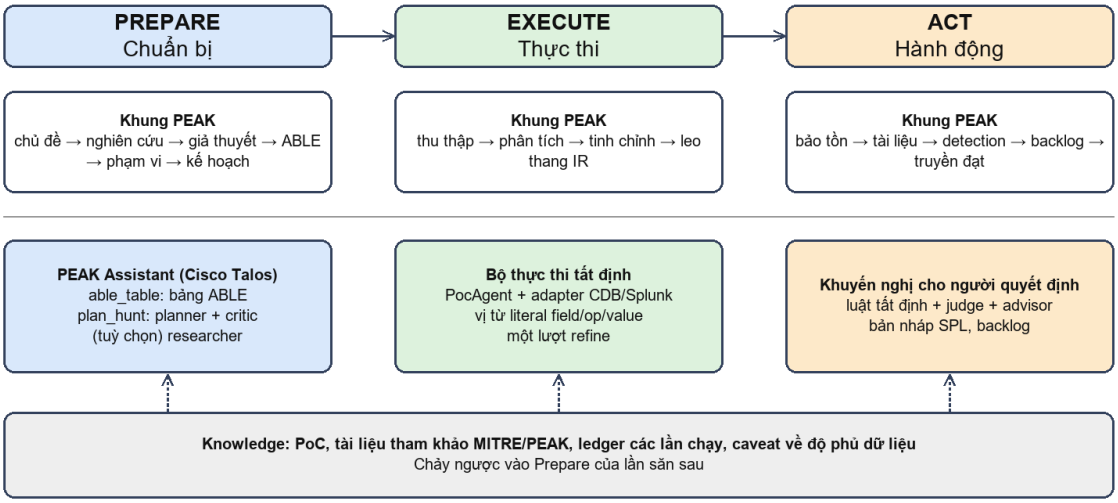
Kim tự tháp đau đớn (Pyramid of Pain) xếp các loại chỉ báo theo mức khó khăn mà kẻ tấn công phải chịu khi bị buộc đổi chúng: giá trị băm, địa chỉ IP, tên miền, dấu vết mạng và máy, công cụ, và cuối cùng là chiến thuật, kỹ thuật, quy trình (TTP) [7]. Săn theo hành vi nằm ở đỉnh tháp vì kẻ tấn công khó đổi hành vi hơn là đổi một địa chỉ IP. Đồ án thể hiện điều này trong các PoC: PoC khai thác Joomla và PoC mã hoá PowerShell là chỉ báo hành vi, còn PoC beacon tới `ad.networkfilter.co` là chỉ báo ở mức tên miền, vốn dễ bị thay đổi.

2.2. Khung PEAK

2.2.1. Ba pha và yếu tố tri thức

PEAK do nhóm Splunk SURGe (trong đó có David Bianco, người sau này cũng là tác giả chính của PEAK Assistant) giới thiệu năm 2023 [1]. Tên gọi là chữ viết tắt của ba pha cộng một yếu tố xuyên suốt:

- **Prepare:** chọn chủ đề, nghiên cứu, phát biểu giả thuyết, xác định phạm vi và dữ liệu cần dùng, lập kế hoạch.
- **Execute:** thu thập dữ liệu, xử lý sơ bộ, phân tích, tinh chỉnh giả thuyết, và leo thang khi có phát hiện nghiêm trọng.
- **Act:** bảo tồn kết quả, ghi tài liệu, tạo luật phát hiện, đưa việc còn lại vào backlog và truyền đạt cho các bên liên quan.
- **Knowledge:** threat intelligence, ngữ cảnh của tổ chức, kinh nghiệm người săn và chính kết quả các cuộc săn trước, chảy vào cả ba pha.



Hình 1. *Ba pha PEAK và thành phần hiện thực tương ứng trong hệ thống*

PEAK là quy trình dành cho nhà phân tích con người chứ không phải đặc tả phần mềm. Vì vậy, hệ thống trong đồ án không tự nhận là "hiện thực PEAK được chứng nhận". Nó chỉ hiện thực hoá ba pha theo hướng hỗ trợ: PEAK Assistant làm Prepare, bộ thực thi tất định làm Execute, và mô-đun khuyến nghị làm Act.

2.2.2. Ba loại săn của PEAK

PEAK định nghĩa ba loại săn, cùng đi qua ba pha trên [2][3][4].

Bảng 1. *Ba loại săn trong PEAK và phạm vi của đồ án*

Loại	Mục đích	Cách làm	Trong đồ án
Hypothesis-driven	Kiểm chứng một giả	Chọn chủ đề, làm cho	**Có** (đối tượng chính)

Loại	Mục đích	Cách làm	Trong đồ án
	thuyết về hành vi kẻ tấn công	kiểm chứng được, tinh chỉnh cho đủ cụ thể, rồi truy vấn dữ liệu	
Baseline	Mô tả "bình thường" để nhìn ra độ lệch	Từ điển dữ liệu, phân bố, ngoại lệ, khoảng trống; khuyến nghị cửa sổ 30–90 ngày	Không (ngoài phạm vi)
M-ATH (Model-Assisted)	Dùng thuật toán tìm đầu mối khi cách đơn giản không đủ	Phân loại, phân cụm, phát hiện bất thường, có người trong vòng lặp	Không (ngoài phạm vi)

2.2.3. Mô hình ABLE

ABLE là cách PEAK biến một giả thuyết thành kế hoạch hành động. Mỗi chữ cái là một câu hỏi người săn phải trả lời:

Bảng 2. *Bốn thành phần của ABLE*

Thành phần	Câu hỏi	Ví dụ (rò rỉ dữ liệu qua DNS)
Actor	Ai thực hiện (được phép để trống)	Chưa rõ nhóm cụ thể
Behavior	Hành vi cụ thể, nên chỉ 1–2 kỹ thuật	DNS tunneling
Location	Hành vi xảy ra ở phần nào của mạng	Máy trạm phòng tài chính và biên mạng
Evidence	Cần nguồn dữ liệu nào, và dấu hiệu trúng trông ra sao	Nhật ký DNS: truy vấn dài, loại bản ghi lạ

ABLE đóng vai trò cầu nối giữa phát biểu giả thuyết bằng văn xuôi và truy vấn cụ thể. PEAK Assistant có một tác tử riêng sinh bảng ABLE từ giả thuyết, tài liệu nghiên cứu và mô tả dữ liệu cục bộ [5]. Trong đồ án, bảng ABLE do PEAK sinh ra là thông tin tham khảo cho người săn, còn các vị từ thực thi do PoC khai báo.

2.3. MITRE ATT&CK và chỉ báo thực thi

MITRE ATT&CK là cơ sở tri thức về chiến thuật và kỹ thuật của kẻ tấn công dựa trên quan sát thực tế [6]. Mỗi kỹ thuật có mã định danh, ví dụ T1190 (khai thác ứng dụng hướng ra Internet), T1110 (dò mật khẩu), T1059.001 (PowerShell), T1071.001 (giao thức tầng ứng dụng: web). Mã này giúp người săn tìm tài liệu nghiên cứu và đặt tên kết quả theo ngôn ngữ chung. Trong các PoC của đồ án, mỗi PoC gắn một kỹ thuật ATT&CK trong trường `references` và `able.behavior`.

2.4. Mô hình ngôn ngữ lớn và hệ thống đa tác tử

2.4.1. LLM như một thành phần tạo văn bản

LLM sinh văn bản bằng cách dự đoán chuỗi token tiếp theo. Chúng mạnh ở các việc như tóm tắt tài liệu, diễn đạt lại, chuyển tài liệu thành cấu trúc, và gợi ý bước tiếp theo. Chúng yếu ở những việc đòi hỏi tính đúng đắn tuyệt đối trên dữ liệu cụ thể, vì không có cơ chế nội tại để đối chiếu mỗi câu với nguồn. Hiện tượng sinh ra thông tin nghe hợp lý nhưng sai hoặc không có căn cứ được gọi là ảo giác (hallucination) [12]. Truy hồi tăng cường (retrieval-augmented generation) giảm rủi ro này bằng cách đưa tài liệu nguồn vào ngữ cảnh [14]; tuy nhiên nó không loại bỏ hoàn toàn, vì mô hình vẫn có thể diễn giải sai tài liệu được cung cấp.

2.4.2. Tác tử và đa tác tử

Một tác tử LLM là một vòng lặp trong đó mô hình quyết định hành động tiếp theo, quan sát kết quả và tiếp tục; ReAct là một mô hình điển hình kết hợp suy luận và hành động [10]. Hệ thống đa tác tử cho nhiều tác tử với vai trò khác nhau trao đổi với nhau. AutoGen là một khung phổ biến cho kiểu này, hỗ trợ các nhóm tác tử trò chuyện theo lượt và điều kiện dừng [9]. PEAK Assistant dùng AutoGen: nhóm lập kế hoạch có một tác tử viết kế hoạch và một tác tử phê bình, luân phiên cho tới khi tác tử phê bình trả về tín hiệu kết thúc [5]. Mô hình "người viết và người phản biện" này thường cải thiện chất lượng văn bản, nhưng cũng làm số lượt gọi LLM không cố định trước, và khó kiểm soát chi phí.

2.4.3. LLM làm giám khảo

Dùng LLM để chấm hoặc đánh giá đầu ra (LLM-as-a-judge) được nghiên cứu rộng rãi và cho thấy mức đồng thuận với người đánh giá khá cao trên một số tác vụ, nhưng cũng có thiên lệch và không ổn định giữa các lần chạy [11]. Vì vậy trong đồ án, "judge" chỉ được dùng như ý kiến tham khảo thứ hai: nó nhận hàng bằng chứng đã được bộ thực thi lấy ra và trả lời một trong bốn nhãn (TRUE_POSITIVE, FALSE_POSITIVE, INCONCLUSIVE, NO_SIGNAL) kèm độ tin cậy. Luật khuyến nghị tất định luôn là bên quyết định, và judge chỉ có thể đẩy kết quả đi tối đa một bậc.

2.5. Độ tin cậy của bằng chứng và bằng chứng âm tính

2.5.1. Vắng bằng chứng không phải bằng chứng vắng mặt

Altman và Bland nhấn mạnh rằng một nghiên cứu không tìm thấy hiệu ứng chưa chứng minh hiệu ứng không tồn tại [13]. Trong săn mối đe dọa, điều tương đương là một truy vấn rỗng. Có ít nhất bốn nguyên nhân khiến truy vấn rỗng mà vẫn có tấn công: nguồn dữ liệu không được thu thập; nguồn có nhưng không chứa loại sự kiện cần tìm; cửa sổ thời gian không bao trùm hoạt động; và vị từ literal bỏ sót biến thể. Hệ

thống trong đồ án không xử lý hết được cả bốn, nhưng hiện thực hoá ba (độ phủ nguồn trong cửa sổ; độ phủ trong phạm vi host kèm chạy lại không lọc host; giới hạn cửa sổ) và nêu rõ, kèm liệt kê loại sự kiện thực có, điểm còn lại trong phần "giới hạn của kết luận" của mỗi báo cáo.

2.5.2. Tách bạch bằng chứng và diễn giải

Nguyên tắc thiết kế chung của đồ án là hai lớp. Lớp bằng chứng chỉ gồm các bản ghi do adapter trả về từ telemetry, có thể tái lập. Lớp diễn giải gồm mọi văn bản do LLM sinh ra (ABLE, kế hoạch, judge, advisor), luôn được gắn nhãn là tham khảo. Bản ghi không bao giờ đi từ lớp diễn giải sang lớp bằng chứng. Nguyên tắc này đối ứng với khuyến nghị của OWASP về ứng dụng dùng LLM: không tin tưởng đầu ra của mô hình như dữ liệu đã xác thực và không giao cho mô hình quyền thực thi không kiểm soát [17].

2.6. Hỗ trợ ra quyết định

Hệ thống hỗ trợ ra quyết định khác hệ thống ra quyết định tự động ở chỗ nó đưa phương án, lý do và rủi ro, còn quyền chọn thuộc về người dùng. Một khuyến nghị tốt cho người săn cần: nêu rõ kết luận và mức độ tin cậy; liệt kê các lựa chọn theo thứ tự ưu tiên; chỉ ra bằng chứng nào ủng hộ và điều gì còn chưa biết; nêu hậu quả của việc chọn sai. Đồ án chuyển các yêu cầu này thành cấu trúc dữ liệu `Recommendation` với các trường `disposition`, `confidence`, `reasons`, `caveats`, `options`, `next_steps`, `questions_for_hunter`, `risks` và cờ `decision_required` luôn bằng đúng.

2.7. Telemetry và bộ dữ liệu Boss of the SOC v1

Telemetry là dữ liệu quan sát được từ hệ thống: tạo tiến trình (sự kiện 4688 của Windows hoặc sự kiện 1 của Sysmon), xác thực (4624 đăng nhập thành công, 4625 đăng nhập thất bại), truy cập chia sẻ SMB (5140, 5145), truy vấn DNS và yêu cầu web. Mỗi nguồn trả lời được một lớp câu hỏi, và không nguồn nào trả lời được tất cả. Ví dụ, yêu cầu web chứa trong dữ liệu của đồ án chỉ có tên miền và đường dẫn, không có mã trạng thái, nên không đủ để chứng minh một cuộc khai thác thành công.

Boss of the SOC (BOTS) v1 là bộ dữ liệu do Splunk công bố cho các cuộc thi và huấn luyện an ninh, chứa nhật ký từ một môi trường mô phỏng bị tấn công trong tháng 8 năm 2016, bao gồm cả giai đoạn quét và khai thác một máy chủ Joomla [15]. Trong đồ án, dữ liệu được nạp vào một bảng SQLite `events` gồm 4.417.543 dòng trải trên 28 ngày.

2.8. Tổng hợp: từ lý thuyết đến quyết định thiết kế

Bảng 3. *Từ cơ sở lý thuyết đến quyết định thiết kế*

Cơ sở lý thuyết	Hệ quả thiết kế

Cơ sở lý thuyết	Hệ quả thiết kế
PEAK có ba pha, Prepare tốn công và hợp với LLM	PEAK Assistant đảm nhiệm Prepare; Execute và Act tách riêng
ABLE nối giả thuyết với truy vấn	Bảng ABLE do PEAK sinh là đầu vào cho kế hoạch và cho advisor
LLM có thể ảo giác [12]	LLM không bao giờ tạo hoặc sửa bản ghi bằng chứng
LLM-as-judge không ổn định [11]	Judge chỉ tham khảo, chỉ đẩy khuyến nghị tối đa một bậc
Vắng bằng chứng không phải bằng chứng vắng [13]	Kiểm tra độ phủ nguồn; rỗng + thiếu nguồn thành `COLLECT_DATA_THEN_RERUN`
Khuyến nghị cần lý do và rủi ro	Cấu trúc `Recommendation` đầy đủ và cờ `decision_required`
Đa tác tử khó dự đoán số lượt gọi [9]	Giới hạn thời gian từng bước, gọi lại có backoff, quay về kế hoạch của PoC

Chương 3. Phân tích bài toán và yêu cầu hệ thống

3.1. Hiện trạng kho mã v6

Trước khi tái cấu trúc, kho `AI_Agent_Hunting` có hai đường chạy độc lập. Đường thứ nhất là engine giả thuyết tự do: người dùng gõ một câu hỏi hoặc một mã CVE/TTP, một LLM biên dịch thành đồ thị khẳng định, engine chọn nhà cung cấp dữ liệu qua đồ thị năng lực, thực thi truy vấn, xác minh quan hệ và dựng đồ thị bằng chứng. Đường thứ hai là đường PoC: người dùng viết một PoC dạng JSON gồm vài vị từ cụ thể, tác tử PoC chạy từng vị từ qua adapter, rồi một bước judge tuỳ chọn đánh giá kết quả.

Kiểm kê cho thấy các điểm sau:

- Tổng mã nguồn `src/` khoảng 36.400 dòng; riêng tệp `engine.py` hơn 3.300 dòng, `cli.py` hơn 1.700 dòng.
- Đường PoC chỉ phụ thuộc vào khoảng 7.200 dòng (gồm hai adapter, các hợp đồng dữ liệu mà adapter cần, và mô-đun Act). Phần còn lại phục vụ riêng engine giả thuyết tự do.
- Phần "Prepare" của PEAK đã được tự xây (`peak.py`: cổng kiểm tra trường bắt buộc, trình hướng dẫn nhập, đọc kế hoạch YAML). Nó kiểm tra việc *có* điền đủ trường, nhưng không giúp người dùng *viết* ra nội dung tốt.
- Kết quả cuối của đường PoC là một nhãn (MATCHED/EMPTY) kèm một nhận định judge. Người săn phải tự suy ra nên làm gì tiếp.

3.2. Vấn đề cần giải quyết

Từ hiện trạng, đồ án tập trung vào bốn vấn đề.

Vấn đề 1: độ phức tạp vượt nhu cầu. Phần lớn người dùng cần chạy một giả thuyết đã cụ thể hoá và nhận kết quả dễ hiểu, không cần một engine suy diễn tổng quát. Độ phức tạp thừa làm tăng chi phí bảo trì và rào cản học.

Vấn đề 2: chuẩn bị săn đang là việc thủ công. Viết bảng ABLE, xác định nguồn dữ liệu và lập kế hoạch cần đọc tài liệu và viết nhiều văn bản. PEAK Assistant tự động hoá chính các việc này, nên việc tiếp tục tự xây là lãng phí.

Vấn đề 3: đầu ra chưa phục vụ quyết định. "MATCHED, 200 bản ghi" chưa cho người săn biết có nên chuyển cho IR hay không, độ tin cậy ra sao, và còn thiếu gì. Một cuộc săn rỗng còn tệ hơn: nhãn "EMPTY" dễ bị đọc nhầm thành "an toàn".

Vấn đề 4: ràng buộc vận hành LLM. Endpoint LLM có thể không sẵn sàng, bị giới hạn tốc độ, trả lỗi thoáng qua hoặc tốn phí. Người dùng có thể chưa có khoá API. Hệ thống không được sập hay trả kết quả sai trong những tình huống đó.

3.3. Yêu cầu chức năng

Bảng 4. *Yêu cầu chức năng*

Mã	Yêu cầu	Căn cứ
FR1	Nạp PoC từ tệp JSON, kiểm tra các trường Prepare bắt buộc	Vấn đề 2
FR2	Gọi PEAK Assistant sinh bảng ABLE và kế hoạch săn cho mỗi PoC	Vấn đề 2
FR3	Thực thi các vị từ literal trên telemetry qua adapter (CDB)	Giữ giá trị đường PoC
FR4	Kiểm tra độ phủ của từng nguồn dữ liệu trong cửa sổ thời gian	Vấn đề 3
FR5	Sinh khuyến nghị xếp hạng với lý do, giới hạn, câu hỏi, rủi ro	Vấn đề 3
FR6	Chạy được nhiều PoC một lần và tổng hợp bảng kết quả	Vấn đề 1
FR7	Xuất báo cáo Markdown và JSON, bản nháp SPL, backlog	Pha Act của PEAK
FR8	Chạy được hoàn toàn không có LLM	Vấn đề 4
FR9	Chọn được mô hình LLM từ dòng lệnh	Vấn đề 4

3.4. Yêu cầu phi chức năng

Bảng 5. *Yêu cầu phi chức năng*

Mã	Yêu cầu	Cách kiểm chứng
NFR1	LLM không tạo hoặc sửa bản ghi bằng chứng	Mọi hàng đều đến từ adapter; có kiểm thử
NFR2	Kết quả phần tất định tái lập được	Cùng PoC và cùng dữ liệu cho cùng số bản ghi
NFR3	Lỗi LLM không làm sập pipeline	Kiểm thử mô phỏng lỗi; chạy thật gặp lỗi 404
NFR4	Không ghi khoá API ra đĩa hay báo cáo	Cấu hình PEAK chỉ chứa `${ENV}`; quét kết quả không thấy khoá
NFR5	Hệ thống nhỏ, đọc được	Mã nguồn dưới 10.000 dòng
NFR6	Khôi phục được engine cũ	Gắn thẻ git `v6-engine-final`
NFR7	Python 3.12 trở lên	Yêu cầu của PEAK Assistant

3.5. Các phương án thiết kế

Ba phương án được cân nhắc cho việc dùng PEAK Assistant.

Phương án A: dùng PEAK Assistant nguyên bản qua giao diện Streamlit. Ưu điểm là không viết mã. Nhược điểm: hoàn toàn thủ công, không chạy được hàng

loạt, không có bước thực thi trên telemetry của kho, không có khuyến nghị cuối. Phương án này chỉ cho phần Prepare.

Phương án B: gọi PEAK Assistant qua máy chủ MCP của nó (`peak-mcp`). Hợp với việc để một tác tử LLM khác điều phối. Nhược điểm: thêm một tầng giao thức và một tiến trình máy chủ, khó kiểm thử và khó chịu lỗi; điều phối bởi LLM đi ngược với yêu cầu phần Execute tất định.

Phương án C (được chọn): dùng PEAK Assistant như thư viện Python. Các hàm `able_table`, `plan_hunt` (và tuỳ chọn `researcher`) là hàm bất đồng bộ nhận chuỗi và trả chuỗi, nên đóng gói thành một lớp cầu nối mỏng trong tiến trình của hệ thống. Ưu điểm: tích hợp chặt, dễ chịu lỗi, dễ kiểm thử bằng cách giả lập lớp cầu nối. Nhược điểm: phụ thuộc vào API nội bộ của một dự án chứng minh khái niệm, nên cần ghim phiên bản theo mã băm commit (`dfabbb0`).

Bảng 6. *So sánh các phương án tích hợp*

Tiêu chí	A: Streamlit	B: MCP	C: thư viện
Chạy hàng loạt	Không	Có	Có
Thực thi tất định tách khỏi LLM	Không áp dụng	Khó	Dễ
Kiểm thử tự động	Khó	Khó	Dễ
Độ phức tạp thêm	Thấp	Cao	Trung bình
Rủi ro thay đổi API	Thấp	Trung bình	Cao (giảm bằng ghim phiên bản)

3.6. Các bất biến thiết kế

Từ vấn đề và yêu cầu, năm bất biến được đặt ra và áp dụng cho mọi thành phần mới.

1. **LLM không tạo bằng chứng.** Mọi bản ghi bằng chứng đến từ adapter. Văn bản của LLM chỉ xuất hiện ở các mục được gắn nhãn tham khảo.
2. **Khớp là literal và tái lập.** Cùng PoC, cùng dữ liệu cho cùng kết quả.
3. **Vắng dữ liệu không thành "sạch".** Nguồn có 0 bản ghi trong cửa sổ cho `COLLECT_DATA_THEN_RERUN`; kết quả rỗng khi nguồn có dữ liệu vẫn kèm cảnh báo về giới hạn.
4. **Người quyết định cuối cùng.** Mọi khuyến nghị có `decision_required = true`; hệ thống không thực hiện hành động phản ứng.
5. **LLM là tuỳ chọn.** Mọi chức năng cốt lõi chạy được khi không có LLM; LLM chỉ làm giàu thêm.

3.7. Định hướng mới: chỉ làm pha Prepare

Sau khi pipeline PoC → Prepare → Execute → Act chạy được trên dữ liệu BOTS v1, phạm vi được thu hẹp lại theo hướng sau. Dự án chỉ đảm nhận chữ **P** của khung PEAK. Pha Execute và Act do một đội khác làm trên hệ thống của họ; dự án này **không đụng vào hệ thống nội bộ** để tránh lộ lọt thông tin. Đầu vào là PoC công khai trên Internet (CVE, kho GitHub có nhiều sao). Đầu ra là một kế hoạch dạng "giả sử PoC này được dùng để tấn công, hãy kiểm tra hệ thống có dính không": các giai đoạn tấn công, truy vấn tìm kiếm, giới hạn và điểm dừng. Khi đội kia chạy xong và trả kết quả, dự án **xác minh**: chấp nhận kết quả, hoặc chạy lại kế hoạch xuất phát từ kết quả đó để săn tiếp.

Bảng 7. *Yêu cầu của luồng Prepare-only*

Mã	Yêu cầu
PR1	Thu thập tình báo công khai chỉ bằng yêu cầu GET tới một danh sách host cố định (NVD, GitHub, raw.githubusercontent.com), có giới hạn kích thước và thời gian, không tải về hay chạy mã PoC
PR2	Trích dấu vết bằng quy tắc tất định và đối chiếu mọi dấu vết do LLM nêu với văn bản PoC/CVE (có trong PoC hay chỉ suy luận)
PR3	Kế hoạch là dữ liệu có schema: giai đoạn tấn công, dấu vết, truy vấn, giới hạn, điểm dừng, nguồn gốc
PR4	Mọi truy vấn do LLM viết phải qua kiểm tra tĩnh trước khi rời dự án; truy vấn không hợp lệ bị loại và ghi lý do
PR5	Truy vấn dùng placeholder cho chỉ mục và thời gian, không biết gì về hạ tầng nội bộ
PR6	Xác minh kết quả không dùng LLM; kết quả rỗng không bao giờ là "sạch"
PR7	Vòng săn tiếp có điều kiện dừng (chuyển IR, chấp nhận, số vòng tối đa, hết dấu vết mới)
PR8	Đọc được bằng mắt (Markdown) và bằng máy (JSON Schema); phần xác minh và schema chạy không cần LLM

Có hai phương án. Giữ nguyên pipeline cũ và thêm một nhánh đầu vào công khai sẽ buộc dự án vẫn chạm vào dữ liệu telemetry. Chọn hướng Prepare-only thì ranh giới rõ hơn: dự án chỉ nhận văn bản công khai và kết quả đã được đội kia đồng ý trả, nên bề mặt rò rỉ nhỏ hơn rất nhiều; đổi lại dự án không còn tự kiểm tra được truy vấn của mình trên dữ liệu thật. Phương án thứ hai được chọn. Pipeline cũ vẫn còn trong kho làm bộ thử cục bộ trên dữ liệu công khai.

Ba bất biến được thêm vào năm bất biến ở mục 3.6:

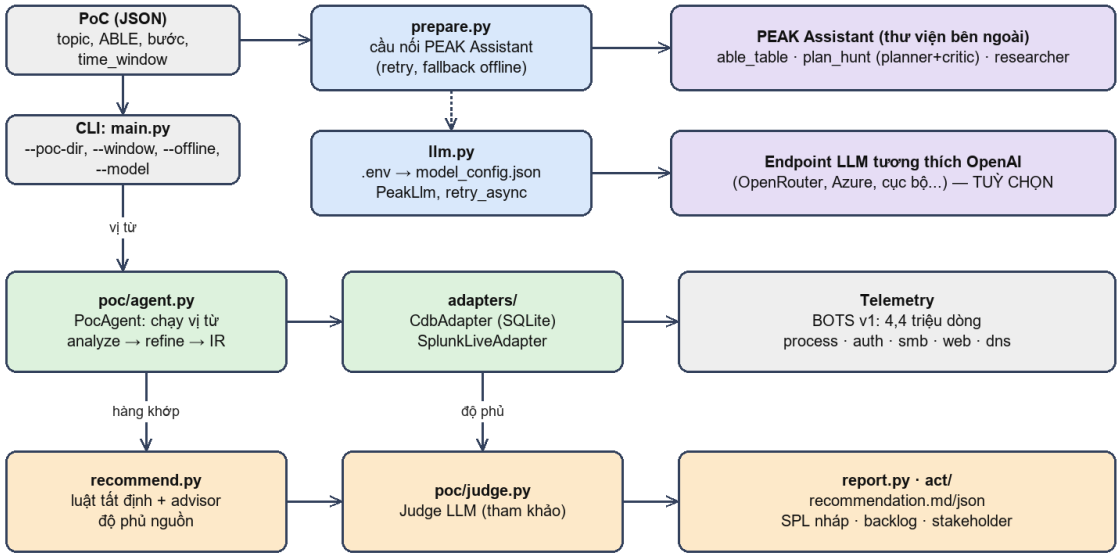
6. **Không đụng hệ thống nội bộ, không chạy PoC.** Dự án chỉ đọc văn bản công khai và kết quả do đội thực thi trả về.

7. **Nội dung Internet và đầu ra LLM đều là dữ liệu không tin cậy.** README và mã khai thác có thể chứa lệnh giả mạo nhắm vào LLM; giá trị trong log kết quả có thể do kẻ tấn công kiểm soát.
8. **Không truy vấn nào rời dự án mà chưa qua kiểm tra tĩnh.** LLM không quyết định điểm dừng, giới hạn hay truy vấn độ phủ; các thứ đó do mã sinh.

Chương 4. Thiết kế hệ thống

4.1. Tổng quan kiến trúc

Hệ thống gồm ba nhóm thành phần ứng với ba pha PEAK, cộng một lớp hạ tầng LLM dùng chung. Hình 2 trình bày các mô-đun và luồng dữ liệu chính.

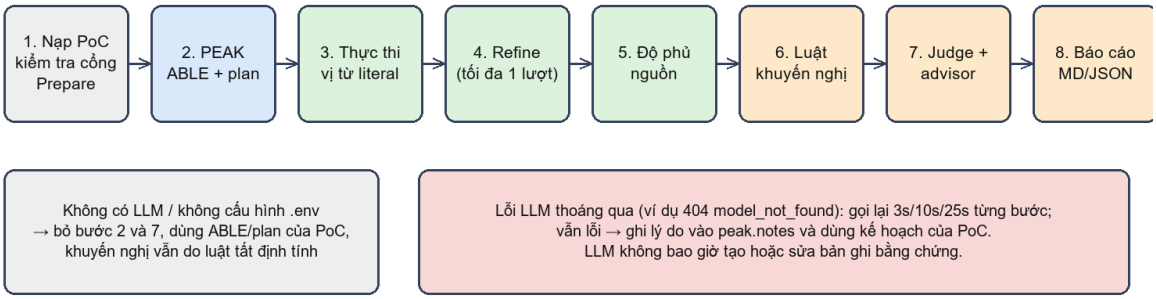


Hình 2. *Kiến trúc tổng thể của hệ thống*

- **Nhóm Prepare** (`prepare.py`, `llm.py`): gọi PEAK Assistant để sinh bảng ABLE và kế hoạch săn. Đây là nơi duy nhất mã của hệ thống gọi vào thư viện PEAK Assistant.
- **Nhóm Execute** (`poc/`, `adapters/`): nạp PoC, chạy các vị từ literal qua adapter, tinh chỉnh tối đa một lượt, ghi sổ cái (ledger). Không có LLM trong việc khớp bản ghi.
- **Nhóm Act** (`recommend.py`, `poc/judge.py`, `report.py`, `act/`): tính độ phủ nguồn, sinh khuyến nghị bằng luật tất định, hỏi judge và advisor (tuỳ chọn), xuất báo cáo, bản nháp SPL, backlog và ghi chú cho các bên liên quan.
- **Lớp LLM dùng chung** (`llm.py`): biến cấu hình trong `.env` thành cấu hình mô hình của PEAK, và cung cấp một bộ gọi đồng bộ cho judge và advisor, để mọi lần gọi đều đi qua cùng một đường và cùng một chính sách gọi lại.

4.2. Luồng xử lý một PoC

Hình 3 trình bày luồng tám bước khi chạy một PoC. Các bước 2 và 7 là các bước có LLM; khi không có LLM, chúng được bỏ qua và phần còn lại không đổi.



Hình 3. *Luồng xử lý một PoC*

1. **Nạp PoC và kiểm tra cổng Prepare.** PoC phải có chủ đề, hành vi, vị trí, bằng chứng, phạm vi, thời lượng tối đa, kế hoạch và tài liệu nghiên cứu (trường actor được để trống). Thiếu một trường thì PoC bị từ chối trước khi chạm vào telemetry.
2. **PEAK Prepare.** Dựng đầu vào cho PEAK từ PoC và từ mô tả telemetry của adapter, gọi `able_table` rồi `plan_hunt`.
3. **Thực thi vị từ.** Mỗi bước của PoC được thực hiện qua adapter bằng phép tìm văn bản thô (quét tối đa 2.000 hàng), rồi bộ lọc đúng toán tử được áp dụng lên trường đích; tối đa 100 hàng được giữ làm bằng chứng và tổng số hàng khớp được ghi lại. Ngoài vị từ, truy vấn còn chịu phạm vi suy ra từ ABLE **của PoC** (host từ `able.location`, các chuỗi cụ thể từ `able.behavior` và `able.evidence`); phạm vi này được hiển thị trong báo cáo.
4. **Tinh chỉnh.** Tối đa một lượt: nếu mọi hit đến từ đúng một máy thì chạy lại các bước còn lại trên máy đó; nếu rỗng thì chạy lại đúng các vị từ gốc. Toán tử không bao giờ bị nới (EQUALS không thành CONTAINS).
5. **Độ phủ nguồn.** Với mỗi `source_kind` của các bước, đếm số bản ghi của nguồn đó trong cửa sổ và trong phạm vi host của PoC, đồng thời liệt kê các loại sự kiện thực có. Nếu nguồn có dữ liệu nhưng không có bản ghi nào trong phạm vi host, hệ thống chạy lại PoC một lần không lọc host (thư mục `unscoped_probe/`) để phân biệt "host này không ghi nguồn" với "không có hoạt động ở đâu cả".
6. **Luật khuyến nghị.** Tính `disposition` và `confidence` từ bằng chứng, độ phủ và nhận định judge.
7. **Judge và advisor.** Judge chỉ chạy khi có bản ghi khớp. Advisor luôn chạy nếu có LLM, nhận kế hoạch PEAK và tóm tắt bằng chứng, và trả về bước tiếp theo, câu hỏi và rủi ro.
8. **Báo cáo.** Ghi `recommendation.md`, `recommendation.json`, bản nháp SPL và các tệp phụ.

4.3. Mô hình dữ liệu

4.3.1. PoC

PoC là cấu trúc bất biến gồm hai phần. Phần Prepare: `topic`, `actor`, `behavior`, `location`, `evidence`, `research_refs`, `scope`, `max_duration`, `plan`. Phần thực thi: danh sách `steps` (mỗi bước có `step_id`, `description`, `target_field`, `op`, `value`, `source_kind`), danh sách `fallbacks`, `references` và `expected_chain`. Hệ thống bổ sung `time_window` làm cửa sổ mặc định.

Sáu toán tử được hỗ trợ: `EQUALS`, `CONTAINS`, `STARTS_WITH`, `ENDS_WITH`, `MATCHES` (biểu thức chính quy) và `EXISTS`. Bốn toán tử đầu so sánh không phân biệt hoa thường; `EQUALS` còn khớp theo tên tệp cuối đường dẫn (ví dụ `C:\Windows\...\powershell.exe` khớp `powershell.exe`) nhưng không khớp `splunk-powershell.exe`.

4.3.2. Kết quả thực thi và bằng chứng

`PocHuntResult` giữ danh sách `StepResult` (số hàng và các hàng của từng bước), danh sách bước đã khớp, nhận định judge (nếu có), nhật ký tinh chỉnh và đường dẫn sổ cái. `EvidenceSummary` là bản tóm tắt phục vụ khuyến nghị: với mỗi bước có số hàng khớp và số bản ghi nguồn trong cửa sổ; ngoài ra có thời điểm đầu và cuối của hit, các giá trị xoay trục hàng đầu (máy, người dùng, IP, tên miền), giá trị phổ biến của các trường đích và tối đa năm hàng mẫu. `EvidenceSummary` còn mang phạm vi truy vấn thực tế (`scope`), số bản ghi khớp tổng và cờ bị cắt của từng bước, cơ cấu loại sự kiện của nguồn, các host thực sự ghi nguồn khi phạm vi host rỗng, và kết quả chạy lại không lọc host (`unscoped_probe`).

4.3.3. Khuyến nghị

`Recommendation` là đầu ra cuối. Các trường chính và ý nghĩa:

Bảng 8. *Các trường của Recommendation*

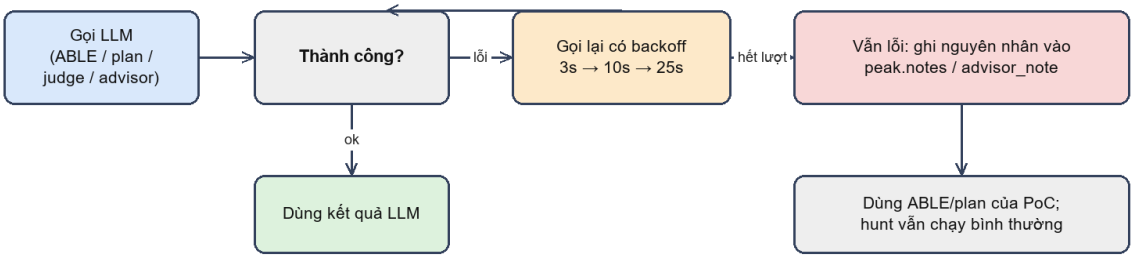
Trường	Ý nghĩa
`disposition`	Một trong năm giá trị: `ESCALATE_TO_IR`, `INVESTIGATE_FURTHER`, `TUNE_POC_OR_CLOSE`, `COLLECT_DATA_THEN_RERUN`, `CLOSE_WITH_CAVEAT`
`confidence`	`HIGH`, `MEDIUM` hoặc `LOW`
`reasons`	Các lý do tất định dẫn tới kết luận
`caveats`	Giới hạn của kết luận (độ phủ, biến thể, cửa sổ, thành công chưa chứng minh)
`options`	Các lựa chọn xếp hạng cho người quyết định, mỗi lựa chọn có việc cần làm
`judge`	Nhận định tham khảo của LLM (nếu có)
`next_steps`, `questions_for_hunter`, `risks`	Phần advisor tạo, tối đa năm mục mỗi loại
`decision_required`	Luôn bằng đúng

4.4. Lớp cầu nối PEAK Assistant

PEAK Assistant cung cấp các hàm bất đồng bộ độc lập, nên lớp cầu nối chỉ cần dựng đúng đầu vào, gọi, kiểm tra đầu ra và xử lý lỗi. Đầu vào được dựng như sau:

- **Giả thuyết:** tên PoC ghép với phần tóm tắt.
- **Tài liệu nghiên cứu:** dựng từ chủ đề, tài liệu tham khảo, tham chiếu MITRE, chuỗi quan sát kỳ vọng và các vị từ do nhà phân tích viết. Với tuỳ chọn `--research`, tác tử `researcher` của PEAK được dùng thay thế (cần máy chủ MCP nghiên cứu).
- **Tài liệu dữ liệu cục bộ và khám phá dữ liệu:** mô tả do adapter sinh ra (`describe_data`): tên bảng, các cột, các loại sự kiện, số dòng và khoảng thời gian của từng loại. Trong cài đặt gốc của PEAK, tài liệu này đến từ bước khám phá dữ liệu qua máy chủ MCP của Splunk; đồ án thay bằng mô tả trực tiếp từ adapter.
- **Ngữ cảnh cục bộ:** một đoạn văn bản cố định nói rõ rằng hunt chạy bằng bộ thực thi tất định, mỗi bước là một vị từ literal, các toán tử và quy tắc phân biệt hoa thường, LLM không tạo bằng chứng, và kế hoạch phải nói rõ chỗ nào dữ liệu hiện có không trả lời được. Ngữ cảnh này làm cho kế hoạch do PEAK viết bám vào năng lực thật của hệ thống thay vì giả định một Splunk đầy đủ.

Hình 4 mô tả chính sách lỗi. Mỗi bước (ABLE, kế hoạch) được gọi lại tối đa ba lần với khoảng chờ 3, 10 và 25 giây. Nếu cả cuộc Prepare thất bại, thử lại một lần nữa toàn bộ. Nếu vẫn thất bại, hệ thống dùng bảng ABLE và kế hoạch dựng từ chính PoC, ghi nguyên nhân vào `peak.notes` và đánh dấu `used_peak = false`. Việc săn không bị gián đoạn.



Hình 4. *Xử lý lỗi của lớp LLM*

4.5. Bộ thực thi và adapter

4.5.1. Khớp hai tầng

Adapter CDB thực hiện phép tìm văn bản thô: mỗi giá trị tìm kiếm được so khớp bằng `LIKE` trên các cột văn bản (`raw_ref`, `cmdline`, `image`, `file_path`, `domain`, `user`, `host`, `native_type`) với tham số hoá an toàn. Phép tìm này có chủ ý rộng để không bỏ

sót. Sau đó bộ thực thi áp dụng toán tử thật của bước lên đúng trường đích. Thiết kế hai tầng này sinh ra từ một lỗi thực tế: khi chỉ dùng `LIKE`, vị từ `image EQUALS powershell.exe` khớp luôn cả `splunk-powershell.exe` và tạo 100 báo động giả trên 4,38 triệu dòng; bộ lọc chính xác ở tầng hai đưa tỷ lệ này về không.

Giới hạn hàng và quét. Nếu adapter chỉ trả 100 hàng rồi mới lọc, các hàng khớp đúng có thể nằm ngoài 100 hàng đầu và bị bỏ sót; một kiểm tra thực tế cho thấy bước `EQUALS` của PoC Joomla chỉ còn 99 hàng vì một hàng trong 100 hàng đầu bị bộ lọc loại, trong khi tổng số khớp thật lớn hơn nhiều. Vì vậy mỗi bước quét tối đa `SCAN_LIMIT = 2000` hàng, áp dụng toán tử, rồi giữ `ROW_CAP = 100` hàng đầu làm bằng chứng và lưu `matched_total`. Cờ `scan_truncated` cho biết lượt quét chạm giới hạn, nghĩa là tổng chỉ là cận dưới (báo cáo ghi ≥).

Hai toán tử cần xử lý riêng ở tầng truy xuất. `MATCHES` không thể đưa nguyên biểu thức chính quy vào `LIKE` (dấu `.` hay `\d` sẽ không khớp gì); hệ thống chỉ dùng đoạn literal bắt buộc dài nhất của biểu thức (ví dụ `acunet` cho `acunet.x-\d+`) để thu hẹp, và bỏ thu hẹp nếu biểu thức có phép chọn `|`. `EXISTS` được đẩy xuống SQL thành điều kiện "trường khác rỗng" để giới hạn hàng không che mất kết quả.

4.5.2. Độ phủ nguồn

Mỗi bước khai báo `source_kind` (`process`, `web`, `dns`, `authentication`, `file`, `smb`). Adapter ánh xạ loại nguồn sang các giá trị `native_type` tương ứng và có ba truy vấn có tham số: đếm bản ghi trong cửa sổ (tuỳ chọn theo host), liệt kê loại sự kiện `native_type/event_id` kèm số lượng, và liệt kê các host thực sự ghi nguồn. Nếu adapter không có các phương thức này (như adapter Splunk hiện tại), độ phủ được ghi là "không kiểm tra được", và luật khuyến nghị hạ độ tin cậy tương ứng.

Phạm vi host ngầm. Hàm `able_drive` suy ra một host từ `able.location` (token đầu tiên trông giống tên máy, ví dụ `we1149srv`), một tài khoản từ `able.actor` và các literal cụ thể từ `able.behavior`/`able.evidence`; chúng được thêm vào mọi bước. Đây là hành vi có chủ ý của thiết kế Execute từ phiên bản trước nhưng không hiển thị trong predicate, nên đồ án bổ sung ba biện pháp: (i) báo cáo ghi rõ phạm vi thực tế; (ii) độ phủ được đếm cả trong phạm vi host; (iii) khi phạm vi host không có dữ liệu mà cửa sổ có, hệ thống chạy lại không lọc host. Lý do cần cả ba được nêu ở mục 7.2.2: trong dữ liệu thực nghiệm, máy được nêu trong `location` của cả ba PoC rỗng không ghi nguồn tương ứng, nên các PoC này chưa từng thực sự tìm kiếm trên dữ liệu nào.

4.5.3. Giới hạn cửa sổ và thời lượng

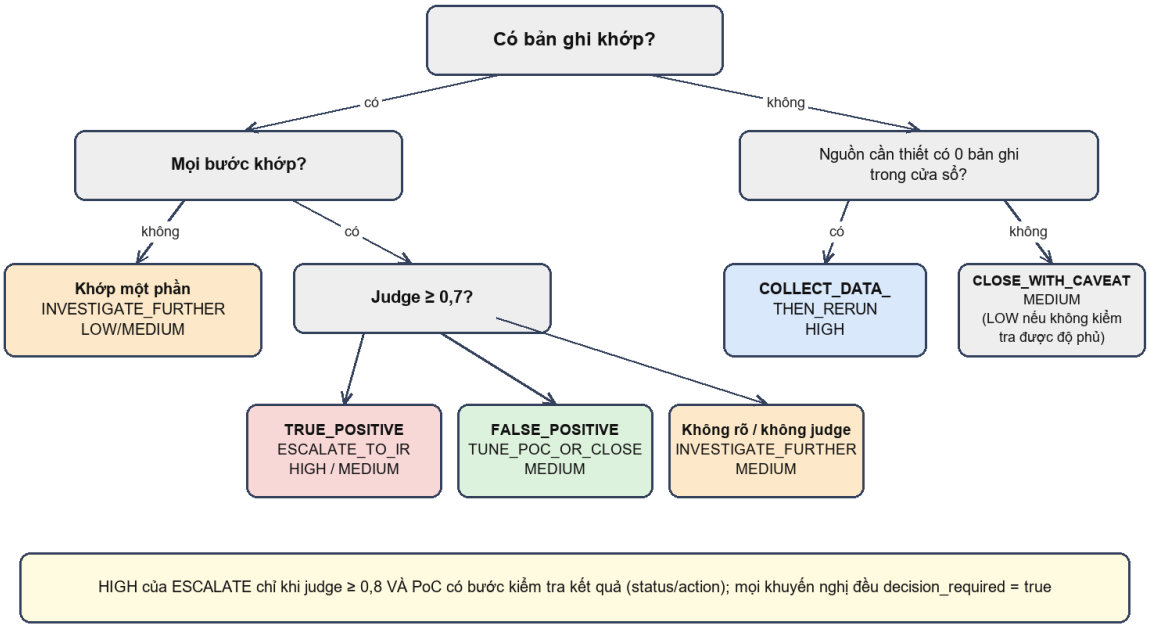
Trường `max_duration` làm hai việc: cắt cửa sổ telemetry dài hơn thời lượng này về phần cuối, và đặt hạn cho lượt tinh chỉnh. Cửa sổ cụ thể lấy từ `--window` hoặc từ `time_window` của PoC.

4.6. Bộ luật khuyến nghị

4.6.1. Các disposition

- `ESCALATE_TO_IR`: chuỗi khớp đủ và judge nhận định có dấu hiệu độc hại với độ tin cậy từ 0,7.
- `INVESTIGATE_FURTHER`: có bản ghi khớp nhưng chuỗi chưa đủ, hoặc ngữ cảnh chưa chứng minh.
- `TUNE_POC_OR_CLOSE`: chuỗi khớp đủ nhưng judge thấy hoạt động hợp lệ; nên thêm điều kiện loại trừ hoặc đóng.
- `COLLECT_DATA_THEN_RERUN`: không có hit và có nguồn cần thiết với 0 bản ghi trong cửa sổ (hoặc 0 bản ghi trong phạm vi host mà không có lần chạy lại không lọc host); không có dữ liệu thì không có kết luận.
- `CLOSE_WITH_CAVEAT`: không có hit trong khi nguồn có dữ liệu; có thể đóng nhưng phải ghi rõ giới hạn.

4.6.2. Cây quyết định



Hình 5. *Cây quyết định của luật khuyến nghị*

Bảng 9. *Luật xác định khuyến nghị và độ tin cậy*

Điều kiện	Khuyến nghị	Tin cậy
Mọi bước khớp, judge TRUE_POSITIVE ≥ 0,7	`ESCALATE_TO_IR`	HIGH nếu judge ≥ 0,8 và PoC có bước kiểm tra kết quả; ngược lại MEDIUM
Mọi bước khớp, judge FALSE_POSITIVE ≥ 0,7	`TUNE_POC_OR_CLOSE`	MEDIUM

Điều kiện	Khuyến nghị	Tin cậy
Mọi bước khớp, judge không rõ hoặc không có	`INVESTIGATE_FURTHER`	MEDIUM
Chỉ khớp một phần	`INVESTIGATE_FURTHER`	LOW (không judge) hoặc MEDIUM
Không khớp, một nguồn cần thiết có 0 bản ghi	`COLLECT_DATA_THEN_RERUN`	HIGH
Không khớp, nguồn có dữ liệu nhưng 0 bản ghi của host PoC; chạy lại không lọc host vẫn 0	`CLOSE_WITH_CAVEAT`	MEDIUM
Như trên nhưng chạy lại không lọc host có hit ở host khác	`INVESTIGATE_FURTHER`	LOW
Không khớp, nguồn có dữ liệu trong phạm vi tìm	`CLOSE_WITH_CAVEAT`	MEDIUM
Không khớp, không kiểm tra được độ phủ	`CLOSE_WITH_CAVEAT`	LOW

4.6.3. Lý giải các quyết định thiết kế

Chuỗi không đầy đủ không bao giờ lên escalate. Nếu chỉ một phần các bước khớp, giả thuyết về một chuỗi tấn công hoàn chỉnh chưa được ủng hộ, bất kể judge nói gì. Quy tắc này giữ cho một nhận định LLM đơn lẻ không thể đẩy hệ thống tới hành động tốn kém.

Trần độ tin cậy khi chưa chứng minh thành công. Một PoC không có bước nào trên trường kết quả (`status`, `action`, `result`, `response`, `outcome`) chỉ chứng minh được có hoạt động tương ứng, không chứng minh được hoạt động đó thành công. Trường hợp này độ tin cậy của `ESCALATE_TO_IR` bị hạ về MEDIUM và lý do được ghi vào báo cáo. Quy tắc xuất hiện từ lần chạy thử đầu tiên: phiên bản đầu của luật cho PoC khai thác Joomla độ tin cậy HIGH dù dữ liệu chỉ có yêu cầu web không có mã trạng thái.

Judge chỉ đẩy một bậc và chỉ khi tự tin. Judge dưới 0,7 không thay đổi kết quả. Advisor không thay đổi `disposition` hay `confidence`.

Thiếu nguồn là kết luận riêng. Khi một nguồn cần thiết có 0 bản ghi, kết quả được gọi bằng tên riêng chứ không gộp vào "rỗng"; đây là hiện thực trực tiếp của nguyên tắc ở mục 2.5.

Phạm vi rỗng khác với dữ liệu rỗng. Nguồn có dữ liệu trong cửa sổ nhưng không có bản ghi của host mà PoC nhắm tới là trường hợp thứ ba, không phải "thiếu nguồn" và cũng không phải "không có hoạt động". Hệ thống không tự chọn một cách đọc: nó chạy lại không lọc host và để kết quả quyết định. Nếu vẫn không có hit, khuyến nghị là đóng kèm cảnh báo (và nói rõ PoC chưa từng được thử trên host đó); nếu có hit ở host khác, khuyến nghị là điều tra với độ tin cậy thấp vì bằng chứng nằm ngoài phạm vi mà PoC khai báo.

4.6.4. Lựa chọn xếp hạng

Mỗi khuyến nghị đi kèm danh sách lựa chọn khả dĩ, xếp theo ưu tiên với lựa chọn của luật ở đầu. Với kết quả có bản ghi, các lựa chọn thuộc tập {escalate, điều tra thêm, tinh chỉnh/đóng}; với kết quả rỗng thuộc tập {bổ sung telemetry, đóng kèm cảnh báo}. Văn bản cho lựa chọn escalate nhắc người dùng xác nhận vai trò thật của máy được nêu (nạn nhân, nguồn tấn công hay cảm biến ghi log) trước khi cô lập, vì trong dữ liệu thực nghiệm máy xuất hiện nhiều nhất chính là một máy thu log.

4.7. Thiết kế bảo mật và quyền riêng tư

- **Bí mật:** khoá API chỉ nằm trong `.env` và biến môi trường của tiến trình. Tệp `model_config.json` sinh ra chỉ chứa `${LLM_API_KEY}` và `${PEAK_LLM_BASE_URL}`. Đã kiểm tra rằng khoá không xuất hiện trong bất kỳ tệp kết quả nào.
- **Truy vấn:** mọi giá trị đưa vào SQL đều là tham số; tên cột và thao tác đi qua danh sách cho phép (`allowlist`); cửa sổ thời gian được kiểm tra định dạng.
- **Dữ liệu gửi cho LLM:** đây là rủi ro cần nêu rõ. Judge nhận tối đa mười hàng đã rút gọn, advisor nhận bản tóm tắt bằng chứng gồm tối đa năm hàng mẫu và các giá trị phổ biến, và PEAK nhận mô tả cấu trúc dữ liệu cùng kết quả đếm. Nếu endpoint LLM nằm ngoài tổ chức, các đoạn telemetry này rời khỏi môi trường. Có ba lớp giảm thiểu: cảnh báo khi endpoint không nằm trên máy cục bộ; cờ `--redact` thay tên máy, tên người dùng và mọi địa chỉ IPv4 bằng token ổn định (`<HOST_1>`, `<USER_2>`, `<IP_3>`) trong mọi lời nhắc gửi judge và advisor rồi khôi phục trong báo cáo; và với dữ liệu thật nên dùng mô hình cục bộ (Ollama, vLLM, chỉ cần đổi `.env`) hoặc `--offline`. `--redact` không che chuỗi tự do (dòng lệnh, URL, tên miền) và không che văn bản PoC cùng mô tả dữ liệu gửi cho PEAK, nên nó giảm rủi ro chứ không loại bỏ.
- **Vận hành LLM:** các cơ chế sau đều nằm ngoài đường bằng chứng. Bộ đếm `UsageMeter` bọc lời gọi `chat.completions.create` của SDK nên thấy mọi yêu cầu, kể cả của các tác tử PEAK, và ghi tên mô hình thực sự trả lời; `--token-budget` đặt trần token cho mỗi PoC (chạm trần thì lời gọi sau bị từ chối và không được gọi lại); `LLM_MODEL_FALLBACKS` chuyển sang mô hình dự phòng sau khi mô hình chính hỏng hẳn; judge chạy `--judge-votes` lần (mặc định 3) ở temperature 0 và lấy đa số, với độ tin cậy là mức thấp nhất trong nhóm đa số, còn không có đa số thì không đưa ra nhận định nào; kết quả Prepare được lưu theo khoá gồm nội dung PoC, mô tả dữ liệu, mô hình và phiên bản PEAK.
- **Thành phần ngoài:** PEAK Assistant tự công bố là chưa qua kiểm thử bảo mật; hệ thống chỉ nên chạy cục bộ và không mở cổng dịch vụ.

4.8. Cấu trúc đầu ra

Mỗi lần chạy tạo một thư mục theo thời gian (hoặc theo `--out`) chứa tệp `summary.md`/`summary.json` tổng hợp và một thư mục con cho mỗi PoC với các tệp sau.

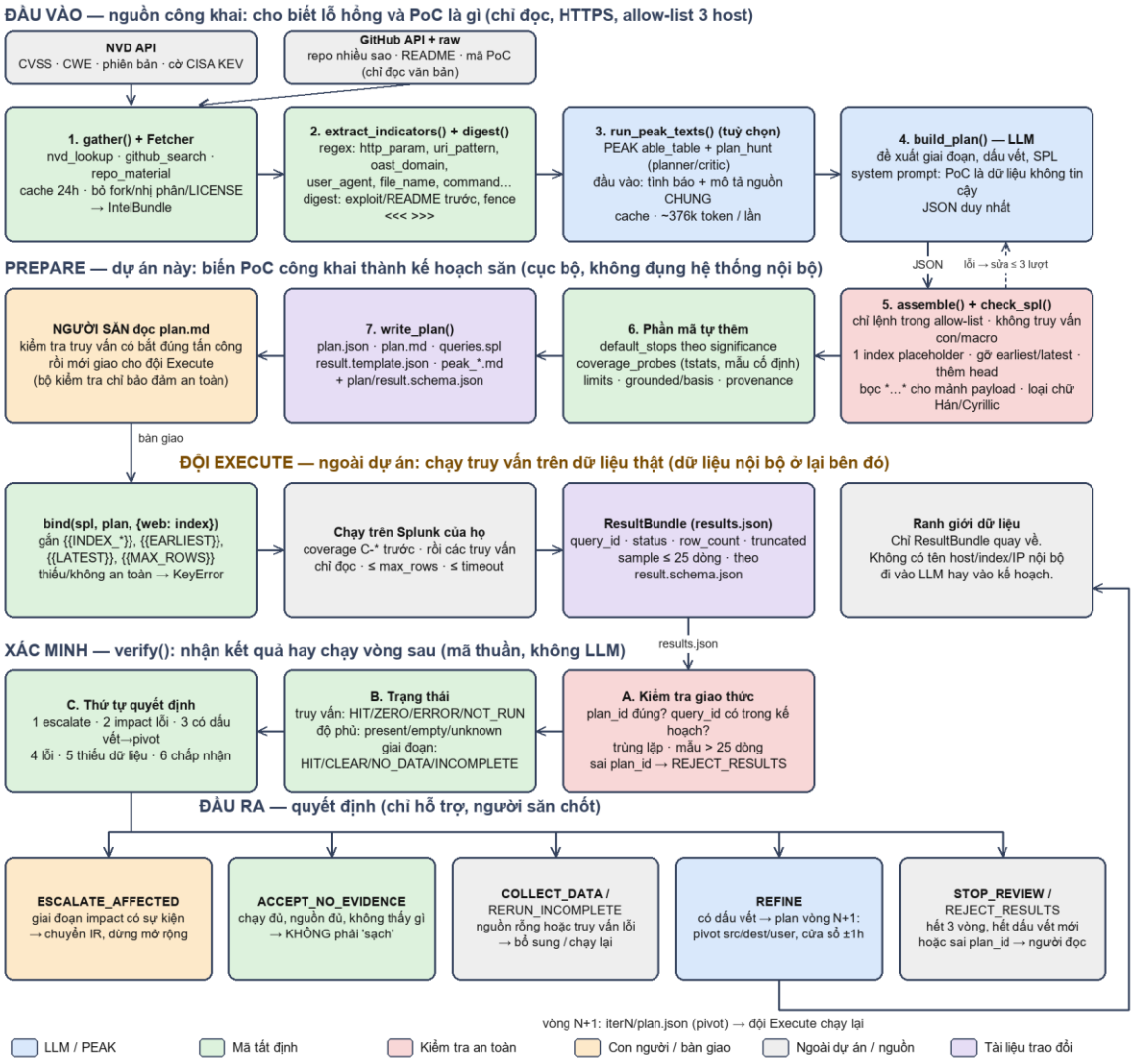
Bảng 10. *Các tệp đầu ra của một PoC*

Tệp	Nội dung
`recommendation.md`	Báo cáo cho người quyết định, tiếng Việt
`recommendation.json`	Cùng nội dung ở dạng máy đọc, kèm phần thực thi, PEAK và siêu dữ liệu
`peak_able.md`	Bảng ABLE do PEAK sinh (hoặc từ PoC khi không dùng PEAK)
`peak_hunt_plan.md`	Kế hoạch săn do PEAK sinh
`<poc_id>.json`	Sổ cái thực thi: từng bước, số hàng, hàng khớp, nhật ký tinh chỉnh
`act/`	Bản nháp SPL và kết quả kiểm tra tĩnh, backlog, ghi chú stakeholder
`ir/`	Gói bàn giao cho IR khi có hit

4.9. Thiết kế luồng Prepare-only

4.9.1. Tổng quan

Hình 6 trình bày luồng theo năm vùng. Vùng đầu là nguồn công khai. Vùng Prepare (do dự án này làm) gồm bảy bước, trong đó hai bước dùng LLM (PEAK và bộ lập kế hoạch) còn năm bước là mã tất định. Vùng Execute do đội khác thực hiện; ranh giới dữ liệu nằm ở đây: chỉ `ResultBundle` quay về. Vùng xác minh (`verify`) không dùng LLM. Vùng cuối là các quyết định, chỉ để hỗ trợ người săn.



Hình 6. *Quy trình Prepare-only: từ PoC công khai đến kế hoạch săn, đội Execute chạy, xác minh và vòng pivot*

Có hai quyết định thiết kế xuyên suốt. Thứ nhất, **LLM chỉ đề xuất, mã mới quyết định**: LLM nêu giai đoạn, dấu vết và truy vấn; mã kiểm tra từng thứ, và tự thêm điểm dừng, giới hạn, truy vấn độ phủ. Thứ hai, **dữ liệu đi một chiều**: văn bản công khai đi vào, kế hoạch đi ra, và chỉ kết quả đã được đội thực thi đồng ý trả mới quay lại.

4.9.2. Mô hình dữ liệu

Kế hoạch (`HuntPlan`) và kết quả (`ResultBundle`) đều là mô hình Pydantic có `schema_version` và được xuất thành JSON Schema (`plan.schema.json`, `result.schema.json`) để hai đội thống nhất hợp đồng.

Bảng 11. *Các trường chính của* `HuntPlan`

Trường	Ý nghĩa
`hypothesis`, `applicability`	Giả thuyết "giả sử PoC này được dùng"; sản phẩm, phiên bản bị ảnh hưởng, điều kiện, khi nào không áp dụng

Trường	Ý nghĩa
`stages[]`	Giai đoạn tấn công: `phase` (chiến thuật ATT&CK), `technique_ids`, `significance`, `observables[]`, `queries[]`, `stop_conditions[]`
`significance`	`context` (nhiễu mong đợi), `indicator` (nỗ lực khớp PoC), `impact` (thành công hoặc hậu khai thác); quyết định điểm dừng mặc định
`observables[].basis`	`from_poc` nếu chuỗi có nguyên văn trong văn bản PoC/CVE đã thu thập, ngược lại `inferred`
`queries[]`	SPL với placeholder `{{INDEX_*}}`, `{{EARLIEST}}`, `{{LATEST}}`, `{{MAX_ROWS}}`; `data_source`; `grounded`; `benign_notes`
`coverage_probes[]`	Truy vấn `tstats` kiểm tra mỗi nguồn có dữ liệu trong cửa sổ (do mã sinh từ mẫu)
`limits`	Cửa sổ, số truy vấn, số dòng và thời gian tối đa, số vòng, chỉ đọc
`provenance`, `dropped`	Nguồn gốc (không chạy PoC, không dùng dữ liệu nội bộ, mô hình, PEAK); truy vấn bị loại và ghi chú của bộ kiểm tra

`ResultBundle` gồm `plan_id`, `iteration` và với mỗi truy vấn: `status` (`ok`, `error`, `timeout`, `skipped`), `row_count`, `truncated` và `sample` tối đa 25 dòng. "Kết quả" của một truy vấn là số dòng trả về, nên truy vấn `stats count by ...` trả vài dòng chứ không phải số sự kiện.

4.9.3. Kiểm tra truy vấn trước khi rời dự án

Hàm `check_spl` biến một truy vấn do LLM viết thành dạng chuẩn hoặc từ chối. Nguyên tắc là *từ chối trừ khi chắc chắn vô hại*: truy vấn phải bắt đầu bằng `search`, có đúng một `index={{INDEX_*}}` thuộc nhóm cho phép, chỉ dùng lệnh trong danh sách cho phép (không `delete`, `outputlookup`, `sendemail`, `rest`, `join`, `map`), không truy vấn con, không macro, không `$token$`, có bộ lọc ngoài chỉ mục, tối đa 1.500 ký tự và 10 công đoạn. Mã tự gỡ `earliest/latest` do LLM viết, chèn cửa sổ từ placeholder và thêm `head {{MAX_ROWS}}`. Trong Splunk `field="x"` là khớp chính xác, nên mảnh payload như `${jndi:ldap://` sẽ không bao giờ khớp; mã tự bọc `*...*` và ghi chú lại.

Hai quy tắc được thêm vào về sau, sau khi quan sát đầu ra thật.

- **Hậu khai thác phải gắn với dịch vụ bị tấn công.** `whoami` hay một shell do quản trị viên chạy trông y hệt khi chạy qua webshell; chỉ tiến trình cha mới phân biệt được. Do đó truy vấn trên nguồn `endpoint` phải *lọc* theo `parent_process_name` hoặc `parent_process` (nhắc tới trong `stats ... by` không tính). Truy vấn thiếu bị gửi lại để sửa kèm gợi ý (ví dụ `java` với ứng dụng Java, `w3wp.exe` với IIS); còn thiếu thì bị loại và ghi vào `dropped`. Truy vấn về file hay registry được miễn.
- **Chữ lẫn ngôn ngữ khác.** Mô hình miễn phí đôi khi chèn một từ ngoại ngữ vào câu tiếng Việt (`przeciwko`, `tentativa`, `explotación`, `ungewö`). Bộ lọc ký tự

CJK/Cyrillic/Ả Rập không bắt được chữ Latin. Mô-đun `language.py` xét từng từ: chữ cái không có trong tiếng Việt lẫn tiếng Anh thì chắc chắn lạ; từ là một âm tiết tiếng Việt hợp lệ (phụ âm đầu, nguyên âm, phụ âm cuối, bỏ dấu thanh) thì tiếng Việt; còn lại phải nằm trong từ vựng tiếng Anh dùng trong kế hoạch, hoặc trong chính văn bản PoC/CVE, hoặc là mã định danh, từ viết tắt, tên riêng, nếu không thì bị gắn cờ. Việc gắn cờ chỉ yêu cầu viết lại một lần và liệt kê từ còn sót; không loại truy vấn.

Một giai đoạn mất hết truy vấn vì bị loại được in cảnh báo trong `plan.md`, và `verify` coi giai đoạn đó là `INCOMPLETE`, không bao giờ `CLEAR`.

4.9.4. Xác minh và vòng săn tiếp

`verify` nhận `HuntPlan` và `ResultBundle` và chạy tuần tự các bước sau, không có lời gọi LLM nào.

1. **Giao thức:** `plan_id` khác thì từ chối cả bộ (`REJECT_RESULTS`); truy vấn không có trong kế hoạch bị bỏ qua và ghi lại; mẫu quá 25 dòng bị cắt.
2. **Trạng thái truy vấn:** `HIT`, `ZERO`, `ERROR`, `NOT_RUN`; kết quả bị cắt khi `truncated` hoặc số dòng chạm trần.
3. **Độ phủ:** mỗi truy vấn `C-*` cho biết nguồn có dữ liệu (`present`), rỗng (`empty`) hay chưa biết (`unknown`).
4. **Trạng thái giai đoạn:** `HIT`, `INCOMPLETE`, `NO_DATA`, `CLEAR`.
5. **Quyết định theo thứ tự ưu tiên.**

Bảng 12. *Thứ tự quyết định của* `verify`

#	Điều kiện	Quyết định
1	Giai đoạn `impact` có kết quả và có hành động `escalate`	`ESCALATE_AFFECTED`
2	Truy vấn của giai đoạn `impact` lỗi hoặc chưa chạy, và không có kết quả	`RERUN_INCOMPLETE`
3	Có kết quả ở giai đoạn khác	`REFINE`, hoặc `STOP_REVIEW` nếu hết số vòng, không còn giá trị an toàn để pivot
4	Có giai đoạn `INCOMPLETE`	`RERUN_INCOMPLETE`
5	Có giai đoạn `NO_DATA`	`COLLECT_DATA`
6	Còn lại	`ACCEPT_NO_EVIDENCE`, kèm câu "không phải kết luận sạch"

Vòng `REFINE` lấy từ các truy vấn trúng các giá trị `src`, `dest`/`host`, `user` (mỗi loại tối đa ba giá trị phổ biến nhất, bỏ giá trị đã pivot) và thu cửa sổ về `[sớm nhất − 1 giờ, muộn nhất + 1 giờ]`. Giá trị lấy từ log có thể do kẻ tấn công kiểm soát nên chỉ được nhúng vào truy vấn nếu qua bộ lọc ký tự nghiêm ngặt (`spl_literal`; `9.9.9.9\"`

| delete bị loại), và truy vấn pivot vẫn qua check_spl. Ba giai đoạn pivot cố định: yêu cầu web từ nguồn tấn công; tiến trình shell hoặc công cụ tải về trên host bị nhắm cùng DNS tới tên miền out-of-band; kết nối đi ra ngoài dải IP riêng cùng hoạt động xác thực. Vòng dừng khi chuyển IR, chấp nhận, hết số vòng tối đa (3) hoặc không còn giá trị mới để pivot.

4.9.5. Ranh giới tin cậy

Bảng 13. *Ai tin ai trong luồng Prepare-only*

Dữ liệu	Nguồn	Cách đối xử
README, mã PoC, mô tả CVE	Internet	Không tin cậy: đặt trong hàng rào <<< >>> gắn nhãn "UNTRUSTED DATA"; system prompt yêu cầu bỏ qua mọi chỉ dẫn trong đó
Đầu ra của LLM	Mô hình	Không tin cậy: không bao giờ thực thi; qua check_spl, đối chiếu grounded, quy tắc tiến trình cha, bộ phát hiện ngôn ngữ
Điểm dừng, giới hạn, truy vấn độ phủ	Mã của dự án	Sinh từ mẫu, không do LLM quyết định
Kết quả thực thi	Đội Execute	Kiểm tra giao thức; giá trị trong mẫu có thể do kẻ tấn công kiểm soát nên chỉ nhúng qua spl_literal
Yêu cầu HTTP ra ngoài	Dự án	Chỉ GET, https, ba host cho phép; chuyển hướng cũng bị kiểm tra lại từng bước (tối đa ba lần); khoá API chỉ đi cùng yêu cầu tới host của nó

4.9.6. Ba bảo đảm của phiên bản 0.4

Lõi tất định và gợi ý của LLM. Mỗi giai đoạn và truy vấn của kế hoạch mang trường origin. Phần core do mô-đun plan/core.py dựng từ các dấu vết đã trích trong văn bản PoC/CVE: giai đoạn S1 (ngữ cảnh: User-Agent, đường dẫn), S2 (nỗ lực khai thác: chuỗi payload, tham số, header), S3 (hậu khai thác: tên miền out-of-band, tệp do PoC tạo, tiến trình con của dịch vụ bị tấn công khi suy ra được họ tiến trình cha từ tên sản phẩm trong CVE). Điểm dừng, giới hạn và truy vấn độ phủ cũng sinh từ mẫu. LLM chỉ được đề xuất tối đa hai giai đoạn bổ sung, mỗi giai đoạn tối đa hai truy vấn, mang origin="llm", đi qua cùng bộ kiểm tra, và không bao giờ có điểm dừng escalate. Hai bản băm tách bạch hai phần: core_digest chỉ phủ phần lõi (không đổi theo mô hình), plan_digest phủ toàn bộ nội dung người duyệt.

Quy trình từng bước cho con người. Từ kế hoạch, plan/runbook.py sinh RUNBOOK.md không dùng LLM: R0 duyệt kế hoạch, R1 gắn chỉ mục và thời gian, kiểm tra độ phủ, một bước cho mỗi truy vấn (kèm mục "ghi lại" và "dừng nếu"), một cổng

quyết định của người sau mỗi giai đoạn, nộp kết quả, quyết định cuối. Kết quả trả về phải nhắc `plan_digest`, `approved_by`, `approved_at`; thiếu hoặc sai thì `verify` trả `REJECT_RESULTS`. Gợi ý của LLM chỉ được tính khi liệt kê trong `accepted_llm_stages`, và kể cả khi đó kết quả chỉ để tham khảo.

Kiểm tra SPL hai lần. `check_spl` chạy lúc lập kế hoạch; lệnh `bind` chạy `lint_bound` trên truy vấn sau khi gắn giá trị thật. `bind` chèn mốc thời gian tuyệt đối, từ chối `index=*`, chỉ mục nội bộ, cửa sổ rộng quá 90 ngày hoặc kết thúc trong tương lai, thiếu `head N`, lệnh giữ cả tập kết quả trong bộ nhớ (`eventstats`, `streamstats`, `mvexpand`, `makemv`), nhóm hoặc khử trùng theo `_raw`, `limit=0` và `sort 0`. Việc từ chối là toàn lô: có lỗi nào thì không ghi file nào.

Chương 5. Cài đặt hệ thống

5.1. Môi trường và công nghệ

Hệ thống được viết bằng Python 3.12, chạy trên Windows với môi trường ảo `.venv`; không có thành phần nào phụ thuộc hệ điều hành. Các thư viện chính:

Bảng 14. *Công nghệ sử dụng*

Thành phần	Vai trò	Ghi chú
Python 3.12	Ngôn ngữ	Yêu cầu của PEAK Assistant
PEAK Assistant	Prepare: ABLE, kế hoạch	Ghim theo commit `dfabbb0`, giấy phép MIT
AutoGen (`autogen-agentchat`, `autogen-ext`)	Khung đa tác tử mà PEAK dùng	Được kéo theo PEAK
SQLite	Kho telemetry CDB	Thư viện chuẩn `sqlite3`
Pydantic, PyYAML, requests	Hợp đồng dữ liệu, đọc cấu hình, gọi HTTP	Phụ thuộc lõi
pytest, ruff	Kiểm thử, kiểm tra kiểu mã	Phụ thuộc phát triển
NVD API 2.0, GitHub REST API	Nguồn tình báo công khai cho luồng Prepare-only	Chỉ GET; khoá `NVD_API_KEY`, `GITHUB_TOKEN` là tuỳ chọn

PEAK Assistant được khai báo là phụ thuộc **tuỳ chọn** (`pip install -e ".[peak]"`), nhờ đó cài đặt lõi không phải kéo theo AutoGen, Streamlit và nhiều thư viện khác. Mọi `import` vào PEAK nằm trong thân hàm, nên mô-đun của hệ thống nạp được cả khi PEAK chưa cài.

5.2. Cấu trúc mã nguồn

Sau khi làm gọn, `src/hunting/` gồm các mô-đun sau với tổng khoảng 9.400 dòng, trong đó khoảng 2.300 dòng thuộc luồng Prepare-only (`intel/`, `plan/`, `plan_cli.py`).

Bảng 15. *Cấu trúc mã nguồn*

Đường dẫn	Dòng	Vai trò
`llm.py`	~415	Cấu hình LLM, gọi lại, mô hình dự phòng, bộ đếm token, bộ gọi đồng bộ
`redact.py`	~140	Che host/user/IP trước khi gửi LLM, khôi phục khi nhận
`prepare.py`	~290	Cầu nối PEAK Assistant, cache kết quả Prepare
`recommend.py`	~500	Tóm tắt bằng chứng, luật khuyến nghị, advisor
`report.py`	~160	Dựng báo cáo Markdown và bảng tổng hợp

Đường dẫn	Dòng	Vai trò
`pipeline.py`	~155	Điều phối một PoC từ đầu đến cuối
`cli.py`	~140	Dòng lệnh của pipeline PoC (chuyển `plan`, `verify`, `schema` sang `plan_cli.py`)
`plan_cli.py`	~165	Dòng lệnh `plan`, `verify`, `schema` của luồng Prepare-only
`intel/`	~700	Thu thập tình báo: `http.py` (Fetcher), `sources.py` (NVD, GitHub), `extract.py` (dấu vết), `gather.py` (gom và tóm tắt cho LLM)
`plan/`	~1.620	`schema.py`, `catalog.py`, `safety.py`, `language.py`, `build.py`, `render.py`, `verify.py`, `files.py`
`poc/`	~1.700	Mô hình PoC, nạp JSON, tác tử thực thi, judge, báo cáo
`adapters/`	~1.170	Adapter CDB, danh sách cho phép, kiểm soát
`act/`	~365	Bản nháp SPL, backlog, ghi chú stakeholder
`contracts/`, `controller/`	~1.320	Hợp đồng dữ liệu mà adapter cần, theo dõi chi phí
`peak.py`	~450	Cổng Prepare, phân tích thời lượng, cửa sổ, ABLE → quan sát cụ thể

Hai nguyên tắc tổ chức: các mô-đun mới (`llm`, `redact`, `prepare`, `recommend`, `report`, `pipeline`, `cli`) chỉ phụ thuộc vào `poc/` và `adapters/` qua giao diện công khai; và không có mô-đun nào của lõi phụ thuộc trực tiếp vào PEAK ngoài `prepare.py` và `llm.py`.

5.3. Lớp LLM (`llm.py`)

5.3.1. Từ `.env` đến cấu hình PEAK

PEAK Assistant đọc cấu hình mô hình từ một tệp `model_config.json` có thể nội suy biến môi trường dạng `${TÊN}`. Hệ thống đọc ba biến `LLM_ENDPOINT`, `LLM_API_KEY`, `LLM_MODEL` từ `.env` (hoặc từ biến môi trường, hoặc đối số `--model`), tách địa chỉ gốc ra khỏi đường dẫn `/chat/completions`, rồi sinh cấu hình chỉ chứa placeholder:

```
def model_config(self) -> dict:
    return {
        "version": "1",
        "providers": {"hunting-llm": {
            "type": "openai",
            "config": {"api_key": "${LLM_API_KEY}",
                       "base_url": "${PEAK_LLM_BASE_URL}",
                       "timeout": self.timeout, "max_tokens": self.max_tokens},
```

```
            "models": {self.model: {"model_info": {...}}},
        }},
        "defaults": {"provider": "hunting-llm", "model": self.model},
    }
```

Trường `model_info` là bắt buộc của AutoGen khi dùng tên mô hình không thuộc danh sách OpenAI. Hệ thống điền các giá trị trung tính (không hỗ trợ hình ảnh, không dùng gọi hàm) để mọi mô hình tương thích OpenAI đều dùng được. Khoá API và địa chỉ cơ sở được đặt vào `os.environ` của tiến trình ngay trước khi nạp cấu hình, nên không tệp nào trên đĩa chứa khoá.

5.3.2. Tính tuỳ chọn

Hàm `LlmSettings.from_env` ném `LlmUnavailable` khi thiếu cấu hình hoặc khi giá trị còn là placeholder của `.env.example` (chứa `<` hoặc bắt đầu bằng `sk-or-...`). `configure_peak` cũng ném `LlmUnavailable` nếu thư viện PEAK chưa cài. CLI bắt ngoại lệ này, in một dòng thông tin (không phải lỗi) và tiếp tục chạy phần tất định. Nhờ vậy cùng một lệnh chạy được trên máy có và không có LLM.

5.3.3. Gọi lại có backoff

Quan sát thực tế cho thấy endpoint LLM đôi lúc trả HTTP 404 `model_not_found` dù mô hình tồn tại; ứng dụng khách OpenAI không thử lại lỗi 404. Hàm sau gọi lại bất kỳ lời gọi bất đồng bộ nào với khoảng chờ tăng dần, và ném lại lỗi cuối nếu hết lượt:

```
RETRY_DELAYS = (3.0, 10.0, 25.0)

async def retry_async(factory, *, delays=RETRY_DELAYS, label="LLM call", notes=None):
    for attempt in range(len(delays) + 1):
        try:
            return await factory()
        except asyncio.CancelledError:
            raise
        except Exception as exc:
            if attempt >= len(delays):
                raise
            if notes is not None:
                notes.append(f"{label}: retry {attempt + 1}/{len(delays)} after
{type(exc).__name__}")
            await asyncio.sleep(delays[attempt])
```

`PeakLlm.__call__` (dùng cho judge và advisor) và hai bước ABLE, kế hoạch trong `prepare.py` đều dùng hàm này. Một sai cấu hình thật (khoá sai, tên mô hình sai) vẫn thất bại sau lượt cuối với nguyên thông báo gốc.

5.3.4. Vòng lặp sự kiện

PEAK là bất đồng bộ còn pipeline là đồng bộ. Hàm `run_async` tạo vòng lặp mới cho mỗi lần chạy, đóng vòng lặp đúng cách và cài một bộ xử lý ngoại lệ bỏ qua cảnh báo vô hại `Event loop is closed` do các ứng dụng khách HTTP bị dọn sau khi vòng lặp đã đóng. Cảnh báo này xuất hiện trong lần chạy thật đầu tiên và được loại bỏ ở đây; các lỗi khác vẫn được chuyển tiếp bình thường.

5.4. Cầu nối PEAK (`prepare.py`)

Hàm `_peak_prepare` thực hiện hai bước, mỗi bước bọc trong `retry_async` và trong `asyncio.wait_for` với thời hạn mặc định 420 giây:

```
async def _able() -> str:
    text = await asyncio.wait_for(
        able_table(hypothesis=hypothesis, research_document=research,
                   local_data_document=data_document, local_context=LOCAL_CONTEXT),
        timeout)
    if not text or text.startswith("Error while generating"):
        raise RuntimeError(text or "empty ABLE table")   # able_table nuốt lỗi thành
chuỗi
    return text
able = await retry_async(_able, label="ABLE", notes=result.notes)
```

Chi tiết quan trọng: hàm `able_table` của PEAK bắt mọi ngoại lệ và trả về chuỗi bắt đầu bằng `Error while generating`; nếu không nhận ra điều này, lỗi sẽ bị coi là một bảng ABLE hợp lệ. Bước kế hoạch có kiểm tra tương tự với các chuỗi `Could not create a hunt plan` và `no plan was generated`.

PEAK và AutoGen in nhiều dòng traceback cho mỗi lời gọi lỗi. Ngữ cảnh `_quiet_peak` chuyển hướng đầu ra chuẩn và đầu ra lỗi, và nâng ngưỡng ghi log của các logger `autogen_*` lên mức nghiêm trọng trong lúc gọi PEAK; nguyên nhân lỗi vẫn được ghi lại bằng chữ trong `peak.notes`. Hàm `run_prepare` bọc tất cả: nếu `llm` là rỗng thì dùng ABLE và kế hoạch dựng từ PoC; ngược lại thử tối đa hai lượt và, nếu cả hai thất bại, quay về chế độ này với ghi chú rõ ràng.

5.5. Bộ thực thi PoC (`poc/agent.py`)

`PocAgent.run` thực hiện: kiểm tra cổng Prepare, cắt cửa sổ theo `max_duration`, chạy vòng thực thi, gọi judge (nếu bật), ghi gói IR khi có hit, ghi sổ cái. Vòng thực thi `_execute_loop` có đặc tính đáng chú ý:

- Lượt 1 chạy mọi bước chính, mỗi bước gọi `adapter.execute_query(operation_id="search_text", search_terms=...)` với giới hạn quét `SCAN_LIMIT = 2000` hàng; sau khi áp dụng toán tử, tối đa `ROW_CAP = 100` hàng được giữ và `matched_total` ghi tổng số khớp.
- Các giá trị cụ thể rút từ ABLE **của PoC** (tên tệp, cờ dòng lệnh, địa chỉ IP, chuỗi trong ngoặc kép; không phải bảng ABLE do PEAK sinh) được AND thêm vào truy vấn; văn xuôi không thêm điều kiện nào. Host suy ra từ `able.location` cũng được dùng làm bộ lọc thực thể. Việc rút trích dùng biểu thức chính quy trong `peak.py`, và mọi điều kiện thêm này được ghi vào `StepResult.scope`.
- Nếu các hit ở bước đầu tiên đến từ đúng một máy khác với máy ABLE nêu, lượt 2 chạy lại các bước còn lại trên máy đó.
- Nếu lượt 1 hoàn toàn rỗng và PoC khai báo `fallbacks` thì chạy các bước thay thế; sau đó có thể chạy lại đúng các vị từ gốc một lần.

- Toán tử không bao giờ được nới: EQUALS không biến thành CONTAINS ở lượt thứ hai.

Hàm _apply_op hiện thực toán tử trên giá trị ô. EQUALS so khớp không phân biệt hoa thường với cả giá trị đầy đủ và phần tên tệp cuối đường dẫn; MATCHES dùng re.search và lùi về CONTAINS nếu biểu thức lỗi; EXISTS yêu cầu ô không rỗng và, nếu value rỗng, loại bỏ mọi hàng để tránh khớp bừa (một lỗi thật đã được phát hiện khi đánh giá trên dữ liệu lớn). Ở tầng truy xuất, MATCHES chỉ gửi cho adapter đoạn literal bắt buộc dài nhất của biểu thức (_regex_literal) và EXISTS được đẩy xuống SQL (parameters={"require_nonempty": [trường]}); nếu không, regex bị đưa nguyên vào LIKE sẽ không khớp gì.

5.6. Adapter

Adapter CDB mở một tệp SQLite và hiện thực execute_query với các thao tác có tham số. Với search_text, mỗi từ khoá tạo một điều kiện COALESCE(cột, '') LIKE ? trên các cột văn bản, nối bằng OR giữa cột và AND giữa các từ khoá; cửa sổ thời gian là hai điều kiện trên timestamp. Hai phương thức mới được thêm cho đồ án:

- source_presence(window, source_kind): đếm số bản ghi trong cửa sổ có native_type thuộc tập tương ứng với loại nguồn; trả None nếu loại nguồn không biết.
- describe_data(): tạo tài liệu Markdown mô tả bảng, các cột, và với mỗi cặp (native_type, event_id) là số dòng, thời điểm đầu và cuối. Đây chính là "tài liệu dữ liệu cục bộ" mà PEAK nhận.

Adapter Splunk trực tiếp (SplunkLiveAdapter, khoảng 2.100 dòng, cùng cổng AST query_safety/, mô hình capabilities/, hai tệp khai báo configs/splunk_*.yaml và hai tệp kiểm thử) đã được gỡ khỏi nhánh chính (mã còn nguyên ở commit 4dace56). Lý do: bản nộp không có máy chủ Splunk để kiểm chứng, tám kiểm thử dành cho Splunk thật luôn bị bỏ qua, và khoảng 2.300 dòng chưa từng được chạy là gánh nặng bảo trì. Nhánh chính chỉ giữ adapter CDB; mã Splunk vẫn nguyên vẹn trong lịch sử git (commit 4dace56).

5.7. Khuyến nghị và báo cáo

Mô-đun recommend.py gồm ba phần. summarize_evidence dựng EvidenceSummary: lấy hàng của các bước chính, khử trùng lặp, tính khoảng thời gian, đếm giá trị xoay trục, và gọi source_presence cho từng loại nguồn (có bộ nhớ đệm theo loại). decide hiện thực bảng luật ở mục 4.6, được trích ở đây phần nhánh có bản ghi khớp:

```
if ev.chain_complete:
    if judge_ok and judge.verdict == TRUE_POSITIVE:
        disposition, confidence = ESCALATE, "HIGH" if judge.confidence >= 0.8 else
"MEDIUM"
        if not outcome_observed and confidence == "HIGH":
```

```
            confidence = "MEDIUM"      # chưa kiểm tra kết quả => chưa chứng minh thành
công
    elif judge_ok and judge.verdict == FALSE_POSITIVE:
        disposition, confidence = TUNE, "MEDIUM"
    else:
        disposition, confidence = INVESTIGATE, "MEDIUM"
else:
    disposition, confidence = INVESTIGATE, "LOW" if judge is None else "MEDIUM"
```

Hàm `advise` gửi cho LLM một lời nhắc chứa giả thuyết, kết luận do luật tính, các lý do, bản tóm tắt bằng chứng (cắt 6.000 ký tự), bảng ABLE (2.500 ký tự) và kế hoạch PEAK (6.000 ký tự), yêu cầu trả JSON chặt với ba mảng. Hàm chịu được đầu ra lệch chuẩn: thử tối đa ba lần khi JSON lỗi hoặc không có bước nào dùng được; chấp nhận mục bước là chuỗi thuần hoặc đối tượng có khoá `step`/`title`/`check` thay cho `action` và `reason`/`rationale` thay cho `why`; nếu thất bại vẫn trả khuyến nghị theo luật cùng một ghi chú giải thích. Hàm `advise` không có đường nào ghi vào `disposition` hay `confidence`.

Mô-đun `report.py` dựng báo cáo Markdown bằng tiếng Việt với các mục cố định: khuyến nghị và lý do, giới hạn, bảng lựa chọn, bằng chứng, judge, gợi ý bước tiếp theo, câu hỏi, rủi ro, kế hoạch PEAK, bản nháp SPL, chi phí và tái lập. Bản nháp SPL do `act/` sinh từ các vị từ: `EQUALS` thành `trường="giá trị"`, các toán tử còn lại thành `match(trường, "(?i)giá trị")`; một kiểm tra tĩnh từ chối các lệnh nguy hiểm như `delete`, `outputlookup`, `sendemail`.

5.8. Dòng lệnh

`cli.py` dùng `argparse` với các nhóm đối số: đầu vào PoC (`--poc`, `--poc-dir`, `--window`), CSDL telemetry (`--db`), LLM (`--env`, `--model`, `--offline`, `--research`, `--peak-timeout`) và `--out`. Với mỗi PoC, lỗi nạp hoặc lỗi cổng Prepare được in ra và tính vào mã thoát, nhưng không dừng các PoC còn lại. Sau cùng ghi `summary.md` và `summary.json`.

5.9. Kiểm thử

Bộ kiểm thử gồm 146 bài, tất cả chạy được và đều đạt (sau khi gỡ adapter Splunk không còn bài nào bị bỏ qua); mã kiểm tra tĩnh `ruff` không báo lỗi. Các bài của pipeline PoC (`tests/unit/test_pipeline.py`, `tests/unit/test_scope_and_caps.py` và `tests/unit/test_llm_ops.py`) kiểm tra:

- **Luật khuyến nghị:** 11 tổ hợp tham số (bằng chứng × judge) ứng với bảng luật; chuỗi một phần không escalate dù judge tin cậy cao; trần độ tin cậy khi chưa có bước kiểm tra kết quả và mở trần khi có.
- **Rỗng không thành sạch:** thiếu nguồn cho `COLLECT_DATA_THEN_RERUN` với câu cảnh báo.
- **Advisor:** phân tích JSON có hàng rào mã; thử lại khi rác hoặc rỗng; chấp nhận dạng lệch chuẩn; lỗi LLM không đổi khuyến nghị.

- **Cấu hình LLM:** đọc `.env`, tách địa chỉ gốc, không để khoá lọt vào cấu hình, từ chối giá trị mẫu.
- **Chịu lỗi PEAK:** khi PEAK ném lỗi thì quay về PoC và có ghi chú; lỗi thoáng qua được thử lại thành công; cơ chế `retry_async` thành công sau vài lần và ném lại lỗi dai dẳng.
- **Độ phủ CDB:** đếm theo loại nguồn (và theo host), cơ cấu loại sự kiện, host ghi nguồn, mô tả dữ liệu.
- **Phạm vi ngầm và giới hạn hàng (mục 7.4):** phạm vi host rỗng phải được ghi nhận và chạy lại không lọc host (cả hai nhánh có hit và không hit); báo cáo ghi `hiển thị / tổng` khi chạm trần 100 hàng; `MATCHES` với regex thật tìm được; `EXISTS` không bị che bởi giới hạn quét; `_regex_literal` chỉ thu hẹp, không bỏ sót; cửa sổ SPL lấy từ PoC.
- **Vận hành LLM (mục 7.5.4):** bộ đếm token thấy mô hình thực sự trả lời và dừng ở ngân sách; mô hình dự phòng thay thế và giữ nguyên; temperature bị bỏ khi mô hình từ chối; bỏ phiếu judge (đa số, không đa số); cache Prepare (trúng, trượt, làm mới, đổi mô hình, không lưu khi lỗi); che dữ liệu cả khi giá trị xuất hiện dạng thoát JSON và khôi phục ở phản hồi.
- **Đầu cuối:** chạy CLI trên một CSDL nhỏ ở chế độ `--offline`, và chạy CLI khi không có tệp `.env` nào.

Các bài này không gọi LLM thật; việc gọi thật được kiểm chứng bằng thực nghiệm ở Chương 7.

5.10. Cài đặt luồng Prepare-only

5.10.1. Thu thập tình báo (`intel/`)

`Fetcher` (`http.py`) là điểm duy nhất gọi mạng. Nó chỉ gửi GET, từ chối mọi URL không phải https hoặc có host ngoài ba host cho phép, giới hạn kích thước (2 MB cho JSON, 60 KB cho tệp văn bản) và thời gian (25 giây), từ chối nội dung nhị phân (có byte `\x00` trong 2 KB đầu) và lưu đệm 24 giờ trên đĩa để không đốt hạn mức 60 yêu cầu mỗi giờ của GitHub khi chưa có token. Khoá API chỉ được gắn vào yêu cầu tới host của nó. Chuyển hướng không được `requests` tự theo (`allow_redirects=False`): `Fetcher` tự đọc `Location`, kiểm tra lại https và host ở mỗi bước, tối đa ba bước; một chuyển hướng sang host lạ, hạ xuống `http://` hay lặp vô hạn đều bị từ chối. Quy tắc này được thêm sau khi rà soát phát hiện rằng việc kiểm tra host chỉ áp dụng cho URL đầu tiên.

`nvd_lookup` lấy mô tả, CVSS, CWE, sản phẩm và phiên bản bị ảnh hưởng (rút từ CPE) và cờ CISA KEV mà NVD mang sẵn. `github_search` tìm theo tên CVE, sắp theo số sao, bỏ kho fork; kho đủ `--min-stars` được lấy, nếu không có kho nào đủ thì lấy vài kho tốt nhất, gắn cờ `below_threshold` và cảnh báo tín hiệu yếu. `repo_material`

lấy cây tệp của kho bằng một lời gọi, loại tệp nhiễu (LICENSE, `node_modules`, ảnh, tệp lớn hơn 200 KB), chấm điểm (mẫu nuclei và tệp tên `exploit`/`poc`/`scan` cao hơn README) rồi đọc tối đa sáu tệp. Không có kho nào bị clone, cài hay chạy.

`extract_indicators` (`extract.py`) trích dấu vết bằng biểu thức chính quy: đường dẫn HTTP, tham số HTTP dạng `a.b.c.d` (như `class.module.classLoader...`), header tuỳ biến, chuỗi `${...:...}`, tên miền out-of-band đã biết, giao thức (`ldap://`), tên tệp, lệnh, User-Agent; mỗi loại giữ 12 giá trị xuất hiện nhiều nhất. Đây chỉ là *ứng viên*: có thể là đường dẫn trên máy chủ phụ của kẻ tấn công chứ không phải thứ nạn nhân nhận. `digest` (`gather.py`) gói sự thật về CVE, danh sách dấu vết và nội dung tệp (tệp khai thác và mẫu nuclei được ưu tiên trước README, gấp đôi chỗ) trong khoảng 16.000 ký tự, mỗi tệp đặt trong `<<< >>>` kèm nhãn không tin cậy.

5.10.2. Lập kế hoạch (`plan/build.py`)

`build_plan` gửi một lời nhắc gồm nhiệm vụ, định nghĩa ba mức `significance`, luật viết SPL (kèm danh mục nguồn và tên trường theo Splunk CIM), hình dạng JSON mong muốn, bản tóm tắt tình báo và phần văn bản của PEAK. Phản hồi được `assemble` chuyển thành các đối tượng đã kiểm tra: `significance` lạ thành `indicator`; mã kỹ thuật sai định dạng `Txxxx` bị bỏ; dấu vết được gắn `from_poc` nếu chuỗi có nguyên văn trong tình báo (hàm `grounded`), nếu không thì `inferred`; `data_source` ngoài bảy nguồn thì loại truy vấn; truy vấn qua `check_spl` và quy tắc tiến trình cha; chữ mô tả qua `_scrub` (ký tự lạ và từ lạ). Sau đó mã tự thêm điểm dừng theo `significance` (`default_stops`), truy vấn độ phủ cho mỗi nguồn có dùng, giới hạn và nhãn xuất xứ.

Nếu có vấn đề (JSON hỏng, truy vấn bị loại, thiếu giai đoạn `impact`, chữ lạ), lần sau gửi `REPAIR_PROMPT` gồm danh sách lỗi và câu trả lời trước, tối đa ba lượt, giữ bản có nhiều truy vấn hợp lệ nhất. Vấn đề chỉ về chữ (tiền tố `language:`) thì chỉ được một lượt viết lại, rồi kế hoạch được giữ và từ còn sót được liệt kê trong `dropped`, để không tốn thêm lời gọi cho lỗi không ảnh hưởng an toàn. Hết lượt mà chưa có truy vấn hợp lệ nào thì ném `PlanError` và lưu câu trả lời cuối ra `planner_last_reply.txt`. Lời nhắc sửa lỗi có các gợi ý cụ thể, bổ sung sau khi mô hình miễn phí viết dấu ngoặc kép lệch ở cả ba lượt sửa Spring4Shell và không còn truy vấn nào dùng được.

5.10.3. Bộ kiểm tra ngôn ngữ và quy tắc tiến trình cha

`language.foreign_words(text, known)` bỏ qua đoạn mã trong dấu huyền, URL, chuỗi có ký tự định danh (`/ \ . _ = : $ { } [ ] * | @ # 0-9`), từ viết tắt, tên riêng và từ dưới 4 chữ cái. Với các từ còn lại: chữ cái ngoài tiếng Việt và tiếng Anh thì gắn cờ ngay; từ khớp mẫu âm tiết tiếng Việt (`phụ âm đầu? + nguyên âm 1–3 + phụ âm cuối?`, sau khi bỏ dấu thanh) thì bỏ qua; từ có trong từ vựng tiếng Anh (khoảng 1.500 từ gốc cùng các biến cách `-s`, `-es`, `-ed`, `-ing`, `-ly`, `-er`) hoặc trong `known` (từ vựng của chính văn bản PoC/CVE) thì bỏ qua. Từ điển tiếng Anh do tác giả soạn, không dùng

danh sách tần suất của bên thứ ba vì giấy phép của chúng không rõ ràng cho một kho công khai.

`safety.parent_process_problem(spl, data_source)` chỉ áp dụng cho nguồn `endpoint`: tách các công đoạn bằng `_scan`, chỉ xét phần lọc (câu `search` đầu, `where`, `regex`, `search`), và chấp nhận khi có `parent_process[_name|_path]` đi với `=`, `IN`, hoặc `match(...)`/`like(...)`, hoặc khi truy vấn lọc theo `file_name`/`file_path`/registry. Chỉ liệt kê trường trong `stats ... by` hay `table` thì không đủ.

5.10.4. Xác minh và dòng lệnh

`verify` (`plan/verify.py`, khoảng 365 dòng) hiện thực các bước ở mục 4.9.4 bằng mã thuần. `extract_leads` chỉ đọc các truy vấn trúng; `_pivot_stages` dựng ba giai đoạn pivot từ mẫu cố định, mỗi giá trị đi qua `spl_literal`. `plan_cli.py` cung cấp ba lệnh: `plan` (thu thập, lập kế hoạch và ghi `plan.json`, `plan.md`, `queries.spl`, `result.template.json`, các tệp `peak_*.md` và hai schema), `verify` (đọc kế hoạch và kết quả, ghi `verification.md`, `verification.json` và nếu cần thư mục vòng kế tiếp `iterN/`) và `schema`. Cờ `--no-peak` bỏ PEAK để chạy nhanh và ít token; không cấu hình LLM thì `plan` vẫn thu thập và lưu tình báo rồi dừng với mã thoát 3, còn `verify` và `schema` không cần LLM. Phần mới có 48 bài kiểm thử (`test_intel_plan.py` 40 bài và `test_plan_language_parent.py` 8 bài), không gọi mạng thật hay LLM thật: dùng phiên HTTP giả và bộ gọi LLM kịch bản.

Chương 6. Hướng dẫn thực hành

Chương này hướng dẫn một người chưa biết hệ thống đi từ cài đặt đến chạy được cuộc săn đầu tiên, đọc kết quả, và tự viết PoC mới. Các lệnh ghi theo cú pháp Windows (PowerShell hoặc Git Bash) và đều đã được chạy thử trong quá trình thực hiện đồ án. Trên Linux hoặc macOS, thay `.venv\Scripts\python.exe` bằng `.venv/bin/python`.

6.1. Chuẩn bị môi trường

6.1.1. Yêu cầu

- Python 3.12 trở lên (PEAK Assistant không chạy trên bản thấp hơn).
- Git, để cài PEAK Assistant từ GitHub.
- Khoảng 2 GB dung lượng đĩa nếu nạp toàn bộ BOTS v1; chỉ vài MB nếu dùng mẫu nhỏ.
- Khoá API của một dịch vụ LLM tương thích OpenAI. **Đây là điều kiện tuỳ chọn**, xem mục 6.2.

6.1.2. Cài đặt lõi (không cần LLM)

```
git clone <địa-chỉ-kho> AI_Agent_Hunting
cd AI_Agent_Hunting
python -m venv .venv
.venv\Scripts\python.exe -m pip install -e ".[dev]"
```

Cài đặt này chỉ kéo theo `pydantic`, `pyyaml`, `requests`, cùng `pytest` và `ruff`. Kiểm tra:

```
.venv\Scripts\python.exe -m pytest tests -q
```

Kết quả mong đợi: `146 passed`.

6.1.3. Cài thêm PEAK Assistant (tuỳ chọn)

```
.venv\Scripts\python.exe -m pip install -e ".[peak]"
```

Lệnh này cài PEAK Assistant từ GitHub tại commit đã ghim `dfabbb0`, kéo theo AutoGen và nhiều thư viện khác nên mất vài phút. Kiểm tra bằng `python -c "import peak_assistant"`; không có lỗi nghĩa là đã cài xong.

6.2. Cấu hình LLM

6.2.1. Ba biến cần thiết

Sao chép `.env.example` thành `.env` rồi điền ba giá trị:

```
LLM_ENDPOINT=https://openrouter.ai/api/v1/chat/completions
LLM_API_KEY=<khoá-của-bạn>
LLM_MODEL=<tên-mô-hình>
```

`LLM_ENDPOINT` là địa chỉ đầy đủ tới `/chat/completions` của bất kỳ dịch vụ tương thích OpenAI nào (OpenRouter, OpenAI, một máy chủ cục bộ như Ollama hoặc vLLM). Có hai biến phụ: `LLM_TIMEOUT` (giây, mặc định 600) và `LLM_MAX_TOKENS` (mặc định 16000). Tệp `.env` đã nằm trong `.gitignore`; **không bao giờ** đưa khoá vào mã nguồn, báo cáo hay ảnh chụp màn hình.

6.2.2. Ba mức dùng LLM

Bảng 16. *Các chế độ chạy theo cấu hình LLM*

Chế độ	Cách bật	Prepare	Judge và advisor	Khi nào dùng
Không LLM	`--offline`, hoặc `.env` chưa điền	ABLE và kế hoạch dựng từ PoC	Không có	Chưa có khoá; dữ liệu nhạy cảm; chạy nhanh, tái lập
LLM mặc định	`.env` đã điền	PEAK Assistant	Có	Chạy đầy đủ
Ghi đè mô hình	`--model <tên>`	PEAK Assistant	Có	Thử mô hình khác mà không sửa `.env`

Hệ thống nhận biết trường hợp `.env` còn giá trị mẫu (`<your-key>`, `sk-or-...`) và chuyển sang chế độ không LLM với một dòng thông tin, không báo lỗi. Điều này đã được kiểm thử bằng bài `test_cli_runs_without_any_llm_configuration`.

6.2.3. Chọn mô hình miễn phí

Khi chưa muốn trả phí, OpenRouter có các mô hình đánh dấu `:free`. Danh sách thay đổi liên tục, nên cách chắc chắn là hỏi trực tiếp danh mục của dịch vụ rồi lọc theo giá bằng không:

```
curl https://openrouter.ai/api/v1/models
```

Trong kết quả, mô hình có `pricing.prompt` và `pricing.completion` cùng bằng `"0"` là miễn phí. Không phải mô hình nào cũng gọi được: một số bị giới hạn tốc độ, một số không trả JSON, một số không phải mô hình văn bản. Mục 7.5 trình bày kết quả thử thật và mô hình đã chạy trọn bốn PoC. Để dùng:

```
.venv\Scripts\python.exe main.py --poc-dir pocs --model "nvidia/nemotron-3-super-120b-
a12b:free"
```

Lưu ý: mô hình miễn phí thường có giới hạn số lượt gọi mỗi phút/ngày và có thể dùng dữ liệu gửi lên để huấn luyện; đừng gửi telemetry nhạy cảm tới chúng.

6.3. Chuẩn bị dữ liệu telemetry

6.3.1. Mẫu nhỏ để thử nhanh

```
.venv\Scripts\python.exe scripts/seed_botsv1_sample.py --db data/cdb_sample.sqlite
```

Lệnh tạo một CSDL SQLite với khoảng vài chục sự kiện đại diện cho các pha tấn công của BOTS v1. Đủ để kiểm tra luồng chạy, nhưng chưa đủ để thấy các hiện tượng thật như kết quả rỗng do thiếu nguồn.

6.3.2. Bộ dữ liệu đầy đủ

Kết quả ở Chương 7 dùng `data/botsv1_eval.sqlite` với 4.417.543 dòng. Tệp này lớn và nằm ngoài git. Để dựng lại cần tải các tệp CSV nén của BOTS v1 [15] vào `data/raw/`, rồi chạy:

```
.venv\Scripts\python.exe scripts/ingest_botsv1_eval.py   # Security log + sysmon + DNS
.venv\Scripts\python.exe scripts/ingest_http.py          # stream:http cho PoC Joomla
```

Các tập lệnh đọc theo luồng, không giữ toàn bộ CSV trong bộ nhớ, và chỉ giữ các loại sự kiện phục vụ đánh giá (đăng nhập 4624/4625, tạo tiến trình 4688, SMB 5140/5145, sysmon, DNS, HTTP). Lược đồ CSDL là một bảng `events` gồm các cột `timestamp`, `event_id`, `native_type`, `host`, `user`, `pid`, `ppid`, `cmdline`, `image`, `ip`, `port`, `domain`, `file_path`, `action`, `status`, `raw_ref`.

6.3.3. Kiểm tra dữ liệu có gì

Trước khi viết PoC, hãy xem CSDL thật sự chứa gì. Đây là bài học lớn nhất của đồ án (mục 7.4): nhiều PoC rỗng chỉ vì dữ liệu không có loại sự kiện đó.

```
.venv\Scripts\python.exe -c "from hunting.adapters import CdbAdapter;
print(CdbAdapter('data/botsv1_eval.sqlite').describe_data())"
```

Kết quả là bảng Markdown liệt kê từng cặp `native_type`/`event_id` với số dòng và khoảng thời gian; đây cũng chính là tài liệu mà PEAK nhận làm "dữ liệu cục bộ".

6.4. Chạy cuộc săn đầu tiên

6.4.1. Một PoC, không LLM

```
.venv\Scripts\python.exe main.py --poc pocs/poc-joomla-rce.json --offline --out
artifacts/runs/thu1
```

Màn hình in một khối cho mỗi PoC:

```
=== poc-joomla-rce  window=2016-08-10T21:36:00Z/2016-08-10T22:00:00Z
    verdict=MATCHED obs=200 -> INVESTIGATE_FURTHER (MEDIUM) | PEAK=no
```

Khi không dùng LLM, Joomla dừng ở `INVESTIGATE_FURTHER` vì không có judge nào đánh giá ngữ cảnh; với LLM, cùng PoC cho `ESCALATE_TO_IR` (MEDIUM). Hệ thống ghi rõ điều đó trong mục giới hạn của báo cáo.

6.4.2. Toàn bộ PoC, có LLM

```
.venv\Scripts\python.exe main.py --poc-dir pocs --out artifacts/runs/day-du
```

Mỗi PoC mất vài giây đến vài phút tuỳ mô hình; phần lớn thời gian là hai lượt gọi PEAK (ABLE và kế hoạch). Nếu thấy dòng `PEAK=no`, mở `recommendation.json` và đọc `peak.notes` để biết lý do quay về chế độ không LLM.

6.4.3. Các tuỳ chọn thường dùng

Bảng 17. *Tham chiếu nhanh tuỳ chọn dòng lệnh*

Tuỳ chọn	Tác dụng
`--poc FILE` (lặp được), `--poc-dir DIR`	Chọn PoC
`--window START/END`	Ghi đè cửa sổ thời gian của mọi PoC
`--db PATH`	Đường dẫn CSDL SQLite
`--env FILE`, `--model TÊN`	Tệp cấu hình LLM; ghi đè mô hình
`--offline`	Bỏ hoàn toàn PEAK và mọi lời gọi LLM
`--research`	Bật tác tử nghiên cứu của PEAK (cần máy chủ MCP)
`--peak-timeout GIÂY`	Hạn cho mỗi bước PEAK (mặc định 420)
`--out DIR`	Thư mục kết quả

6.5. Đọc và dùng báo cáo

Mở `recommendation.md` của một PoC. Nên đọc theo thứ tự sau, vì thứ tự đó phản ánh mức độ quan trọng đối với người quyết định.

1. **Dòng đầu: khuyến nghị và độ tin cậy.** Ví dụ `ESCALATE_TO_IR (MEDIUM)`. Đây là gợi ý, không phải quyết định.
2. **Lý do.** Mỗi lý do gắn với một sự kiện kiểm chứng được: số bước khớp, khoảng thời gian hit, nhận định judge.
3. **Giới hạn của kết luận.** Mục dễ bị bỏ qua nhất nhưng quan trọng nhất. Với kết quả rỗng, mục này nói rõ cửa sổ đã quét, rằng predicate là literal, liệt kê các loại sự kiện thực có trong nguồn (để bạn tự kiểm tra loại sự kiện cần tìm có trong đó không), và nêu các điều kiện phụ suy ra từ ABLE (host, chuỗi AND). Nếu có bước bị cắt ở 100 hàng, mục này ghi tổng số khớp.
4. **Bảng lựa chọn.** Các hành động khả dĩ xếp theo ưu tiên.
5. **Bằng chứng.** Bảng từng bước với số bản ghi nguồn trong cửa sổ, trong phạm vi lọc và số khớp (`hiển thị / tổng`, dấu ≥ nghĩa là quét chạm giới hạn nên tổng chỉ là cận dưới); mục "Phạm vi truy vấn thực tế"; các giá trị xoay trục (máy, IP, miền); vài hàng mẫu. Đây là chỗ để tự kiểm chứng.
6. **Phần do LLM viết** (judge, bước tiếp theo, câu hỏi, rủi ro, kế hoạch PEAK): luôn được gắn nhãn "chỉ mang tính tham khảo".

   Ba tình huống thường gặp và cách xử lý:

Bảng 18. *Từ khuyến nghị đến hành động của người săn*

Khi báo cáo nói	Người săn nên
`ESCALATE_TO_IR`	Mở bằng chứng, kiểm tra vai trò thật của máy ở "Top host" (nạn nhân hay cảm biến), tìm bước kiểm tra kết quả (status), rồi mới chuyển gói trong

Khi báo cáo nói	Người săn nên
	`ir/` cho IR
`COLLECT_DATA_THEN_RERUN`	Làm việc với đội hạ tầng để bật thu thập nguồn bị thiếu; **không** đóng cuộc săn
`CLOSE_WITH_CAVEAT`	Đọc từng giới hạn; nếu chấp nhận được thì đóng và lưu lại các giới hạn như ghi chú tri thức
`INVESTIGATE_FURTHER`	Xem bước nào không khớp và vì sao; bổ sung bước hoặc mở rộng cửa sổ
`TUNE_POC_OR_CLOSE`	Thêm điều kiện loại trừ cho hoạt động hợp lệ, hoặc đóng PoC

6.6. Viết PoC mới

6.6.1. Cấu trúc

Một PoC là tệp JSON. Hãy xem ví dụ đầy đủ tại Phụ lục A. Các trường bắt buộc của cổng Prepare: `topic`, `able` (đủ `behavior`, `location`, `evidence`; `actor` được để trống), `research_refs`, `scope`, `max_duration`, `plan`. Các trường thực thi: `steps` (bắt buộc), `fallbacks`, `references`, `expected_chain`, `time_window`.

6.6.2. Quy trình năm bước

1. **Phát biểu giả thuyết cụ thể và kiểm chứng được.** Tốt: "một địa chỉ IP gửi nhiều yêu cầu có chuỗi `acunetix` tới site Joomla". Kém: "có ai đó tấn công website".
2. **Xem dữ liệu có gì** (mục 6.3.3). Nếu không có nguồn nào chứa dấu hiệu bạn tìm, hãy dừng và coi đó là phát hiện về khoảng trống dữ liệu.
3. **Cẩn thận với `able.location`.** Nếu trường này chứa một token giống tên máy (ví dụ `we1149srv`), **mọi bước sẽ chỉ tìm trên máy đó** (báo cáo ghi rõ trong mục phạm vi truy vấn). Chỉ ghi tên máy khi bạn thật sự muốn giới hạn, và kiểm tra bằng `describe_data`/bảng độ phủ rằng máy đó thực sự ghi nguồn bạn cần: nhật ký web trong BOTS v1 nằm dưới tên máy thu log `splunk-02`, không phải máy client. Tương tự, các chuỗi cụ thể (tên tệp, cờ `-enc`, IP, chuỗi trong ngoặc kép) trong `able.behavior`/`able.evidence` được AND vào mọi bước.
4. **Chọn trường và toán tử.** Dùng `EQUALS` cho giá trị chính xác (tên tệp, tên miền); `CONTAINS` cho chuỗi con; `MATCHES` khi cần biểu thức chính quy; `EXISTS` khi chỉ cần trường có giá trị.
5. **Thêm một bước kiểm tra kết quả nếu dữ liệu có.** Bước trên trường `status`, `action`, `result` hoặc `response` cho phép hệ thống nâng độ tin cậy khi leo thang; thiếu bước này, giới hạn "chưa chứng minh thành công" sẽ luôn xuất hiện.
6. **Đặt `time_window` hợp lý** và chạy thử với `--offline` trước, rồi mới bật LLM.

6.6.3. Chạy thử ví dụ

Kho có sẵn `pocs/examples/poc-web-scanner-acunetix.json`. Chạy:

```
.venv\Scripts\python.exe main.py --poc pocs/examples/poc-web-scanner-acunetix.json --offline
```

Kết quả khi chạy thật trên BOTS v1: 204 bản ghi hiển thị, 3 trên 3 bước có kết quả (100 / ≥1.999, 6 và 100 / ≥2.000), tất cả từ địa chỉ 40.80.148.42, hit đầu tiên lúc 21:36:45; khuyến nghị `INVESTIGATE_FURTHER` (MEDIUM) vì ở chế độ `--offline` không có judge. Bước tìm `acunetix` chỉ có 6 hàng, đủ để nhận diện máy quét.

6.6.4. Những lỗi người mới hay mắc

Bảng 19. *Lỗi thường gặp khi viết PoC*

Triệu chứng	Nguyên nhân thường gặp	Cách sửa
PoC bị từ chối trước khi chạy	Thiếu trường của cổng Prepare	Đọc thông báo; điền trường thiếu
Kết quả luôn rỗng	Sai trường hoặc sai dạng giá trị (hoa thường không thành vấn đề); hoặc bị lọc theo host ngầm	Xem mục "Phạm vi truy vấn thực tế" và cột "Trong phạm vi lọc" của báo cáo; chạy `describe_data`, nhìn mẫu hàng thật trong CSDL
`EQUALS` không khớp đường dẫn đầy đủ	Giá trị ghi cả đường dẫn nhưng tên tệp trong dữ liệu khác	Dùng chỉ tên tệp (`powershell.exe`)
Quá nhiều hit vô nghĩa	Dùng `CONTAINS` với chuỗi quá ngắn	Dùng `EQUALS` hoặc `MATCHES` chính xác hơn, thêm bước loại trừ
`COLLECT_DATA_THEN_RERUN`	Nguồn đúng là không có dữ liệu trong cửa sổ	Kiểm tra `source_kind` và `time_window` trước khi kết luận

6.7. Dùng với Splunk thật

Adapter Splunk trực tiếp không còn trong nhánh chính; nó chỉ còn trong lịch sử git (commit `4dace56`; xem bằng `git show 4dace56:src/hunting/adapters/splunk_adapter.py`) cùng các tham số `--provider splunk`, `--splunk-url`, `--splunk-user`, `--splunk-index` và biến `SPLUNK_PASSWORD`. Cần lưu ý trung thực: adapter đó chưa từng chạy trên Splunk thật trong đồ án, chưa có `source_presence` nên độ phủ nguồn được ghi là "không kiểm tra được" và kết quả rỗng có độ tin cậy `LOW`. Với luồng Prepare-only (mục 6.10) không cần adapter nào: đội thực thi tự chạy các truy vấn `queries.spl` trên Splunk của họ.

6.8. Xử lý sự cố

Bảng 20. *Sự cố thường gặp và cách xử lý*

Hiện tượng	Giải thích	Xử lý
`[i] LLM is optional and not available (...)` rồi chạy tiếp	`.env` thiếu hoặc còn giá trị mẫu	Bình thường; điền khoá nếu muốn dùng LLM
`PEAK Assistant is not`	Chưa cài extra `peak`	`pip install -e ".[peak]"`

Hiện tượng	Giải thích	Xử lý
`installed`		
`PEAK=no` kèm ghi chú `HTTP 404 model_not_found`	Endpoint trả lỗi thoáng qua	Đã tự gọi lại ba lần; nếu vẫn lỗi, thử `--model` khác
`HTTP 401`	Khoá sai hoặc hết hạn	Kiểm tra `LLM_API_KEY` (không in khoá ra màn hình)
`HTTP 429`	Vượt giới hạn tốc độ, hay gặp với mô hình miễn phí	Đợi hoặc đổi mô hình; hệ thống đã gọi lại có chờ
Hết thời gian ở bước PEAK	Mô hình chậm	Tăng `--peak-timeout` hoặc `LLM_TIMEOUT`
Advisor ghi "không dùng được"	Mô hình trả JSON sai ba lần	Khuyến nghị theo luật vẫn đúng; thử mô hình khác
Không tìm thấy `botsv1_eval.sqlite`	Tệp ngoài git	Dựng theo mục 6.3.2 hoặc dùng `--db data/cdb_sample.sqlite`

6.9. Quy trình tái lập kết quả của luận văn

Để chạy lại thực nghiệm của Chương 7: dựng CSDL (6.3.2), điền `.env`, rồi chạy `main.py --poc-dir pocs --out artifacts/runs/tai-lap`. Phần tất định (số bản ghi khớp, độ phủ) phải giống hệt; phần LLM (văn bản ABLE, kế hoạch, nhận định judge, số token) sẽ khác từng lần, chỉ kết luận (disposition, độ tin cậy) được kỳ vọng ổn định. Thư mục `results/botsv1-peak-run/` lưu bản chạy đã dùng trong luận văn.

6.10. Chạy luồng Prepare-only

6.10.1. Lập kế hoạch từ một CVE

Cần `.env` có LLM (xem 6.2). Không cần dữ liệu telemetry hay Splunk.

```
python main.py plan --cve CVE-2022-22965 --min-stars 200 --out artifacts/plans
python main.py plan --cve CVE-2022-22965 --min-stars 200 --no-peak   # nhanh, ít token hơn nhiều
python main.py plan --repo owner/name                                  # từ một kho PoC cụ thể
python main.py plan --query "log4j poc" --min-stars 500                 # tìm theo từ khoá
```

Chú ý hạn mức GitHub: không có `GITHUB_TOKEN` thì chỉ 60 yêu cầu mỗi giờ (một lần lập kế hoạch tốn khoảng 23 yêu cầu, có lưu đệm). Có thể đặt `GITHUB_TOKEN` và `NVD_API_KEY` trong `.env`.

Thư mục kết quả `artifacts/plans/<CVE>/` có `intel.json` và `intel_digest.md` (đúng đoạn văn bản LLM đã đọc), hai schema, và `iter1/` với `plan.md` (bản cho người), `plan.json` (bản cho máy), `queries.spl`, `result.template.json`, `peak_able.md`, `peak_hunt_plan.md`.

6.10.2. Đọc `plan.md` trước khi giao đi

Bộ kiểm tra chỉ bảo đảm an toàn và hình thức, không bảo đảm truy vấn bắt đúng tấn công. Người săn nên kiểm tra: dấu vết có thực sự là thứ nạn nhân nhận được không

(đường dẫn của ứng dụng minh hoạ trong PoC thường không có ở hệ thống thật); truy vấn có dùng nhật ký mà môi trường thực sự ghi (nhật ký web thường không ghi thân POST); truy vấn hậu khai thác có lọc theo tiến trình cha đúng với dịch vụ không; mục "Đã loại bỏ / ghi chú" ở cuối để biết truy vấn nào bị loại và từ lạ nào còn sót.

6.10.3. Giao cho đội thực thi, rồi xác minh

Đội thực thi gắn placeholder bằng hàm `bind` (ánh xạ nguồn sang chỉ mục thật; ánh xạ thiếu hoặc có ký tự lạ bị từ chối), chạy các truy vấn độ phủ `C-*` trước rồi các truy vấn còn lại, rồi điền `result.template.json` thành một `ResultBundle`. Sau đó:

```
python main.py verify --plan artifacts/plans/CVE-2022-22965/iter1/plan.json --results
results.json
```

Kết quả là `verification.md` kèm một trong các quyết định ở mục 4.9.4. Với `REFINE`, thư mục `iter2/` chứa kế hoạch pivot để giao đội thực thi lần nữa. Lệnh `main.py schema --out schemas/` xuất hai JSON Schema để hai đội thống nhất hợp đồng.

Bảng 21. *Từ quyết định của* **`verify`** *đến việc cần làm*

Quyết định	**Việc cần làm**
`ESCALATE_AFFECTED`	Chuyển IR kèm bằng chứng; không mở rộng thêm
`ACCEPT_NO_EVIDENCE`	Ghi nhận "không thấy trong phạm vi đã quét"; không báo là sạch
`COLLECT_DATA`	Bổ sung telemetry cho nguồn rỗng rồi chạy lại
`RERUN_INCOMPLETE`	Chạy lại truy vấn lỗi hoặc chưa chạy (kể cả giai đoạn không có truy vấn hợp lệ)
`REFINE`	Giao `iterN/` cho đội thực thi để pivot
`STOP_REVIEW`, `REJECT_RESULTS`	Người đọc quyết định (hết vòng, hết giá trị pivot, hoặc kết quả thuộc kế hoạch khác)

Chương 7. Thực nghiệm và đánh giá

Chương này trình bày các lần chạy thật của hệ thống. Mọi con số lấy từ tệp kết quả trong `results/botsv1-peak-run/` (lần chạy cuối bằng mô hình miễn phí Nemotron) và `artifacts/runs/` (các lần chạy `auto` và so sánh, không đưa vào kho mã), hoặc từ lệnh kiểm tra chạy lại khi viết chương này. Phần nào chưa kiểm chứng được sẽ được nêu rõ ở mục 7.9.

7.1. Thiết lập thực nghiệm

Dữ liệu. CSDL `data/botsv1_eval.sqlite` có 4.417.543 sự kiện trải trên 28 ngày (từ 2016-08-01 đến 2016-08-28), khoảng 154 nghìn dòng mỗi ngày, trừ hai ngày cao hơn là 10/8 (179 nghìn, ngày tấn công Joomla) và 24/8 (224 nghìn, ngày có dữ liệu DNS).

Bảng 22. *Thành phần của CSDL đánh giá*

Loại sự kiện (`native_type/event_id`)	**Số dòng**
`process_creation` / 4688	3.642.895
`authentication` / 4624 (đăng nhập **thành công**)	579.580
`smb` / 5140	63.859
`web_request` (`stream:http`)	39.010
`smb` / 5145	55.023
`dns` (`stream:dns`, chỉ ngày 24/8)	35.904
`smb` / 4648	1.206
`process_creation` / 1 (sysmon)	66

Điểm cần chú ý ngay: không có sự kiện đăng nhập thất bại (4625), không có tệp, và DNS chỉ có trong một ngày. Điều này quyết định kết quả của ba trong bốn PoC.

Bốn PoC. Đây là các PoC có sẵn trong kho, không được viết lại để "chạy được".

Bảng 23. *Bốn PoC thực nghiệm*

PoC	**Giả thuyết**	**Kỹ thuật ATT&CK**	**Vị từ chính**	**Cửa sổ**
`poc-joomla-rce`	Quét và khai thác Joomla trên site nạn nhân	T1190	`domain EQUALS imreallynotbatman.com` và `cmdline CONTAINS /joomla/` (web)	10/8, 21:36–22:00
`poc-bruteforce-we1149srv`	Dò mật khẩu tài khoản quản trị	T1110	`cmdline CONTAINS Logon Failed` và `user EQUALS admin` (authentication)	21/8, cả ngày
`poc-c2-beacon-networkfilter`	Beacon tới tên miền C2	T1071.001	`cmdline CONTAINS ad.networkfilter.co` và `/banner/` (web)	21/8, cả ngày
`poc-pdf-exploit-`	PowerShell mã	T1059.001	`image EQUALS`	21/8, cả ngày

PoC	Giả thuyết	Kỹ thuật ATT&CK	Vị từ chính	Cửa sổ
enc	hoá, ẩn cửa sổ sau khi mở PDF độc		powershell.exe, cmdline CONTAINS -enc, -w hidden (process)	

Mô hình LLM và lần chạy. Ba cấu hình được so sánh: (i) `--offline`, không LLM; (ii) `LLM_MODEL=auto` của OpenRouter (dịch vụ tự chọn mô hình mỗi yêu cầu; một lần nhận thấy được chọn là `deepseek/deepseek-v4.1-flash`); (iii) `nvidia/nemotron-3-super-120b-a12b:free`, mô hình miễn phí. Cùng PEAK Assistant tại commit `dfabbb0`, cùng dữ liệu, cùng PoC. Thực nghiệm chạy trên một máy Windows 11, Python 3.12.

7.2. Kết quả chính

Các kết quả dưới đây là của bản hệ thống đã sửa ba lỗi phát hiện ở lượt rà soát cuối (mục 7.4, lỗi 8–10). Bản chạy trước đó cho các kết luận trùng ở bốn PoC nhưng ba PoC rỗng đúng **vì lý do sai**; điểm này được phân tích ở mục 7.2.2.

Bảng 24. *Kết quả bốn PoC với mô hình* `auto`

PoC	Bản ghi hiển thị	Số khớp tổng	Bước khớp	Khuyến nghị	Tin cậy	Judge
poc-joomla-rce	200	≥1.999 và ≥2.000 (quét chạm giới hạn)	2/2	ESCALATE_TO_IR	MEDIUM	TRUE_POSITIVE, 0,85
poc-bruteforce-we1149srv	0	0	0/2	CLOSE_WITH_CAVEAT	MEDIUM	không chạy (không có hit)
poc-c2-beacon-networkfilter	0	0	0/2	CLOSE_WITH_CAVEAT	MEDIUM	không chạy
poc-pdf-exploit-enc	0	0	0/3	CLOSE_WITH_CAVEAT	MEDIUM	không chạy

Ghi chú: trong tệp JSON, trường `judge` của ba PoC rỗng được ghi `NO_SIGNAL` với độ tin cậy 0, là giá trị mặc định khi không có hàng để đánh giá, không phải nhận định thật của LLM.

Phần tất định ổn định: cả ba cấu hình (không LLM, `auto`, Nemotron miễn phí) cho cùng số bản ghi hiển thị (200 và ba lần 0) và cùng cơ cấu độ phủ.

7.2.1. PoC Joomla: có hit, kết luận ở mức vừa phải

Hệ thống giữ 200 bản ghi làm bằng chứng (100 cho mỗi bước), tất cả từ một địa chỉ 40.80.148.42 tới `imreallynotbatman.com`, trong khoảng 21:36:45 đến 21:40:57. Số

khớp thật lớn hơn nhiều: mỗi bước có ít nhất khoảng 2.000 hàng khớp trong cửa sổ 24 phút (lượt quét dừng ở giới hạn 2.000 hàng nên đó chỉ là cận dưới). Phiên bản trước báo "199 bản ghi", một con số do giới hạn 100 hàng và một hàng bị bộ lọc loại, và không cho người đọc biết điều đó; báo cáo hiện ghi `100 / ≥1999` và `100 / ≥2000`. Các hàng mẫu chứa `/acunetix-wvs-test-for-some-inexistent-file`, đường dẫn ngẫu nhiên tám ký tự và các thành phần `/joomla/index.php/component/search/`: đặc trưng của máy quét Acunetix duyệt thành phần Joomla.

Khuyến nghị là `ESCALATE_TO_IR` nhưng **MEDIUM, không phải HIGH**, và lý do được ghi thẳng trong báo cáo: dữ liệu web chỉ có miền và đường dẫn, không có mã trạng thái, và PoC không có bước nào kiểm tra kết quả, nên chỉ chứng minh được hoạt động quét, chưa chứng minh khai thác thành công. Mục "Rủi ro nếu quyết định sai" do advisor viết nêu đúng điểm này và còn gợi ý rằng mẫu Acunetix có thể là một đợt quét hợp pháp. Đây là hành vi mong muốn: kết luận thận trọng hơn cái mà một nhãn "MATCHED" đơn thuần gợi ra. Báo cáo cũng nhắc người dùng kiểm tra vai trò thật của máy `splunk-02`, vốn là máy ghi log chứ không phải nạn nhân.

Lưu ý lịch sử: phiên bản đầu của bộ luật cho kết quả `HIGH` với cùng bằng chứng. Việc đọc báo cáo đầu tiên mới làm lộ ra rằng độ tin cậy này không có căn cứ; luật trần `outcome_observed` được thêm vào sau đó (mục 4.6.3).

7.2.2. Ba PoC rỗng: không phải "sạch", và không hề được tìm đúng chỗ

Đây là phát hiện quan trọng nhất của lượt rà soát cuối. Cả ba PoC rỗng có `able.location` chứa tên máy `we1149srv` ("we1149srv authentication log", "egress web traffic from we1149srv", "we1149srv and workstations opening email attachments"). Hàm suy ra phạm vi ABLE biến tên máy đó thành bộ lọc `host = we1149srv` cho **mọi bước**, và điều này không hiện ra trong predicate, báo cáo hay tài liệu. Khi đếm độ phủ trong đúng phạm vi đó, cả ba đều bằng không.

Bảng 25. *Phạm vi truy vấn và độ phủ của ba PoC rỗng (cửa sổ 21/8)*

PoC	Nguồn	Bản ghi nguồn trong cửa sổ	Trong phạm vi `host = we1149srv`	Host thực sự ghi nguồn	Chạy lại không lọc host
`poc-bruteforce-we1149srv`	authentication	20.657	**0**	we9748srv (218), we5364srv (107), we1864srv (75)	0 bản ghi
`poc-c2-beacon-networkfilter`	web	566	**0**	splunk-02 (566)	0 bản ghi
`poc-pdf-exploit-enc`	process	129.775	**0**	we9748srv (1.335), we5364srv (630),	0 bản ghi

PoC	Nguồn	Bản ghi nguồn trong cửa sổ	Trong phạm vi `host = we1149srv`	Host thực sự ghi nguồn	Chạy lại không lọc host
				we1864srv (537)	

Với PoC `c2-beacon`, vấn đề mang tính cấu trúc: toàn bộ 39.010 dòng web trong CSDL có `host = splunk-02` (máy thu log), nên bộ lọc theo máy client không thể khớp bất kỳ dòng web nào, kể cả khi beacon có thật. Phiên bản trước báo "nguồn web có 566 bản ghi trong cửa sổ" (đúng nhưng đếm mọi host) trong khi phạm vi tìm kiếm thực tế là 0 dòng; kết luận `CLOSE_WITH_CAVEAT` tình cờ đúng chỉ vì dữ liệu cũng không chứa `networkfilter` ở bất cứ đâu.

Bản đã sửa xử lý phạm vi rỗng thành một trường hợp riêng. Hệ thống đếm độ phủ cả toàn cửa sổ lẫn trong phạm vi host, liệt kê host thực sự ghi nguồn, rồi chạy lại cùng PoC một lần **không lọc host** (thư mục `unscoped_probe/`). Cả ba lần chạy lại đều cho 0 bản ghi, nên khuyến nghị là `CLOSE_WITH_CAVEAT` (MEDIUM) với lý do ghi đủ hai vế: dữ liệu không chứa dấu hiệu ở bất kỳ host nào, và host nêu trong PoC không ghi nguồn này nên PoC chưa từng được thử trên host đó. Kết luận cuối vẫn là `CLOSE_WITH_CAVEAT` như trước, nhưng giờ có lý do đúng và có đo.

Độc lập với phạm vi host, truy vấn trực tiếp trên toàn CSDL (mọi host, mọi ngày) cho 0 dòng với từng chuỗi tín hiệu cần tìm:

Bảng 26. *Dấu hiệu cần tìm có trong dữ liệu không (toàn CSDL 4.417.543 dòng)*

PoC	Dấu hiệu	Số dòng khớp	Loại sự kiện thực có trong nguồn (cửa sổ 21/8)
`poc-bruteforce-we1149srv`	`Logon Failed`; `user = admin`	0; 0	chỉ `authentication/4624` (đăng nhập thành công), không có 4625
`poc-c2-beacon-networkfilter`	`networkfilter`	0	`web_request` (chỉ có miền và đường dẫn)
`poc-pdf-exploit-enc`	`powershell.exe` cùng `-enc`; `-w hidden`	0; 0	`process_creation/4688` (các dòng `splunk-powershell.exe` là công cụ của Splunk)

Cột cuối của bảng hiện nay cũng được hệ thống tự liệt kê trong báo cáo (mục "giới hạn của kết luận"), nên người săn không phải tự truy vấn để biết rằng nguồn xác thực chỉ có đăng nhập thành công. Hệ thống vẫn **chưa tự đối chiếu** loại sự kiện này với predicate; nó chỉ trưng bày để người săn đối chiếu (mục 8.3).

Mỗi PoC chỉ quét một ngày trong 28 ngày (3,6% khoảng dữ liệu), nên kể cả loại sự kiện có trong dữ liệu, các ngày khác vẫn chưa được xem. Người đọc nên coi kết quả `CLOSE_WITH_CAVEAT` ở đây là "không tìm thấy trong phạm vi đã quét", đúng như nhãn.

7.3. So sánh ba cấu hình

Bảng 27. *Cùng bốn PoC dưới ba cấu hình*

PoC	`--offline`	`auto`	Nemotron miễn phí
`poc-joomla-rce`	`INVESTIGATE_FURTHER` (MEDIUM)	`ESCALATE_TO_IR` (MEDIUM)	`ESCALATE_TO_IR` (MEDIUM)
`poc-bruteforce-we1149srv`	`CLOSE_WITH_CAVEAT`	`CLOSE_WITH_CAVEAT`	`CLOSE_WITH_CAVEAT`
`poc-c2-beacon-networkfilter`	`CLOSE_WITH_CAVEAT`	`CLOSE_WITH_CAVEAT`	`CLOSE_WITH_CAVEAT`
`poc-pdf-exploit-enc`	`CLOSE_WITH_CAVEAT`	`CLOSE_WITH_CAVEAT`	`CLOSE_WITH_CAVEAT`

Ba nhận xét. Thứ nhất, phần quyết định giữa ba cấu hình khác nhau ở đúng một chỗ: judge đóng góp vào Joomla. Không có LLM, hệ thống nói trung thực rằng "chuỗi khớp đủ nhưng chưa ai đánh giá ngữ cảnh" và dừng ở `INVESTIGATE_FURTHER`. Thứ hai, hai mô hình LLM rất khác nhau về nguồn gốc cho cùng kết luận, và độ tin cậy judge dao động (0,85 và 0,92 trong lần chạy này; các lần chạy trước từng ra 0,88, 0,92, 0,97), điều này đúng với dự đoán rằng judge không ổn định và nên chỉ dùng làm tham khảo. Thứ ba, ba PoC rỗng không đổi, vì judge không chạy khi không có hàng nào, và luật rỗng không phụ thuộc LLM.

Về chi phí gọi LLM do judge và advisor (số lời gọi và token do bộ đo của hệ thống ghi lại, không tính phần bên trong PEAK vì PEAK không báo token):

Bảng 28. *Số lời gọi và token của judge/advisor*

PoC	`auto`: lời gọi / token	Nemotron: lời gọi / token
`poc-joomla-rce`	2 / 9.524	2 / 7.280
`poc-bruteforce-we1149srv`	1 / 7.004	1 / 4.814
`poc-c2-beacon-networkfilter`	1 / 6.695	1 / 6.362
`poc-pdf-exploit-enc`	1 / 8.087	1 / 4.949

Token tăng so với bản trước vì advisor nhận thêm phạm vi truy vấn, cơ cấu loại sự kiện và kết quả chạy lại. Phần chiếm nhiều thời gian nhất và không được đo token là hai lượt gọi PEAK (ABLE và kế hoạch), mỗi lượt là một cuộc hội thoại đa tác tử nhiều lượt. Đây là hạn chế đo lường ghi ở mục 8.3.

7.4. Các lỗi đã phát hiện nhờ thực nghiệm

Giá trị lớn của việc chạy thật nằm ở những thứ kiểm thử đơn vị không bắt được. Bảng sau liệt kê chúng theo thứ tự thời gian.

Bảng 29. *Lỗi phát hiện khi chạy thật và cách xử lý*

#	Hiện tượng	Nguyên nhân	Xử lý
1	`image EQUALS powershell.exe` cho 100 hit trên dữ liệu sạch	Tầng tìm `LIKE` khớp cả `splunk-powershell.exe`	Bộ lọc chính xác ở tầng hai, mọi toán tử (từ đánh giá trước đó; có

#	Hiện tượng	Nguyên nhân	Xử lý
			bài hồi quy)
2	Bước `EXISTS` với giá trị rỗng khớp hàng đầu tiên	Không có điều kiện giá trị	`EXISTS` rỗng trả 0 hàng
3	Báo `Event loop is closed` sau mỗi lần chạy	Ứng dụng khách HTTP đóng sau vòng lặp	`run_async` và bộ xử lý ngoại lệ im lặng
4	Lỗi 404 `model_not_found` thoáng qua	Endpoint chuyển yêu cầu tới nhà cung cấp, đôi lần trả 404	Gọi lại có backoff; đo lại 1/15 (mục 7.5)
5	Độ tin cậy HIGH cho kết quả chưa chứng minh được thành công	Luật chưa tách quét và khai thác	Trần `outcome_observed`
6	Advisor trả JSON sai hoặc danh sách rỗng ở 2/4 PoC với `auto`	Mô hình không tuân thủ định dạng	Thử tới 3 lần, chấp nhận nhiều dạng, ghi lý do khi thất bại
7	`able_table` của PEAK nuốt lỗi thành chuỗi `Error while generating...`	Cách PEAK xử lý ngoại lệ	Nhận ra chuỗi lỗi và coi là lỗi thật để thử lại
8	Ba PoC rỗng bị lọc ngầm theo `host = we1149srv`, host không ghi nguồn tương ứng; độ phủ báo 20.657/566/129.775 bản ghi trong khi phạm vi thực tế là 0	Host suy ra từ `able.location` không hiển thị trong predicate hay báo cáo	Hiển thị phạm vi; đếm độ phủ trong phạm vi; chạy lại không lọc host; luật riêng (mục 7.2.2)
9	`MATCHES` với regex thật (`acunet.x`) cho 0 hàng dù `acunetix` cho 6; `EXISTS` dùng tên trường làm từ khoá	Tầng truy xuất đưa nguyên regex vào `LIKE`	Thu hẹp bằng đoạn literal bắt buộc; `EXISTS` đẩy xuống SQL
10	Joomla báo 199 bản ghi (bước 1 chỉ 99) mà không nói đã chạm giới hạn; SPL dùng `earliest=-14d` cố định	Chỉ lấy 100 hàng rồi mới lọc; cửa sổ SPL không lấy từ PoC	Quét 2.000 hàng rồi lọc, báo `hiển thị / tổng`; SPL dùng cửa sổ của PoC

Lỗi số 6 đáng nói thêm. Trước khi sửa, hai trong bốn PoC không có phần bước tiếp theo, và không có cảnh báo nào cho biết LLM đã trả sai: báo cáo trông "hoàn chỉnh" nhưng thiếu một phần. Sau khi sửa, cả bốn PoC có đủ 5 bước, 5 câu hỏi và 5 rủi ro với `auto`; với Nemotron là 5/5/5, 5/3/5, 5/5/5 và 5/5/5 (bruteforce, c2, Joomla, pdf-exploit). Lần chạy cuối không có traceback nào trong nhật ký.

Ba lỗi 8–10 được tìm thấy ở lượt rà soát cuối bằng cách đối chiếu mô tả trong luận văn với mã nguồn rồi truy vấn thẳng vào dữ liệu, không phải qua kiểm thử đơn vị. Lỗi 8 đặc biệt đáng nói: kết luận của ba PoC không đổi sau khi sửa, nên một người chỉ nhìn bảng kết quả sẽ không thấy gì khác; sai chỉ nằm ở chỗ PoC chưa từng được tìm

đúng chỗ, và chỉ lộ ra khi so cột "bản ghi nguồn trong cửa sổ" với số bản ghi trong phạm vi. Mỗi lỗi có bài hồi quy riêng (`tests/unit/test_scope_and_caps.py`, 15 bài).

7.5. Gọi LLM: miễn phí và độ tin cậy của endpoint

7.5.1. Đo tỷ lệ lỗi 404

Để chẩn đoán lỗi 404, endpoint chat-completions được gọi trực tiếp 15 lần với mô hình mặc định ban đầu: 14 lần HTTP 200 và 1 lần HTTP 404 `model_not_found` (nhà cung cấp Meta, mô hình `meta/muse-spark-1.3-contributor`). Tỷ lệ khoảng 1/15 (6,7%) nhỏ, nhưng một lần chạy PoC gồm nhiều lời gọi (kể cả hàng chục lời gọi nội bộ của các tác tử PEAK; số này chưa được đo), nên xác suất không gặp lỗi nào giảm nhanh. Ví dụ minh hoạ: với 6,7% mỗi lời gọi và giả sử 30 lời gọi, xác suất không lỗi chỉ khoảng 12%. Đây là lý do cơ chế gọi lại cần thiết chứ không phải tuỳ chọn.

Đổi sang `auto` giải quyết nguồn gốc lỗi vì không còn bị buộc vào một nhà cung cấp hay gặp sự cố. Thử sáu mô hình thay thế: chỉ `auto` chạy ổn định 3/3 lần; năm mô hình còn lại báo "not a valid model ID" trên endpoint hiện tại.

7.5.2. Thăm dò mô hình miễn phí

Danh mục OpenRouter được truy vấn qua `/models`, lọc các mô hình có giá prompt và completion bằng 0 (hai kết quả `lyria` là mô hình âm nhạc và một kết quả `content-safety` là mô hình kiểm duyệt nên bị loại khỏi thử nghiệm). Với 17 mô hình còn lại, mỗi mô hình được gọi một lần với yêu cầu trả đúng một đối tượng JSON ngắn (giới hạn 200 token).

Bảng 30. *Kết quả thử một lần các mô hình miễn phí*

Kết quả	Số mô hình	Mô hình
HTTP 200 và trả JSON hợp lệ	6	`nvidia/nemotron-3-super-120b-a12b:free`, `nvidia/nemotron-3-ultra-550b-a55b:free`, `nvidia/nemotron-3-nano-omni-30b-a3b-reasoning:free`, `poolside/laguna-s-2.1:free`, `cohere/north-mini-code:free`, `liquid/lfm-2.5-2.6b:free`
HTTP 200 nhưng không có JSON trong giới hạn 200 token	5	`openrouter/free`, `apodex/apodex-1.1-mini:free`, `inclusionai/ling-3.0-flash-sante:free`, `dots-studio/dots-3-note-preview:free`, `nvidia/nemotron-3.5-lightning:free`
HTTP 429 (nhà cung cấp quá tải hoặc giới hạn tốc độ)	4	`inclusionai/ling-3.1-flash`, `poolside/laguna-xs-2.1:free`, `google/gemma-4-26b-a4b-it:free`, `google/gemma-4-31b-it:free`
HTTP 403 (chỉ dùng trong môi	2	`thinkingmachines/inkling:free`,

Kết quả	Số mô hình	Mô hình
trường tác tử riêng)		`thinkingmachines/inkling-small:free`

Hai điều kiện cần đọc cẩn thận. Một: nhóm "HTTP 200 nhưng không có JSON" có thể là mô hình suy luận dùng hết 200 token để nghĩ trước khi trả lời, nên thử nghiệm này không đủ để kết luận chúng vô dụng. Hai: đây là một lần thử, chưa lặp; HTTP 429 có thể biến mất vào lúc khác. Kết quả chỉ cho biết "gọi được ngay lúc này".

7.5.3. Chạy trọn bốn PoC với một mô hình miễn phí

`nvidia/nemotron-3-super-120b-a12b:free` được chọn và chạy trọn bốn PoC (`--model`), hai lần: một lần trước và một lần sau khi sửa phạm vi truy vấn; số liệu ở chương này là của lần sau. Kết quả: cả bốn PoC cho cùng khuyến nghị như `auto`, PEAK được dùng ở cả bốn (`used_peak = true`), không có lần gọi lại nào phải ghi nhận, không có traceback. Judge cho Joomla là TRUE_POSITIVE 0,92. Bảng ABLE và kế hoạch của PEAK vẫn tuân thủ cấu trúc và nhắc đúng nguồn dữ liệu có trong CSDL (ví dụ ABLE của Joomla nói rằng telemetry có `web_request` với trường `domain` và `cmdline`, và gợi ý pivot sang `process_creation` để tìm hậu khai thác).

Kết luận cho yêu cầu ban đầu "gọi được API miễn phí hay không": **có**, ít nhất với một mô hình, và hệ thống chạy trọn vẹn. Đồng thời mục 6.2.2 bảo đảm rằng nếu không có khoá, hệ thống vẫn chạy ở chế độ không LLM.

7.5.4. Các cơ chế vận hành LLM

Joomla được chạy hai lần liên tiếp với `--redact` và mô hình `auto`, dùng cùng thư mục cache.

Bảng 31. *Hai lần chạy Joomla với cache Prepare và che dữ liệu*

Chỉ số	Lần 1 (cache miss)	Lần 2 (cache hit)
Cache Prepare	chạy mới, đã lưu	dùng lại, PEAK không được gọi
Lời gọi LLM (mọi agent)	9	4
Token (mọi agent)	69.140	13.928
Mô hình thực sự trả lời	`deepseek/deepseek-v4.1-flash`	`deepseek/deepseek-v4.1-flash`
Phiếu judge	0,85 / 0,92 / 0,90	0,85 / 0,85 / 0,95
Kết luận	`ESCALATE_TO_IR` (MEDIUM)	`ESCALATE_TO_IR` (MEDIUM)
Bảng ABLE	giống nhau từng ký tự	giống nhau từng ký tự

Cache bỏ bước tốn nhất (hai phần PEAK) và cho đúng cùng bảng ABLE. Lần chạy đầu mất khoảng 164 giây tính từ lúc ghi cấu hình đến lúc ra tổng hợp, lần hai khoảng 46 giây. Cả hai lần judge đều thống nhất `TRUE_POSITIVE` nhưng độ tin cậy khác nhau, đúng như đã nêu ở mục 8.3.6. Kiểm tra tay các lời nhắc gửi đi trong bài kiểm thử đầu cuối cho thấy tên máy, tên người dùng và IP không xuất hiện trong văn bản

gửi judge và advisor, còn báo cáo vẫn hiện giá trị thật. Mỗi cơ chế (hết ngân sách, chuyển mô hình dự phòng, bỏ temperature khi mô hình từ chối, không cache khi PEAK lỗi) có bài kiểm thử riêng, không cần LLM thật.

7.6. Kiểm thử đơn vị và tính hồi quy

Bộ kiểm thử có 146 bài, tất cả đạt; `ruff` không báo lỗi. Tám bài kiểm thử Splunk thật đã đi cùng adapter, chỉ còn ở commit `4dace56`. Hai nhóm bài có ý nghĩa cho luận văn.

- **Bất biến kiến trúc được khoá bằng kiểm thử:** chuỗi một phần không escalate; judge dưới ngưỡng không đổi luật; kết quả rỗng khi thiếu nguồn cho `COLLECT_DATA_THEN_RERUN`; advisor lỗi không đổi `disposition`; khoá API không có trong cấu hình.
- **Bài hồi quy cho lỗi thật:** `EQUALS` không khớp `splunk-powershell.exe`; `EXISTS` rỗng không khớp; `retry_async` thành công sau lỗi thoáng qua và ném lại lỗi dai dẳng; CLI chạy được khi không có `.env`; phạm vi host rỗng được ghi nhận và chạy lại không lọc host (cả nhánh không hit và nhánh có hit ở host khác); báo cáo ghi `hiển thị / tổng`; `MATCHES` với regex thật tìm được; `EXISTS` không bị che bởi giới hạn quét.

Quá trình làm gọn kho cũng là một thực nghiệm hồi quy: sau khi xoá khoảng 28.000 dòng mã nguồn và các tệp kiểm thử của engine cũ, đường chạy PoC cho đúng cùng kết quả như trước (Joomla có hit, ba PoC còn lại rỗng; lúc đó chưa lộ ra các lỗi 8–10).

7.7. Đánh giá theo yêu cầu

Bảng 32. *Đối chiếu yêu cầu và kết quả*

Yêu cầu	Kết quả	Bằng chứng
FR1, FR3	Đạt	Cổng Prepare; 200 bản ghi hiển thị tái lập qua ba cấu hình
FR2	Đạt	`used_peak = true` ở 4/4 PoC, với hai cấu hình LLM
FR4, FR5	Đạt (sau sửa)	Độ phủ theo cửa sổ, theo host và theo loại sự kiện; 5 disposition; lựa chọn xếp hạng
FR6, FR7	Đạt	`summary.md`, báo cáo, SPL, backlog
FR8	Đạt	`--offline` và bài `test_cli_runs_without_any_llm_configuration`
FR9	Đạt	`--model` và lần chạy Nemotron
NFR1	Đạt	Mọi hàng đến từ adapter; kiểm thử
NFR2	Đạt	Cùng số bản ghi hiển thị qua ba cấu hình
NFR3	Đạt từng phần	Gọi lại và fallback có kiểm thử; lỗi 404 thật chỉ có kiểm thử mô phỏng và quan sát gián tiếp
NFR4	Đạt	Quét không thấy khoá trong `artifacts/runs/*`,

Yêu cầu	Kết quả	Bằng chứng
		`results`, `docs`
NFR5	Đạt	Khoảng 9.400 dòng trong `src/` (gồm luồng Prepare-only)
NFR6	Đạt	Thẻ `v6-engine-final`, nhánh `pre-peak-snapshot`

7.8. Thời gian chạy

Phần tất định nhanh: tìm kiếm trên 4,4 triệu dòng mất vài giây mỗi PoC (6 đến 16 giây gồm cả judge ở lần chạy cuối; quét 2.000 hàng thay vì 100 không làm chậm đáng kể). Phần LLM chiếm hầu hết thời gian. Lần chạy `auto` trước đó mất khoảng bốn phút cho cả bốn PoC, còn lần chạy cuối (ước lượng thô, không bấm giờ chính xác) mất khoảng 12 đến 15 phút cho mỗi cấu hình có LLM; với mô hình Meta Muse ban đầu mất 4 đến 8 phút cho mỗi PoC. Chênh lệch này cho thấy thời gian phụ thuộc mạnh vào dịch vụ LLM tại thời điểm chạy, các con số chỉ dùng để tham khảo.

7.9. Điều chưa được kiểm chứng

Để tránh hiểu nhầm về phạm vi, danh sách sau nêu rõ những điều đồ án chưa làm được.

1. **Splunk thật.** Chưa kiểm tra SPL do hệ thống sinh ra (bản nháp ở pipeline PoC và truy vấn trong kế hoạch Prepare-only) trên một máy chủ Splunk.
2. **`--research`.** Tác tử nghiên cứu của PEAK (cần máy chủ MCP) chưa được thử.
3. **Mô hình dự phòng trên lỗi thật.** Việc chuyển mô hình dự phòng chỉ được kiểm thử bằng lỗi mô phỏng, chưa gặp lỗi thật trong các lần chạy. **Retry trên 404 thật.** Cơ chế gọi lại chỉ được kiểm thử bằng lỗi mô phỏng; trong các lần chạy cuối không gặp lại lỗi 404 nên chưa quan sát nó cứu một lần chạy thật.
4. **Adapter Splunk với các thay đổi truy xuất mới.** Adapter đã được gỡ khỏi nhánh chính (còn ở commit `4dace56`); giới hạn quét 2.000 hàng và tham số `require_nonempty` chưa được chạy trên Splunk thật ở đó, và adapter chưa có độ phủ theo host hay theo loại sự kiện.
5. **Recall.** Chỉ đo được một pha tấn công thật (Joomla); ba pha còn lại thiếu dữ liệu nên chưa đo được khả năng phát hiện. Không có số liệu về độ chính xác tổng thể nào có thể tuyên bố.
6. **Số lần lặp.** Mỗi cấu hình chỉ chạy đầy đủ một đến vài lần; chưa có thống kê về độ biến thiên của judge hoặc advisor ngoài các con số rời rạc đã nêu.
7. **Chất lượng nội dung do PEAK viết.** Hệ thống kiểm tra PEAK có chạy và có trả cấu trúc, nhưng không có người chuyên gia nào chấm chất lượng ABLE hay kế hoạch.

7.10. Thực nghiệm luồng Prepare-only

Mục này báo cáo các lần chạy của luồng mới với mô hình miễn phí `nvidia/nemotron-3-super-120b-a12b:free` (số dư tài khoản không đổi sau mọi lần chạy). Không có lần chạy nào dùng dữ liệu nội bộ; kết quả thực thi trong thử nghiệm xác minh là giả lập và được nêu rõ.

7.10.1. Hai CVE công khai

Bảng 33. *Các lần lập kế hoạch từ PoC công khai*

CVE	Cấu hình	Kho PoC	Giai đoạn	Truy vấn	Độ phủ	Lời gọi LLM	Token
Log4Shell (CVE-2021-44228)	có PEAK	3 kho, ≥ 1.800 sao	3	7	4	18	375.916 (khoảng 10 phút)
Log4Shell	`--no-peak`, bản đầu	3 kho	4	8	—	—	khoảng 16–19 nghìn
Log4Shell	`--no-peak`, sau hai quy tắc mới	3 kho	3	4	3	1	13.142
Spring4Shell (CVE-2022-22965)	có PEAK, bản sửa	3 kho (2.351, 376, 325 sao)	4	4	2	—	—
Spring4Shell	`--no-peak`, sau hai quy tắc mới	3 kho	3	3	2	2	22.726

Các con số "—" là chỗ không còn ghi lại được. Phần PEAK chiếm hầu hết token, tương tự pipeline PoC (mục 7.8). Mỗi dòng là *một* lần chạy với mô hình không tất định, nên bảng cho thấy cỡ độ lớn chứ không cho thấy hiệu quả của từng quy tắc.

Một số quan sát cụ thể:

- **Dấu vết đúng đặc trưng.** Với Spring4Shell, danh sách ứng viên ban đầu thiếu `class.module.classLoader...`, dấu hiệu cốt lõi của lỗ hổng. Nguyên nhân là biểu thức chính quy chỉ nhận tham số dạng đơn giản, và README chiếm hết chỗ trước tệp khai thác. Sau khi sửa biểu thức (tham số có dấu chấm, header tuỳ biến) và ưu tiên tệp khai thác trước README trong bản tóm tắt, các lần chạy đều có dấu hiệu này, cùng `tomcatwar.jsp` và `spring-form.war`.
- **Lỗi thật khi lập kế hoạch.** Lần đầu có PEAK trên Spring4Shell, mô hình viết dấu ngoặc kép lệch trong SPL ở cả ba lượt sửa ("unbalanced double quote") nên không còn truy vấn hợp lệ nào và chương trình dừng với `PlanError`. Lời nhắc

sửa lỗi được bổ sung gợi ý cụ thể (truy vấn đơn giản, đoạn ngắn ổn định thay vì dán payload có dấu ngoặc kép); lần chạy sau đạt.

- **Quy tắc tiến trình cha.** Trong lần chạy Log4Shell `--no-peak` gần nhất, giai đoạn `impact` có một truy vấn endpoint mà mô hình tự viết với `parent_process_name="java"`, và không có truy vấn nào bị loại. Đây chỉ cho thấy lời nhắc mới được mô hình làm theo trong một lần chạy, không phải bằng chứng rằng quy tắc luôn được tuân thủ; các bài kiểm thử đơn vị mới khoá phần mã (truy vấn thiếu tiến trình cha bị gửi lại, rồi bị loại nếu không sửa).
- **Chữ lẫn ngôn ngữ.** Trước khi có bộ phát hiện, `plan.md` của Spring4Shell có các từ "przeciwko" (tiếng Ba Lan), "tentativa" (Bồ Đào Nha), "ungewö" (Đức) và "explotación" (Tây Ban Nha) lẫn trong câu tiếng Việt, mà bộ lọc ký tự không bắt được. Trong lần chạy Spring4Shell sau khi có bộ phát hiện, nó gắn cờ từ `debug`, một từ tiếng Anh hợp lệ chưa có trong từ vựng, và tốn thêm một lời gọi viết lại (22.726 token cho hai lời gọi). Từ vựng đã được bổ sung `debug` cùng các từ kỹ thuật cùng loại; đây là chi phí điển hình của một dương tính giả.
- **Một giai đoạn không còn truy vấn** (khi quy tắc loại hết truy vấn của nó) trước đây sẽ bị `verify` đọc là `CLEAR`. Đã sửa: giai đoạn đó là `INCOMPLETE`, `plan.md` in cảnh báo, và có bài kiểm thử.

7.10.2. Đo bộ phát hiện ngôn ngữ

Hai phép đo nhỏ trên chính mã (không phải trên đầu ra LLM):

- **Độ nhạy:** 62 từ ngoại ngữ thật thuộc chín ngôn ngữ (Bồ Đào Nha, Tây Ban Nha, Đức, Pháp, Ba Lan, Ý, Indonesia, Hà Lan, Thổ Nhĩ Kỳ), mỗi từ đặt vào một câu tiếng Việt: bắt được 62/62 (một tập con của danh sách này nằm trong bài kiểm thử đơn vị). Từ ngắn dưới 4 chữ cái cố ý không xét nên không nằm trong phép đo, và danh sách do tác giả chọn nên 62/62 không phải độ nhạy trên đầu ra thật của mô hình.
- **Dương tính giả:** chạy trên khoảng 17.000 từ văn xuôi tiếng Việt xen thuật ngữ tiếng Anh (README, hai tài liệu thiết kế, báo cáo và sáu chương đầu của luận văn), không từ tiếng Việt nào bị gắn cờ. Lần đầu có 126 từ khác nhau bị gắn cờ, toàn từ tiếng Anh kỹ thuật (`judge`, `adapter`, `placeholder`...); sau khi bổ sung từ vựng còn 8 từ khác nhau (11 lần xuất hiện, như `temperature`, `pytest`, `ruff`). **Từ vựng được chỉnh trên chính tập này, nên con số 8 là lạc quan;** dương tính giả trên văn bản mới sẽ cao hơn, và chi phí của nó là một lời gọi viết lại.

7.10.3. Vòng xác minh bằng kết quả giả lập

Vòng `verify` được thử với `ResultBundle` do tác giả dựng tay (nêu rõ là giả lập) trên kế hoạch Log4Shell. Vòng 1 cho `REFINE`: thư mục `iter2/` chứa các giai đoạn pivot theo nguồn tấn công và host bị nhắm, cửa sổ thu về một giờ quanh các mốc thời gian

trong mẫu. Vòng 2, sau khi kết quả giả lập cho truy vấn pivot ở giai đoạn `impact` có dòng, cho `ESCALATE_AFFECTED`. Đây chỉ chứng minh bộ máy chạy đúng logic đã thiết kế, không chứng minh pivot tìm đúng thứ cần tìm. Các bài kiểm thử đơn vị phủ thêm các nhánh còn lại: kết quả toàn rỗng thành `ACCEPT_NO_EVIDENCE` kèm câu "không phải kết luận sạch"; nguồn rỗng thành `COLLECT_DATA`; truy vấn lỗi hoặc thiếu thành `RERUN_INCOMPLETE`; `plan_id` sai thành `REJECT_RESULTS`; giá trị chứa ký tự nguy hiểm không được nhúng vào truy vấn pivot; số vòng tối đa dừng ở `STOP_REVIEW`.

7.10.4. Điều chưa kiểm chứng ở luồng này

- Mọi truy vấn trong kế hoạch **chưa chạy trên Splunk thật**, kể cả truy vấn độ phủ `tstats`; tên trường theo CIM có thể khác ở mỗi nơi.
- Vòng `verify` chưa nhận **kết quả thật** của một đội thực thi.
- Chất lượng truy vấn (có bắt đúng tấn công không) chưa được chuyên gia chấm; bộ kiểm tra chỉ đảm bảo an toàn và hình thức.
- Hai quy tắc mới (tiến trình cha, ngôn ngữ) mới được thử bằng bài kiểm thử đơn vị và vài lần chạy không PEAK; chưa chạy lại cấu hình có PEAK sau khi thêm chúng.
- Chọn PoC theo số sao là tín hiệu yếu; kho có thể lỗi thời, đã lưu trữ hoặc sai (kế hoạch ghi cảnh báo `archived`, `below_threshold`).

7.10.5. Phiên bản 0.4: độc lập mô hình, quy trình và gắn truy vấn

Các phép thử sau chạy bằng 189 kiểm thử tự động và các lần chạy thật trên CVE-2021-44228.

- **Độc lập mô hình (đã đo ở mức không có LLM).** Với Log4Shell, ba cấu hình cho cùng `core_digest` `ab80eec6…` và cùng `plan_digest`: `--offline`, LLM lỗi xác thực (khoá hết hạn, HTTP 401), và ba mô hình miễn phí bị từ chối (HTTP 404 hết công suất). Trong kiểm thử tự động, năm cấu hình giả lập (mô hình mạnh, yếu, nói lan man, sập, không mô hình) cho phần lõi giống hệt và cùng quyết định `verify` trên cùng kết quả thực thi.
- **Gắn truy vấn.** `bind` bị thử với `index=*`, `_internal`, thiếu ánh xạ cho một nguồn, cửa sổ 200 ngày và cửa sổ trong tương lai; cả năm trường hợp đều bị từ chối và không ghi file. Một lần gắn hợp lệ cho 12 truy vấn với mốc thời gian dạng epoch tuyệt đối.
- **Xác minh bằng kết quả giả lập.** Năm kịch bản (chưa ký, sai digest, đã ký và sạch, đã ký có sự kiện giai đoạn `impact`, đã ký có sự kiện giai đoạn `indicator`) cho lần lượt `REJECT_RESULTS`, `REJECT_RESULTS`, `ACCEPT_NO_EVIDENCE`, `ESCALATE_AFFECTED`, `REFINE`. Kịch bản cuối tạo vòng hai với `plan_digest` mới.
- **Lỗi thật được phát hiện.** Khi NVD mất DNS giữa chừng, CVE do người dùng yêu cầu bị thay bằng một CVE được README nhắc tới; đã sửa để CVE yêu cầu

luôn là CVE chính. Phần lõi ban đầu nhận cả biểu thức chính quy (`Java/.*`) và biến trong mã PoC (`{self.secret}`) làm dấu vết; đã lọc.

- **Có LLM thật (Log4Shell, `--no-peak`, API DeepSeek).** Hai mô hình được chạy trên cùng tình báo: `deepseek-flash` (2 lời gọi, 16.839 token) và `deepseek-v4-pro` (1 lời gọi, 14.846 token). Cả hai cho **cùng `core_digest` `ab80eec6…`** với cấu hình không LLM, trong khi `plan_digest` khác nhau (`a2d86471…` và `d51b5cc9…`) vì gợi ý khác nhau. Lõi gồm 3 giai đoạn, 8 truy vấn; `flash` thêm 4 truy vấn gợi ý (JNDI trong log ứng dụng, `javax.naming`/`JndiLookup`, cổng 1389/1099 trên nguồn mạng, tải tệp `.class` qua proxy), `pro` thêm 2 truy vấn (kết nối cổng 1389/1099, chuỗi `${jndi:ldap|rmi|dns://` trong log ứng dụng). Mọi gợi ý đều qua bộ kiểm tra và gắn được bằng `bind --accept-llm L1,L2`; không gợi ý nào bị loại. Ý nghĩa: mô hình khác nhau cho gợi ý khác nhau, nhưng phần quyết định không đổi.
- **Sự cố cấu hình khi chạy.** Lần đầu gợi ý LLM bị bỏ qua vì tệp `.env` đặt `LLM_MAX_TOKENS` vượt giới hạn của API (400 `Invalid max_tokens`); kế hoạch vẫn ra đủ phần lõi và ghi lý do vào mục "Đã loại bỏ / ghi chú". Chạy lại với `LLM_MAX_TOKENS=16000` thì thành công. Đây cũng là một lần quan sát thực tế cho tính chất "LLM lỗi thì lõi không đổi".

Chưa kiểm chứng: mới có một CVE được chạy với hai mô hình cùng một họ (DeepSeek), chưa so mô hình của nhiều hãng; chưa đánh giá gợi ý nào bắt đúng tấn công thật. Chưa có Splunk thật.

Chương 8. Thảo luận

8.1. Những gì đồ án đạt được

Kết quả quan trọng nhất không nằm ở số liệu mà ở một thay đổi về bản chất của đầu ra. Phiên bản v6 cho người dùng một nhãn (MATCHED hoặc EMPTY) cùng nhận định judge; phiên bản v7 cho một khuyến nghị có cấu trúc: kết luận, mức tin cậy, lý do kiểm chứng được, giới hạn của kết luận, lựa chọn xếp hạng, câu hỏi và rủi ro. Cùng bốn PoC và cùng dữ liệu, sự khác biệt thể hiện rõ nhất ở ba PoC rỗng: nhãn "EMPTY" cũ dễ bị đọc là "không có gì", còn `CLOSE_WITH_CAVEAT` mới buộc người đọc đi qua ba giới hạn trước khi đóng.

Ba điểm thiết kế đáng giữ lại:

1. **Ranh giới tin cậy rõ ràng.** LLM đứng ở hai phía của bộ thực thi (trước: chuẩn bị; sau: giải thích) nhưng không bao giờ ở giữa. Nhờ đó khi LLM sai, thiệt hại giới hạn trong phần văn bản tham khảo.
2. **Luật tất định có giải thích.** Mỗi kết luận có lý do ghi bằng chữ và có kiểm thử khoá hành vi, nên người săn và người phản biện có thể truy vết vì sao hệ thống nói như vậy.
3. **Suy giảm duyên dáng.** Cùng một lệnh chạy được với PEAK, với mô hình miễn phí và không có LLM, và khuyến nghị thay đổi theo cách có thể dự đoán (mục 7.3).

8.2. So sánh với phương án thay thế

Bảng 34. *So sánh ba cách tiếp cận*

Tiêu chí	PEAK Assistant dùng riêng	Engine v6	Hệ thống v7
Chuẩn bị (ABLE, kế hoạch)	Có, tương tác	Tự xây, cổng kiểm tra trường	Có, qua PEAK, chạy hàng loạt
Thực thi trên telemetry	Qua MCP Splunk, do LLM điều khiển	Có, tất định	Có, tất định
Kết quả cuối	Tài liệu kế hoạch	Nhãn và nhận định judge	Khuyến nghị có lý do, giới hạn, rủi ro
Xử lý kết quả rỗng	Không áp dụng	Nhãn EMPTY	Độ phủ nguồn, `COLLECT_DATA_THEN_RERUN`
Chạy khi không có LLM	Không	Một phần	Có, đầy đủ
Kích thước mã	Lớn	~36.400 dòng	~9.200 dòng

Đây không phải so sánh hiệu năng; ba hệ thống phục vụ mục đích khác nhau. PEAK Assistant mạnh ở chỗ cộng tác với người săn qua giao diện trò chuyện, điều mà v7 chủ ý không làm: v7 coi PoC là đầu vào đã được người săn chốt.

8.3. Hạn chế

8.3.1. Khớp literal và độ phủ loại sự kiện

Vị từ là literal: `-enc` không bắt `-EncodedCommand`, `Logon Failed` không bắt biến thể ngôn ngữ khác. Hệ thống ghi nhận giới hạn này nhưng chưa thử biến thể. Độ phủ theo **nguồn** không đủ: nguồn `authentication` có 20.657 bản ghi trong cửa sổ nhưng toàn bộ là đăng nhập thành công, trong khi PoC cần đăng nhập thất bại. Sau lượt rà soát cuối, báo cáo của kết quả rỗng liệt kê các loại sự kiện thực có (`authentication/4624 (20.657)`) để người săn tự đối chiếu, nhưng hệ thống **vẫn chưa tự so** chúng với predicate: điều đó đòi hỏi PoC khai báo loại sự kiện cần có (ví dụ `required_event_types`), một thay đổi định dạng PoC còn để ngỏ.

8.3.2. Phạm vi truy vấn ngầm, cửa sổ thời gian và giới hạn hàng

Đây là điểm yếu thiết kế đã bộc lộ và được xử lý một phần ở lượt rà soát cuối. Phạm vi suy ra từ ABLE của PoC (host trong `able.location`, chuỗi cụ thể trong `able.behavior`) được thêm vào mọi bước. Cơ chế này có ý đồ tốt (PEAK muốn ABLE dẫn dắt truy vấn) nhưng nguy hiểm khi tên máy trong `location` chỉ mang tính mô tả: ở cả ba PoC rỗng, máy đó không ghi nguồn tương ứng nên PoC chưa từng tìm trên dữ liệu thật (mục 7.2.2). Đồ án đã làm ba việc: hiển thị phạm vi, đếm độ phủ trong phạm vi, và chạy lại không lọc host khi phạm vi rỗng. Nhưng gốc vấn đề vẫn còn: việc đọc tên máy từ văn xuôi bằng biểu thức chính quy vẫn là heuristic, có thể chọn nhầm token; một thiết kế sạch hơn là tách một trường `host_scope` tường minh trong PoC.

Về cửa sổ, mỗi PoC chạy trên một cửa sổ (một ngày hoặc 24 phút) và `max_duration` cắt về ba ngày, nên ba PoC rỗng chỉ phủ 1 trên 28 ngày. Về giới hạn hàng, mỗi bước quét tối đa 2.000 hàng và giữ 100 hàng làm bằng chứng; báo cáo hiện ghi `hiển thị / tổng` và dấu ≥ khi quét chạm trần, nhưng số tổng khi bị cắt vẫn chỉ là cận dưới (PoC Joomla có khoảng 19,7 nghìn dòng Joomla trong cả tập dữ liệu). Muốn có số chính xác cần truy vấn `COUNT` riêng. Bản nháp SPL nay dùng cửa sổ của PoC (đổi sang epoch) nhưng chưa thêm bộ lọc host của phạm vi, nên bản nháp phát hiện rộng hơn PoC.

8.3.3. Vai trò của máy và bối cảnh tổ chức

Máy xuất hiện nhiều nhất trong kết quả Joomla là `splunk-02`, thực chất là máy thu log của bộ dữ liệu chứ không phải nạn nhân. Hệ thống chỉ nhắc người dùng xác nhận vai trò, vì nó không biết sơ đồ tài sản. Kết hợp với CMDB hoặc danh sách tài sản sẽ nâng chất lượng khuyến nghị nhưng nằm ngoài phạm vi.

8.3.4. Đo lường chi phí LLM

Bản đầu chỉ đếm token của judge và advisor, trong khi các tác tử bên trong PEAK mới là phần tốn nhất. Hiện nay `UsageMeter` bọc hàm tạo hoàn thành trò chuyện

của SDK OpenAI nên đếm cả hai. Một lần chạy Joomla với `auto` cho thấy khoảng cách: judge và advisor dùng 16.116 token, còn toàn bộ chuỗi là 69.140 token trong 9 lời gọi, tức PEAK chiếm khoảng bốn phần năm. Giới hạn còn lại là các lời gọi dạng stream không trả khối `usage`; chúng chỉ được đếm số lần (`llm_unmetered_calls`). Trong các lần chạy đã thực hiện số này bằng 0. Người dùng có thể đặt trần bằng `--token-budget`.

8.3.5. Phụ thuộc vào một dự án chứng minh khái niệm

PEAK Assistant tự nhận là chưa qua kiểm thử bảo mật và không có bản phát hành PyPI. Hệ thống ghim commit để tái lập, nhưng điều đó đồng nghĩa với việc không tự nhận cập nhật và phải kiểm tra lại mỗi khi nâng cấp. Các hàm PEAK dùng (`able_table`, `plan_hunt`) là giao diện nội bộ, không có cam kết ổn định.

8.3.6. Tính không tất định của LLM

Cùng bằng chứng nhưng judge có thể cho độ tin cậy khác nhau (0,85 đến 0,97 trong các lần chạy). Thiết kế đã giới hạn ảnh hưởng của sự dao động này (chỉ đẩy một bậc, chỉ khi ≥ 0,7), nhưng ngưỡng 0,7 và 0,8 được chọn theo kinh nghiệm chứ chưa hiệu chỉnh trên tập dữ liệu có nhãn. Để giảm dao động, judge nay chạy ở temperature 0 và ba lần, lấy đa số với độ tin cậy thấp nhất trong nhóm đa số. Ở temperature 0 hai lần chạy liên tiếp vẫn cho ba phiếu khác nhau (0,85/0,92/0,90 rồi 0,85/0,85/0,95) dù cùng nhãn `TRUE_POSITIVE`, nên temperature 0 không đảm bảo kết quả lặp lại; bỏ phiếu chỉ làm lộ và giảm độ lệch. Mô hình `auto` còn có thể đổi nhà cung cấp giữa các lần, vì vậy báo cáo ghi mô hình thực sự trả lời (`deepseek/deepseek-v4.1-flash` trong hai lần chạy này) và khi cần tái lập nên cố định `--model`.

8.4. Các mối đe dọa đối với tính hợp lệ

Quy mô nhỏ. Bốn PoC trên một bộ dữ liệu và một pha tấn công có nhãn. Đồ án chứng minh tính khả thi và các tính chất an toàn của thiết kế, không chứng minh hiệu năng phát hiện tổng quát.

Dữ liệu đã biết. BOTS v1 là bộ dữ liệu công khai mà cả PEAK Assistant lẫn mô hình LLM có thể đã thấy trong huấn luyện; việc judge nhận ra chuỗi `acunetix-wvs-test-for-some-inexistent-file` có thể phần nào do kiến thức sẵn có chứ không chỉ do suy luận trên bằng chứng. Với dữ liệu thật của một tổ chức, điều này không đảm bảo.

Thiên lệch người viết PoC. Các PoC được viết sau khi đã biết đáp án (từ bản hướng dẫn BOTS v1), nên kết quả Joomla có hit gần như là hiển nhiên. Giá trị kiểm chứng của Joomla là ở cách hệ thống hạ độ tin cậy và nêu giới hạn, không ở việc tìm ra hit.

Phụ thuộc dịch vụ. Tỷ lệ lỗi 404 và hành vi mô hình `auto` thay đổi theo thời điểm; kết quả thăm dò mô hình miễn phí là bức ảnh tại một thời điểm.

8.5. An toàn và trách nhiệm khi dùng

Khuyến nghị không phải kết luận pháp lý hay lệnh hành động. Hai rủi ro thực tế cần nêu rõ cho người dùng. Thứ nhất, **rò rỉ dữ liệu**: nội dung telemetry rút gọn và mô tả cấu trúc dữ liệu được gửi tới nhà cung cấp LLM; chế độ `--offline`, mô hình cục bộ hoặc ít nhất `--redact` là lựa chọn cho dữ liệu nhạy cảm (nhưng `--redact` không che chuỗi tự do). Thứ hai, **tin cậy thái quá**: một báo cáo trình bày đẹp bằng tiếng Việt dễ khiến người đọc bỏ qua giới hạn. Thiết kế đã đặt giới hạn lên cao trong báo cáo và dùng ngôn ngữ trực tiếp, nhưng không thể thay thế sự đào tạo người dùng. Cuối cùng, hệ thống không thực hiện hành động phản ứng nào; mọi bước cô lập, chặn hay xoá đều thuộc về con người.

8.6. Bài học rút ra

- Chạy bằng dữ liệu thật quan trọng hơn việc thêm kiểm thử. Các lỗi ở mục 7.4 hầu hết chỉ lộ ra khi chạy trên dữ liệu và LLM thật.
- Một kết luận đúng chưa chắc có lý do đúng. Ba PoC rỗng cho cùng khuyến nghị trước và sau khi sửa, nhưng trước đó chúng chưa từng tìm đúng chỗ. Chỉ khi đối chiếu cột "bản ghi nguồn trong cửa sổ" với "trong phạm vi lọc" và truy vấn trực tiếp dữ liệu thì mới thấy. Bài học: mọi bộ lọc ngầm phải hiện ra trong báo cáo, và mọi con số độ phủ phải đếm đúng phạm vi mà truy vấn thực sự chạy.
- Tự đối chiếu tài liệu với mã. Mô tả "MATCHES dùng `re.search`" trong bản thảo luận văn đúng ở tầng lọc nhưng sai ở tầng truy xuất; lỗi chỉ lộ ra khi đọc lại mã khi rà soát.
- Đọc báo cáo như một người dùng. Lỗi độ tin cậy HIGH vô căn cứ không làm hỏng bất kỳ bài kiểm thử nào, nó chỉ lộ ra khi đọc câu "chưa chứng minh được thành công" mà vẫn thấy HIGH.
- Đầu ra của LLM cần được kiểm tra về hình thức và về sự vắng mặt: cả hai lỗi 6 và 7 đều ở dạng "trông hợp lệ nhưng thực chất trống hoặc là thông báo lỗi".
- Làm gọn có thể là một đóng góp. Bỏ khoảng 28.000 dòng mã mà không đổi kết quả khiến hệ thống dễ giải thích hơn nhiều, nhưng cần thẻ git để việc bỏ đi không là mất mát.

8.7. Hạn chế và rủi ro của hướng Prepare-only

Kiểm tra an toàn không phải kiểm tra đúng. `check_spl` chứng minh một truy vấn chỉ đọc và có giới hạn, không chứng minh nó bắt đúng tấn công. Ví dụ điển hình: truy vấn trên đường dẫn `/greeting` của ứng dụng minh hoạ trong PoC trông hợp lệ nhưng ít khi trùng hệ thống thật; truy vấn trên `uri_query` bỏ sót kiểu tấn công gửi

tham số trong thân POST vì nhật ký web thường không ghi thân. Người săn phải đọc `plan.md` trước khi giao đi.

Điểm dừng chủ yếu là khai báo. `verify` chỉ thực sự đọc hành động `escalate` từ kế hoạch; các hành động khác (`narrow`, `collect_data`, `stop_stage`) được ghi vào báo cáo nhưng quyết định cuối do thứ tự cố định ở mục 4.9.4. Sửa tay `stop_conditions` ngoài `escalate` sẽ không đổi hành vi.

"Kết quả" là số dòng. Một truy vấn `stats count by src` trúng 1.000 sự kiện từ ba nguồn chỉ trả ba dòng. Ngưỡng và trần dòng vì vậy phản ánh độ rộng của kết quả hơn là số sự kiện.

Quy tắc tiến trình cha dựa vào mô hình biết tên tiến trình. Ép lọc theo tiến trình cha làm giảm báo nhầm, nhưng nếu mô hình chọn sai họ tiến trình (ví dụ `java` cho một dịch vụ chạy bằng `javaw` hay bằng tên đóng gói khác) thì truy vấn hợp lệ về hình thức nhưng bỏ sót. Các truy vấn pivot do mã sinh (`P2`) không có ràng buộc này vì mã không biết dịch vụ; chúng bị giới hạn bằng host và cửa sổ thời gian, và kết quả của chúng luôn cần người đọc, vì mã không có đường cơ sở để phân biệt shell hợp lệ.

Bộ phát hiện ngôn ngữ là heuristic. Từ vựng tiếng Anh do tác giả soạn nên dương tính giả có chi phí là một lời gọi; từ ngoại ngữ ngắn hơn 4 chữ cái, tên riêng viết hoa đầu câu, và từ ngoại ngữ trùng một âm tiết tiếng Việt hợp lệ sẽ lọt. Nó chỉ bảo vệ chất lượng văn bản cho người đọc, không phải một biện pháp an ninh.

Hạn mức và tính ổn định của nguồn. GitHub không token chỉ cho 60 yêu cầu mỗi giờ; kết quả tìm kiếm theo số sao thay đổi theo thời gian, nên cùng lệnh có thể ra tập kho khác. Mô hình miễn phí cho chất lượng không đều (lỗi ngoặc kép, chữ ngoại ngữ) và mỗi lần chạy khác nhau.

Rủi ro an ninh của chính đầu vào. README và mã PoC có thể chứa lệnh giả mạo nhắm vào LLM (prompt injection). Giảm thiểu bằng hàng rào dữ liệu và nhãn không tin cậy, nhưng quan trọng hơn là *đầu ra của LLM không bao giờ được thực thi và không bao giờ rời dự án mà chưa qua kiểm tra tĩnh*, nên kịch bản xấu nhất là kế hoạch kém chất lượng chứ không phải lệnh nguy hiểm. Dự án cũng không tải hay chạy mã PoC; nó chỉ đọc văn bản.

Đánh giá lại hướng đi. So với pipeline cũ, hướng Prepare-only đánh đổi khả năng tự kiểm tra trên dữ liệu thật để lấy ranh giới dữ liệu sạch. Đổi lại, phần khó nhất là phần dự án không còn kiểm soát: chất lượng truy vấn khi chạy ở môi trường khác. Cách giảm rủi ro hợp lý là hợp đồng dữ liệu rõ (schema, truy vấn độ phủ, `ACCEPT_NO_EVIDENCE` không là "sạch") và một vòng phản hồi qua `verify`, đúng những gì dự án đã làm; nhưng vòng đó chưa được kiểm chứng với kết quả thật.

Độc lập mô hình nghĩa là cùng kế hoạch lõi, không phải cùng chất lượng. Phần lõi hẹp và thận trọng: chỉ tìm đúng chuỗi có trong PoC/CVE, không suy luận biến thể, và không dựng truy vấn tiến trình khi chưa suy ra được tiến trình cha (khi đó

kế hoạch ghi việc này vào mục "Việc con người phải tự làm"). Đổi lại, kết quả lặp lại được và kiểm tra được. Phần gợi ý của LLM vẫn tốt hay xấu theo mô hình, nhưng không ảnh hưởng quyết định của `verify`.

Cổng duyệt của người là quy ước được máy kiểm tra một phần. `verify` từ chối kết quả thiếu người duyệt hoặc sai digest, nhưng không thể biết người đó có thực sự đọc kế hoạch. `plan_digest` chứng minh kết quả thuộc đúng bản kế hoạch, không chứng minh sự cẩn thận.

Kiểm tra `bind` không thay thế quyền hạn trên Splunk. Chặn `index=*` và giới hạn thời gian giảm nguy cơ làm quá tải, nhưng người chạy vẫn có thể dán truy vấn khác vào Splunk. Giới hạn thật (số lượng truy vấn đồng thời, thời gian tối đa) nên đặt thêm bằng vai trò và hạn mức tìm kiếm phía Splunk.

Chương 9. Kết luận và hướng phát triển

9.1. Kết luận

Đồ án đã tái cấu trúc một kho mã săn mối đe dọa cỡ lớn thành một hệ thống nhỏ gọn, tích hợp PEAK Assistant của Cisco Talos cho pha Prepare, giữ pha Execute tất định trên telemetry, và bổ sung pha Act dưới dạng khuyến nghị có xếp hạng để người săn quyết định. Các mục tiêu đề ra ở mục 1.3 được đánh giá như sau.

- **Làm gọn kho:** mã nguồn giảm từ khoảng 36.400 xuống khoảng 9.400 dòng (gồm khoảng 2.300 dòng của luồng Prepare-only), số tệp kiểm thử từ khoảng 80 xuống 9 (146 bài, đều đạt; adapter Splunk khoảng 2.300 dòng cùng tám bài kiểm thử Splunk thật đã gỡ khỏi nhánh chính, còn ở commit `4dace56`), bảo toàn khả năng khôi phục qua thẻ `v6-engine-final`.
- **Tích hợp PEAK Assistant:** cầu nối `prepare.py` gọi `able_table` và `plan_hunt`, chạy thành công trên cả bốn PoC với hai cấu hình LLM khác nhau, có gọi lại có backoff và quay về chế độ không LLM khi lỗi.
- **Khuyến nghị hỗ trợ quyết định:** năm loại khuyến nghị, độ tin cậy, lý do, giới hạn, lựa chọn xếp hạng, câu hỏi và rủi ro; mọi khuyến nghị có `decision_required = true`.
- **LLM tuỳ chọn:** hệ thống chạy trọn vẹn với `--offline` hoặc khi `.env` chưa điền, có kiểm thử riêng, và có thể chọn mô hình miễn phí qua `--model`; một mô hình miễn phí (`nvidia/nemotron-3-super-120b-a12b:free`) đã chạy hết bốn PoC cho cùng khuyến nghị như mô hình trả phí.
- **Thực nghiệm trung thực:** PoC Joomla có 200 bản ghi hiển thị (mỗi bước có ít nhất khoảng 2.000 hàng khớp) và khuyến nghị chuyển IR ở mức tin cậy trung bình, với lý do rõ ràng; ba PoC còn lại cho kết quả rỗng và được đóng kèm cảnh báo, không bị gọi là "sạch". Mười lỗi phát hiện khi chạy thật và khi rà soát cuối đều được ghi lại cùng cách xử lý.
- **Luồng Prepare-only:** từ PoC công khai (NVD, GitHub) sinh `HuntPlan` có schema, mọi truy vấn do LLM viết bị duyệt tĩnh trước khi rời dự án (kể cả quy tắc lọc theo tiến trình cha cho truy vấn hậu khai thác), chữ lẫn ngôn ngữ khác bị yêu cầu viết lại, và `verify` tất định nhận kết quả của đội thực thi để chấp nhận hoặc sinh vòng pivot. Thử trên Log4Shell và Spring4Shell bằng mô hình miễn phí; mục 7.10 nêu số liệu và giới hạn.
- **Tài liệu:** hướng dẫn thực hành từ cài đặt đến viết PoC (Chương 6), tài liệu kiến trúc và định dạng PoC trong thư mục `docs/`.

Đóng góp có tính khái quát hơn là một mẫu kiến trúc: **dùng LLM ở hai đầu của một lõi tất định, ghi rõ ranh giới tin cậy, và trình bày kết quả rỗng như một**

khẳng định có điều kiện. Mẫu này không riêng cho săn mối đe dọa; nó áp dụng được cho bất kỳ quy trình nào mà LLM hỗ trợ con người ra quyết định trên dữ liệu.

9.2. Hạn chế còn lại

Tóm tắt những điều chưa làm hoặc chưa kiểm chứng (chi tiết ở các mục 7.9 và 8.3): chưa chạy trên Splunk thật (kể cả các thay đổi truy xuất mới và mọi truy vấn của kế hoạch Prepare-only); vòng `verify` chưa nhận kết quả thật của một đội thực thi; chưa thử tác tử nghiên cứu `--research`; độ phủ mới liệt kê loại sự kiện chứ chưa tự đối chiếu với predicate; phạm vi host vẫn suy ra từ văn xuôi bằng heuristic; số khớp khi quét chạm trần chỉ là cận dưới; chi phí token của PEAK chưa đo; recall chỉ đo được một pha tấn công; judge chưa được hiệu chỉnh trên dữ liệu có nhãn.

9.3. Hướng phát triển

Các hướng được sắp theo mức độ cấp thiết, từ sửa các thiếu sót đã biết đến mở rộng.

Ngắn hạn (sửa thiếu sót).

1. Đếm số khớp thật bằng truy vấn `COUNT` riêng thay vì cận dưới khi lượt quét chạm giới hạn 2.000 hàng.
2. Tách trường `host_scope` tường minh trong PoC thay vì đọc tên máy từ `able.location` bằng heuristic, và đưa bộ lọc host vào bản nháp SPL.
3. Đối chiếu tự động theo loại sự kiện: PoC khai báo `native_type`/`event_id` cần có (ví dụ 4625), hệ thống so với cơ cấu loại sự kiện đã liệt kê và kết luận "thiếu dữ liệu" khi không có.
4. Chạy kiểm thử tích hợp với Splunk trong một container; cài đặt độ phủ theo cửa sổ, theo host và theo loại sự kiện cho adapter Splunk; kiểm tra giới hạn quét 2.000 hàng và tham số `require_nonempty` ở đó.

Trung hạn (nâng chất lượng).

5. Chạy PoC trên nhiều cửa sổ (quét toàn bộ 28 ngày theo từng ngày) và báo tỷ lệ ngày có hit.
6. Biến thể vị từ có kiểm soát: một tập quy tắc viết lại (ví dụ `-enc` và `-EncodedCommand`) áp dụng tất định chứ không để LLM tự nới.
7. Mở rộng `--redact` sang chuỗi tự do (dòng lệnh, URL, tên miền) và sang văn bản gửi cho PEAK Prepare; thử với mô hình cục bộ trên dữ liệu thật.
8. Hiệu chỉnh ngưỡng judge trên một tập có nhãn: lấy các pha tấn công khác của BOTS v1 và v2, đo tỷ lệ judge trùng nhãn.
9. Làm giàu bằng ngữ cảnh tài sản (vai trò máy, tài khoản dịch vụ) để khuyến nghị không còn phải nhắc "xác nhận vai trò máy".

Cho luồng Prepare-only. Chạy kế hoạch trên một Splunk thật (có thể trong container) và đưa kết quả thật qua `verify`; nhờ chuyên gia chấm chất lượng truy vấn; để `verify` đọc đủ các hành động điểm dừng thay vì chỉ `escalate`; thêm đếm sự kiện thay cho đếm dòng; mở rộng sang PoC không có CVE (bài viết, quy tắc Sigma); mở rộng từ vựng và kiểm tra bộ phát hiện ngôn ngữ trên đầu ra thật nhiều hơn.

Dài hạn (mở rộng phạm vi).

10. Hỗ trợ hai loại săn còn lại của PEAK: baseline (dùng `describe_data` làm nền) và M-ATH.
11. Vòng phản hồi: kết quả quyết định của người săn (chấp nhận, bác bỏ khuyến nghị) được ghi lại và dùng để cải thiện luật và lời nhắc.
12. Tích hợp với hệ thống quản lý sự cố (ticket) để gói `ir/` được chuyển tự động sau khi người săn xác nhận.
13. Đánh giá với người dùng thật: nhờ các nhà phân tích săn mối đe dọa dùng hệ thống trên các giả thuyết của họ và đo mức hữu ích của khuyến nghị, một điều đồ án này chưa làm được.

Tài liệu tham khảo

[1] Splunk SURGe. *Introducing the PEAK Threat Hunting Framework*. Splunk Blog, 2023. https://www.splunk.com/en_us/blog/security/peak-threat-hunting-framework.html

[2] Splunk SURGe. *Hypothesis-Driven Hunting with the PEAK Framework*. Splunk Blog, 2023. https://www.splunk.com/en_us/blog/security/peak-hypothesis-driven-threat-hunting.html

[3] Splunk SURGe. *Baseline Hunting with the PEAK Framework*. Splunk Blog, 11/7/2023. https://www.splunk.com/en_us/blog/security/peak-baseline-hunting.html

[4] Splunk SURGe. *Model-Assisted Threat Hunting (M-ATH) with the PEAK Framework*. Splunk Blog, 17/5/2023. https://www.splunk.com/en_us/blog/security/peak-framework-math-model-assisted-threat-hunting.html

[5] Cisco Talos. *PEAK Assistant: an AI-backed threat hunting assistant that aligns to the PEAK framework* (mã nguồn mở, giấy phép MIT). https://github.com/Cisco-Talos/PEAK-Assistant (commit sử dụng: `dfabbb0fbcc27292ecbb8e32beeccda372bc0783`).

[6] Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., Thomas, C. B. *MITRE ATT&CK: Design and Philosophy*. MITRE Corporation, technical report, 2018. https://attack.mitre.org

[7] Bianco, D. J. *The Pyramid of Pain*. Enterprise Detection & Response (blog), 2013.

[8] Bianco, D. J. *A Simple Hunting Maturity Model*. Enterprise Detection & Response (blog), 2015.

[9] Wu, Q., Bansal, G., Zhang, J., et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv:2308.08155, 2023.

[10] Yao, S., Zhao, J., Yu, D., et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. arXiv:2210.03629.

[11] Zheng, L., Chiang, W.-L., Sheng, Y., et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023 (Datasets and Benchmarks). arXiv:2306.05685.

[12] Ji, Z., Lee, N., Frieske, R., et al. *Survey of Hallucination in Natural Language Generation*. ACM Computing Surveys 55(12), 2023.

[13] Altman, D. G., Bland, J. M. *Absence of evidence is not evidence of absence*. BMJ 311, 485, 1995.

[14] Lewis, P., Perez, E., Piktus, A., et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. arXiv:2005.11401.

[15] Splunk. *Boss of the SOC (BOTS) Dataset Version 1*. https://github.com/splunk/botsv1

[16] Cichonski, P., Millar, T., Grance, T., Scarfone, K. *Computer Security Incident Handling Guide*. NIST Special Publication 800-61 Rev. 2, 2012.

[17] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. https://owasp.org/www-project-top-10-for-large-language-model-applications/

[18] NIST. *National Vulnerability Database: CVE API 2.0*. https://nvd.nist.gov/developers/vulnerabilities

[19] GitHub. *REST API documentation* (search repositories, git trees). https://docs.github.com/en/rest

[20] CISA. *Known Exploited Vulnerabilities Catalog*. https://www.cisa.gov/known-exploited-vulnerabilities-catalog

Ghi chú: các tài liệu [1]–[5] và [15] được kiểm tra bằng tìm kiếm trên web vào thời điểm viết luận văn. Các tài liệu [6]–[14], [16], [17] được trích theo hiểu biết của tác giả về thông tin xuất bản; người nộp luận văn nên đối chiếu năm, số trang và đường dẫn với nguồn gốc trước khi nộp.

Ghi chú thêm: [18] và [19] được chính hệ thống gọi trực tiếp khi chạy thực nghiệm ở mục 7.10; [20] chỉ được tham chiếu gián tiếp qua cờ KEV mà NVD trả về, đường dẫn chưa được kiểm tra riêng.

Phụ lục

Phụ lục A. Tệp PoC mẫu

A.1. PoC Joomla (`pocs/poc-joomla-rce.json`)

```
{
  "poc_id": "poc-joomla-rce",
  "name": "Joomla RCE web compromise (BOTS v1 real attack)",
  "kind": "ttp",
  "summary": "Attacker scans and exploits Joomla search/mailto components on
imreallynotbatman.com.",
  "topic": "web application exploitation",
  "able": {
    "actor": "",
    "behavior": "Exploit public-facing application (T1190) via Joomla search component",
    "location": "web traffic to imreallynotbatman.com",
    "evidence": "web_request telemetry; hit = site imreallynotbatman.com with /joomla/
uri"
  },
  "research_refs": ["BOTS v1 walkthrough - web compromise phase", "MITRE ATT&CK T1190"],
  "scope": "stream:http 2016-08-10 window",
  "max_duration": "3d",
  "plan": "search_text over web telemetry for the victim site + joomla path",
  "steps": [
    {"step_id": "s1-victim-site", "description": "Request to compromised site",
     "target_field": "domain", "op": "EQUALS", "value": "imreallynotbatman.com",
"source_kind": "web"},
    {"step_id": "s2-joomla-path", "description": "Joomla component path",
     "target_field": "cmdline", "op": "CONTAINS", "value": "/joomla/", "source_kind":
"web"}
  ],
  "references": ["MITRE ATT&CK T1190"],
  "expected_chain": ["web_request", "exploitation"],
  "time_window": "2016-08-10T21:36:00Z/2016-08-10T22:00:00Z"
}
```

A.2. PoC mới, có ba bước (`pocs/examples/poc-web-scanner-acunetix.json`)

Ví dụ này khác PoC Joomla ở chỗ thêm bước tìm chuỗi `acunetix` và đặt bước nhận diện máy quét ở vị trí riêng.

```
{
  "poc_id": "poc-web-scanner-acunetix",
  "name": "Automated web vulnerability scanner against the Joomla site (Acunetix)",
  "kind": "behavior",
  "topic": "web application reconnaissance",
  "able": {
    "actor": "",
    "behavior": "Active scanning of a public-facing web application (T1595.002) preceding
exploitation (T1190)",
    "location": "web traffic to imreallynotbatman.com",
    "evidence": "web_request telemetry; hit = domain imreallynotbatman.com with an
acunetix probe uri and /joomla/ paths"
  },
  "research_refs": ["MITRE ATT&CK T1595.002 Vulnerability Scanning", "BOTS v1 walkthrough
- web compromise phase"],
  "scope": "stream:http for imreallynotbatman.com on 2016-08-10 21:36-22:00",
  "max_duration": "3d",
  "plan": "search_text over web telemetry for the victim site, then the acunetix probe
string, then the Joomla path",
```

```
  "time_window": "2016-08-10T21:36:00Z/2016-08-10T22:00:00Z",
  "steps": [
    {"step_id": "s1-victim-site",    "target_field": "domain",  "op": "EQUALS",
"value": "imreallynotbatman.com", "source_kind": "web"},
    {"step_id": "s2-acunetix-probe", "target_field": "cmdline", "op": "CONTAINS",
"value": "acunetix",              "source_kind": "web"},
    {"step_id": "s3-joomla-enum",    "target_field": "cmdline", "op": "CONTAINS",
"value": "/joomla/",              "source_kind": "web"}
  ],
  "references": ["MITRE ATT&CK T1595.002", "MITRE ATT&CK T1190"],
  "expected_chain": ["web_request"]
}
```

Chạy với `--offline` trên BOTS v1: 204 bản ghi hiển thị (100 / ≥1.999, 6 và 100 / ≥2.000 cho từng bước), khuyến nghị `INVESTIGATE_FURTHER` (MEDIUM).

A.3. Bảng trường của PoC

Bảng 35. *Các trường của tệp PoC*

Trường	Bắt buộc	Mô tả
`poc_id`, `name`, `kind`, `summary`	`poc_id`, `name`	Định danh và mô tả ngắn
`topic`	Có	Chủ đề săn (PEAK: select topic)
`able.actor`	Không (được để trống)	Tác nhân đe dọa
`able.behavior`, `able.location`, `able.evidence`	Có	Ba thành phần còn lại của ABLE
`research_refs`	Có	Tài liệu nghiên cứu đã đọc
`scope`	Có	Hệ thống, dữ liệu và khoảng thời gian
`max_duration`	Có	Thời lượng tối đa, ví dụ `3d`
`plan`	Có	Kế hoạch tóm tắt
`steps[]`	Có	`step_id`, `description`, `target_field`, `op`, `value`, `source_kind`
`fallbacks[]`	Không	Bước thay thế khi lượt đầu rỗng
`references[]`, `expected_chain[]`	Không	Tham chiếu MITRE; chuỗi quan sát kỳ vọng
`time_window`	Không	Cửa sổ mặc định `START/END` (ISO-8601 UTC)

Giá trị hợp lệ: `op` thuộc {`EQUALS`, `CONTAINS`, `STARTS_WITH`, `ENDS_WITH`, `MATCHES`, `EXISTS`}; `source_kind` thuộc {`process`, `web`, `dns`, `authentication`, `file`, `smb`}.

Phụ lục B. Trích báo cáo khuyến nghị mẫu

Dưới đây là phần đầu của `recommendation.md` cho PoC Joomla (lần chạy `auto`); các mục từ "Bằng chứng" trở đi được lược bớt.

```
# Khuyến nghị hunt — poc-joomla-rce


PoC: Joomla RCE web compromise (BOTS v1 real attack)
Cửa sổ dữ liệu: 2016-08-10T21:36:00Z/2016-08-10T22:00:00Z
Nguồn dữ liệu: CDB data/botsv1_eval.sqlite
```

```
PEAK Assistant: đã dùng (ABLE + hunt plan)

## Khuyến nghị
### Chuyển IR (escalate)
ESCALATE_TO_IR — độ tin cậy trung bình (MEDIUM).
> Đây là gợi ý hỗ trợ quyết định. Người săn mối đe dọa là người
> quyết định cuối cùng; hệ thống không tự hành động.

Lý do
- 2/2 bước PoC có kết quả, tổng 200 bản ghi khớp.
- Khoảng thời gian hit: 2016-08-10T21:36:45Z → 2016-08-10T21:40:57Z.
- Judge (advisory) đánh giá TRUE_POSITIVE, độ tin cậy 0.85.
- Độ tin cậy bị hạ xuống MEDIUM: PoC không có bước nào kiểm tra kết quả
  (status/action) nên chưa chứng minh được tấn công thành công.

Cần lưu ý (giới hạn của kết luận)
- Khớp predicate literal chứng minh có hoạt động tương ứng trong dữ liệu;
  tự nó chưa chứng minh thành công hay tác động.
- Bước s1-victim-site: chỉ hiển thị 100 bản ghi trong ít nhất 1999 bản ghi khớp
  (quét đã dừng ở giới hạn hàng nên tổng thực tế có thể lớn hơn).
- Bước s2-joomla-path: chỉ hiển thị 100 bản ghi trong ít nhất 2000 bản ghi khớp.

## Các lựa chọn cho người quyết định (xếp theo ưu tiên)
1. ESCALATE_TO_IR       — chuyển gói bằng chứng cho IR; trước khi cô lập,
                          xác nhận vai trò thật của splunk-02.
2. INVESTIGATE_FURTHER  — pivot theo splunk-02 cùng khoảng thời gian.
3. TUNE_POC_OR_CLOSE    — thêm điều kiện loại trừ rồi chạy lại.

## Bằng chứng
| Bước           | Predicate                              | Nguồn | Nguồn/cửa sổ | Trong
phạm vi | Khớp (hiển thị / tổng) |
| s1-victim-site | domain EQUALS imreallynotbatman.com | web   | 12,546       | 12,546
| 100 / ≥1999            |
| s2-joomla-path | cmdline CONTAINS /joomla/           | web   | 12,546       | 12,546
| 100 / ≥2000            |

## Judge (LLM, chỉ tham khảo)
TRUE_POSITIVE — 0.85. Requests to /acunetix-wvs-test-for-some-inexistent-file and
random 8-character URIs followed by Joomla component enumeration within a tight
21:36-21:40 window match automated Joomla vulnerability scanning ...

## Rủi ro nếu quyết định sai
- Chưa có bước kiểm tra status/action nên chưa chứng minh RCE thành công.
- Mẫu có /acunetix-wvs-test-for-some-inexistent-file gợi ý scanner Acunetix.
- Việc escalate lên IR khi chưa xác nhận thành công có thể lãng phí nguồn lực.

## Act: bản nháp phát hiện (SPL)  — trạng thái DRAFT, chưa chạy trên Splunk thật; cửa sổ
lấy từ PoC (epoch UTC)
search index="botsv1" domain="imreallynotbatman.com" match(cmdline, "(?i)/joomla/")
earliest=1470864960 latest=1470866400
| table _time, host, user, image, cmdline, domain, file_path, action
```

Phụ lục C. Tham chiếu dòng lệnh và biến môi trường

C.1. Dòng lệnh

```
python main.py [--poc FILE ...] [--poc-dir DIR] [--window START/END]
               [--db PATH]
               [--env FILE] [--model NAME] [--offline] [--research] [--peak-timeout
SECONDS]
               [--judge-votes N] [--token-budget N] [--redact]
```

```
              [--cache-dir DIR] [--no-cache] [--refresh-prepare]
              [--out DIR]
```

Mã thoát khác 0 nếu có PoC bị lỗi nạp hoặc lỗi cổng Prepare; các PoC còn lại vẫn được chạy.

Luồng Prepare-only dùng ba lệnh con:

```
python main.py plan (--cve CVE-YYYY-NNNN | --repo OWNER/NAME | --query TEXT)
                    [--min-stars N] [--max-repos N] [--lookback 14d] [--max-rows N] [--
max-queries N]
                    [--max-iterations N] [--env FILE] [--model NAME] [--no-peak] [--
refresh]
                    [--token-budget N] [--cache-dir DIR] [--out DIR]
python main.py verify --plan PLAN.json --results RESULTS.json [--out DIR]
python main.py schema [--out DIR]
```

`plan` thoát với mã 2 nếu đầu vào sai hoặc nguồn công khai không đọc được, mã 3 nếu không có LLM (tình báo vẫn được lưu) và mã 4 nếu bộ lập kế hoạch không tạo được truy vấn hợp lệ nào.

C.2. Biến môi trường

Bảng 36. *Biến môi trường*

Biến	Mặc định	Ý nghĩa
`LLM_ENDPOINT`	(không)	URL chat-completions tương thích OpenAI
`LLM_API_KEY`	(không)	Khoá API; không bao giờ ghi vào tệp kết quả
`LLM_MODEL`	(không)	Tên mô hình; `--model` ghi đè
`LLM_TIMEOUT`	600	Hạn của mỗi lời gọi (giây)
`LLM_MAX_TOKENS`	16000	Giới hạn token đầu ra
`LLM_MODEL_FALLBACKS`	(không)	Mô hình dự phòng, cách nhau bằng dấu phẩy; chuyển khi mô hình chính hỏng hẳn
`LLM_TEMPERATURE`	0	Temperature của judge và advisor; `none` = mặc định nhà cung cấp
`LLM_MIN_INTERVAL`	0	Số giây tối thiểu giữa hai lời gọi judge/advisor
`GITHUB_TOKEN`	(không)	Tuỳ chọn, cho `plan`: nâng hạn mức GitHub từ 60 yêu cầu mỗi giờ; chỉ gửi tới `api.github.com`
`NVD_API_KEY`	(không)	Tuỳ chọn, cho `plan`: khoá NVD; chỉ gửi tới `services.nvd.nist.gov`

Biến môi trường của tiến trình, nếu có, được ưu tiên hơn giá trị trong `.env`; `--model` ưu tiên hơn cả hai. Các biến `SPLUNK_*` và cờ `--provider splunk` thuộc adapter Splunk, chỉ còn trong lịch sử git (commit `4dace56`).

Phụ lục D. Lược đồ `recommendation.json`

Bảng 37. *Các khoá cấp cao của* `recommendation.json`

Khoá	Kiểu	Nội dung
`poc_id`, `poc_name`	chuỗi	Định danh PoC
`disposition`, `disposition_label`	chuỗi	Khuyến nghị và nhãn tiếng Việt
`confidence`	chuỗi	`HIGH`, `MEDIUM`, `LOW`
`headline`	chuỗi	Câu tóm tắt
`reasons`, `caveats`	danh sách chuỗi	Lý do và giới hạn
`options`	danh sách	Mỗi phần tử có `action`, `rank`, `rationale`
`evidence`	đối tượng	`window`, `steps[]` (`step_id`, `description`, `predicate`, `source_kind`, `row_count`, `source_rows_in_window`, `source_rows_in_scope`, `matched_total`, `scan_truncated`), `scope` (`host`, `actor`, `observables`), `scope_empty_sources`, `capped_steps`, `source_breakdown`, `source_hosts`, `unscoped_probe`, `matched_steps`, `total_steps`, `observations`, `first_seen`, `last_seen`, `pivots`, `top_values`, `samples`, `chain_complete`, `missing_sources`, `unverifiable_sources`
`judge`	đối tượng	`verdict`, `confidence`, `rationale`, `notes`
`next_steps`, `questions_for_hunter`, `risks`	danh sách	Do advisor tạo
`decision_required`	boolean	Luôn `true`
`advisor_note`	chuỗi	Ghi chú về trạng thái advisor (ví dụ khi thất bại)
`execution`	đối tượng	`verdict` (MATCHED/EMPTY), `window`, `scope_note`, `ir_escalation_path`
`peak`	đối tượng	`used_peak`, `able_markdown`, `hunt_plan_markdown`, `research_markdown`, `notes`
`meta`	đối tượng	`data_source`, `used_peak`, `peak_notes`, `able_file`, `plan_file`, `llm_calls`, `llm_tokens`, `hunt_seconds`, `ledger_path`, `model_configured`, `models_used`, `model_switches`, `llm_all_calls`, `llm_all_tokens`, `token_budget`, `token_budget_exhausted`, `prepare_cache`, `redacted`, `judge_votes`, `temperature`

Phụ lục E. Danh sách kiểm thử

Bảng 38. *Phân bố 146 bài kiểm thử theo tệp*

Tệp	Số bài	Nội dung
`test_pipeline.py`	25	Luật khuyến nghị, advisor, cấu hình LLM, PEAK, độ phủ CDB, CLI
`test_poc.py`	30	Mô hình PoC, toán tử, khớp chính xác, tinh chỉnh, cổng Prepare
`test_act.py`	8	Bản nháp SPL, backlog, ghi chú stakeholder
`test_cdb_adapter.py`	2	Hợp đồng và phạm vi của adapter CDB
`test_peak_execute.py`	2	Phân tích kế hoạch PEAK thành quan sát cụ thể
`test_scope_and_caps.py`	15	Phạm vi host ngầm, giới hạn hàng, `MATCHES`/`EXISTS`, độ phủ theo loại sự kiện, cửa sổ SPL
`test_llm_ops.py`	16	Đếm token và ngân sách, mô hình dự phòng (kể cả `ImportError` không đổi mô hình), temperature, bỏ phiếu judge, cache Prepare, che dữ liệu, chạy đầu cuối
`test_intel_plan.py`	40	Luồng Prepare-only: `Fetcher` (host cho phép, chuyển hướng, hạn mức, token), NVD/GitHub, trích dấu vết, `check_spl`, dựng kế hoạch và sửa lỗi, `bind`, `verify`, vòng pivot, dòng lệnh
`test_plan_language_parent.py`	8	Bộ phát hiện từ ngoại ngữ, quy tắc tiến trình cha, một lượt viết lại cho lỗi chữ, giai đoạn mất hết truy vấn

Tám bài kiểm thử Splunk thật và các bài về adapter Splunk (16 bài trong `test_splunk_live_adapter.py` và 3 bài trong `test_v5_adapters.py`) đã gỡ khỏi nhánh chính (còn ở commit `4dace56`); hai bài CDB còn lại của tệp sau nằm ở `test_cdb_adapter.py`.

Các bài của `test_pipeline.py` (tên rút gọn): `test_disposition_rules` (11 tổ hợp); `test_escalate_is_high_only_when_an_outcome_field_is_tested`; `test_empty_result_never_reads_as_clean_when_source_missing`; `test_advisor_parses_json_and_survives_garbage`; `test_advisor_retries_until_usable_and_accepts_drifted_shapes`; `test_llm_settings_from_env_file`; `test_llm_settings_rejects_placeholders`; `test_prepare_without_llm_uses_poc_plan_and_says_so`;

`test_prepare_degrades_when_peak_agents_fail`; `test_prepare_retries_a_transient_peak_failure`; `test_retry_async_recovers_from_transient_errors_and_reraises_persistent_ones`; `test_cdb_source_presence_and_description`; `test_cli_offline_end_to_end`; `test_cli_requires_a_poc`; `test_cli_runs_without_any_llm_configuration`.

Lệnh chạy: `python -m pytest tests -q` (kết quả: 146 đạt) và `python -m ruff check src tests` (không báo lỗi).

Phụ lục F. Kết quả từng lần chạy

Bảng 39. *Tổng hợp các lần chạy cuối (sau khi sửa phạm vi truy vấn) đã dùng trong luận văn*

Cấu hình	PoC	Khuyến nghị	Tin cậy	Lời gọi LLM	Token	Thời gian Execute (giây)	Judge
`auto`	joomla	`ESCALATE_TO_IR`	MEDIUM	2	9.524	16,1	TRUE_POSITIVE 0,85
`auto`	bruteforce	`CLOSE_WITH_CAVEAT`	MEDIUM	1	7.004	6,1	(không có hit)
`auto`	c2-beacon	`CLOSE_WITH_CAVEAT`	MEDIUM	1	6.695	6,2	(không có hit)
`auto`	pdf-exploit	`CLOSE_WITH_CAVEAT`	MEDIUM	1	8.087	8,8	(không có hit)
Nemotron free	joomla	`ESCALATE_TO_IR`	MEDIUM	2	7.280	8,3	TRUE_POSITIVE 0,92
Nemotron free	bruteforce	`CLOSE_WITH_CAVEAT`	MEDIUM	1	4.814	6,1	(không có hit)
Nemotron free	c2-beacon	`CLOSE_WITH_CAVEAT`	MEDIUM	1	6.362	6,4	(không có hit)
Nemotron free	pdf-exploit	`CLOSE_WITH_CAVEAT`	MEDIUM	1	4.949	9,3	(không có hit)
`--offline`	joomla	`INVESTIGATE_FURTHER`	MEDIUM	0	0	—	—
`--offline`	ba PoC còn lại	`CLOSE_WITH_CAVEAT`	MEDIUM	0	0	—	—

Phụ lục G. Cấu trúc thư mục kho

```
AI_Agent_Hunting/
├── main.py                 # điểm vào: gọi hunting.cli
├── pyproject.toml          # phụ thuộc; extra [peak] và [dev]
├── .env.example            # mẫu cấu hình LLM (và token tuỳ chọn cho nguồn công khai)
├── pocs/                   # 4 PoC; examples/ chứa PoC mẫu
├── src/hunting/
│   ├── llm.py  redact.py  prepare.py  recommend.py  report.py  pipeline.py  cli.py
│   ├── plan_cli.py         # lệnh plan / verify / schema (luồng Prepare-only)
│   ├── intel/              # NVD, GitHub, trích dấu vết, tóm tắt cho LLM
│   ├── plan/               # HuntPlan, kiểm tra SPL, ngôn ngữ, dựng kế hoạch, xác minh
│   ├── poc/                # mô hình PoC, tác tử thực thi, judge
│   ├── adapters/           # CDB (SQLite); Splunk: commit 4dace56
```

```
│   ├── act/                # SPL, backlog, stakeholder
│   └── peak.py  contracts/  controller/
├── scripts/                # nạp BOTS v1 vào SQLite
├── tests/unit/             # 146 bài kiểm thử
├── results/botsv1-peak-run/# kết quả đã commit
└── docs/                   # kiến trúc, định dạng PoC, PREPARE-WORKFLOW, báo cáo,
thesis/, images/
```

Phụ lục H. Bảng thuật ngữ

Bảng 40. *Thuật ngữ và từ viết tắt*

Thuật ngữ	Giải nghĩa
ABLE	Actor, Behavior, Location, Evidence: mô hình mô tả giả thuyết của PEAK
Advisor	Lời gọi LLM sinh bước tiếp theo, câu hỏi và rủi ro; không đổi kết luận
Adapter	Lớp truy cập telemetry (CDB/SQLite; adapter Splunk chỉ còn trong lịch sử git)
ATT&CK	Cơ sở tri thức chiến thuật và kỹ thuật của MITRE
BOTS	Boss of the SOC, bộ dữ liệu và cuộc thi của Splunk
CIM	Common Information Model của Splunk: tên trường chuẩn dùng trong truy vấn của kế hoạch
CDB	Cơ sở dữ liệu telemetry dạng SQLite của dự án
Disposition	Loại khuyến nghị (năm giá trị ở mục 4.6.1)
HuntPlan	Kế hoạch săn dạng dữ liệu do `plan` sinh: giai đoạn, dấu vết, truy vấn, giới hạn, điểm dừng (mục 4.9.2)
KEV	Known Exploited Vulnerabilities: danh mục lỗ hổng đã bị khai thác ngoài thực tế của CISA
NVD	National Vulnerability Database: cơ sở dữ liệu lỗ hổng của NIST
Pivot	Chuyển hướng săn theo giá trị quan sát được (nguồn tấn công, host, tài khoản) trong vòng kế tiếp
Placeholder	Ký hiệu `{{INDEX_*}}`, `{{EARLIEST}}`, `{{LATEST}}`, `{{MAX_ROWS}}` trong truy vấn của kế hoạch, do đội thực thi thay bằng giá trị thật
ResultBundle	Kết quả đội thực thi trả về cho `verify` (mục 4.9.2)
Execute	Pha thực thi của PEAK; trong đề tài là phần tất định
IR	Incident Response: ứng cứu sự cố
Judge	Lời gọi LLM đánh giá bản ghi khớp; chỉ tham khảo
LLM	Mô hình ngôn ngữ lớn
M-ATH	Model-Assisted Threat Hunt: săn có hỗ trợ mô hình
MCP	Model Context Protocol: giao thức nối công cụ ngoài với LLM
PEAK	Prepare, Execute, Act with Knowledge

Thuật ngữ	Giải nghĩa
PoC	Proof of Concept: ở đây là giả thuyết đã cụ thể hoá thành vị từ
Predicate (vị từ)	Điều kiện `trường – toán tử – giá trị` kiểm tra trên bản ghi
SPL	Search Processing Language của Splunk
Telemetry	Dữ liệu quan sát từ hệ thống (log, sự kiện)
TTP	Tactics, Techniques and Procedures

Phụ lục I. Trích kế hoạch săn sinh ra (Log4Shell, không dùng PEAK)

Đây là trích từ tệp `plan.md` do lệnh `python main.py plan --cve CVE-2021-44228 --min-stars 500 --no-peak` sinh ra (mô hình `nvidia/nemotron-3-super-120b-a12b:free`, một lời gọi, 13.142 token), chỉ bỏ các hàng rào mã lồng nhau. Kế hoạch đầy đủ có ba giai đoạn, bốn truy vấn phát hiện và ba truy vấn độ phủ; ở đây giữ phần đầu, giai đoạn hậu khai thác (có truy vấn endpoint lọc theo tiến trình cha) và một truy vấn độ phủ. Phần điểm dừng chung của mỗi giai đoạn do mã sinh, không phải LLM.

I.1. Đầu kế hoạch

```
# Kế hoạch săn: Kế hoạch săn lôg CVE-2021-44228 (Log4Shell)

- **Plan ID:** `hp-cve-2021-44228-a6a545ad` · vòng 1
- **Tạo lúc:** 2026-10-09T10:21:42Z · model `nvidia/nemotron-3-super-120b-a12b:free` · PEAK Assistant: không
- **Phạm vi của bản kế hoạch này:** chỉ giai đoạn **Prepare**. Không truy cập hệ thống nội bộ, không chạy PoC; các truy vấn dưới đây do đội Execute chạy trên dữ liệu của họ.

## Giả thuyết

Giả sử công khai PoC cho CVE-2021-44228 được dùng chống lại môi trường của chúng ta; kiểm tra xem có dấu hiệu tấn công nào xảy ra không.

## Tình báo đầu vào

- **CVE:** CVE-2021-44228 · mức CRITICAL (10.0) · **CISA KEV: đã bị khai thác ngoài thực tế**
- **Mô tả:** Apache Log4j2 2.0-beta9 through 2.15.0 (excluding security releases 2.12.2, 2.12.3, and 2.3.1) JNDI features used in configuration, log messages, and parameters do not protect against attacker controlled LDAP and other JNDI related endpoints. An attacker who can control log messages or log message parameters can execute arbitrary code loaded from LDAP servers when message lookup substitution is enabled. From log4j 2.15.0, this behavior has been disabled by default. From version 2.16.0 (along with 2.12.2, 2.12.3, and 2.3.1), this functionality has been completely removed. Note that this vulnera
- **PoC:** [fullhunt/log4j-scan](https://github.com/fullhunt/log4j-scan) · 3,426 sao
- **PoC:** [NCSC-NL/log4shell](https://github.com/NCSC-NL/log4shell) · 1,881 sao · đã lưu trữ (archived)
- **PoC:** [kozmer/log4j-shell-poc](https://github.com/kozmer/log4j-shell-poc) · 1,846 sao · đã lưu trữ (archived)
- ⚠ NCSC-NL/log4shell is archived
- ⚠ kozmer/log4j-shell-poc is archived
```

I.2. Giai đoạn hậu khai thác

```
### S3 — Kết nối ra ngoài tới miền OAST hoặc tạo tiến trình shell từ dịch vụ Java

*Giai đoạn:* `impact` · ATT&CK: T1071, T1059 · *ý nghĩa:* dấu hiệu thành công / hậu khai thác · chạy sau S2

Sau khi khai thác thành công, attacker thực hiện gọi ra ngoài tới máy chủ OAST (DNS, HTTP) để xác nhận hoặc tải payload, đồng thời tạo tiến trình shell (ví dụ: /bin/sh) dưới dạng tiến trình con của quá trình Java.

| Dấu vết | Giá trị | Nguồn gốc |
|---|---|---|
| oast_domain | `interact.sh` | có trong PoC |
| process | `/bin/sh` | có trong PoC |

**S3-Q1** — Phát hiện truy vấn DNS tới miền OAST phổ biến dùng để xác nhận khai thác (nguồn `dns`, có chuỗi lấy từ PoC)

search index={{INDEX_DNS}} earliest={{EARLIEST}} latest={{LATEST}} query="*interact.sh*" | stats count by src, query | head {{MAX_ROWS}}
Trường kỳ vọng: src, query
Hoạt động hợp lệ có thể khớp: Truy vấn DNS tới các miền OAST có thể xảy ra từ các công cụ bảo mật hợp pháp hoặc dịch vụ nội bộ, nhưng rất hiếm trong môi trường doanh nghiệp thường lệ.

**S3-Q2** — Phát hiện tạo tiến trình shell từ quá trình Java, dấu hiệu của thực thi mã thành công (nguồn `endpoint`, có chuỗi lấy từ PoC)

search index={{INDEX_ENDPOINT}} earliest={{EARLIEST}} latest={{LATEST}} process_name="/bin/sh" parent_process_name="java" | stats count by dest, user, process_name, parent_process_name | head {{MAX_ROWS}}
Trường kỳ vọng: dest, user, process_name, parent_process_name
Hoạt động hợp lệ có thể khớp: Quản trị viên hoặc các script quản trị cũng có thể khởi tạo shell từ Java, nhưng thường không thấy trong môi trường production và thường có tài liệu kèm theo.

**Điểm dừng:**
- khi nguồn không có dữ liệu → bổ sung telemetry — Nguồn dữ liệu không có sự kiện nào trong cửa sổ: dừng giai đoạn này; kết quả rỗng KHÔNG có nghĩa là sạch.
- khi kết quả bị cắt ở trần dòng → thu hẹp rồi chạy lại — Chạm trần 200 dòng: thu hẹp (cửa sổ, host, giá trị) rồi chạy lại, không nới rộng.
- khi lỗi/quá thời gian → dừng giai đoạn — Lỗi hoặc quá thời gian: báo cho người vận hành, không tự thử lại vô hạn.
- khi hết ngân sách → dừng toàn kế hoạch — Hết ngân sách truy vấn hoặc thời gian: dừng toàn kế hoạch và báo phần đã làm.
- khi có ≥ 1 kết quả → chuyển IR — Có dấu hiệu khai thác thành công: dừng mở rộng, chuyển IR kèm bằng chứng.
- khi không có kết quả → tiếp tục — Không thấy trong dữ liệu đã quét; vẫn phải xét độ phủ nguồn.
```

I.3. Truy vấn độ phủ

```
**C-web** — Kiểm tra nguồn 'web' có dữ liệu trong cửa sổ (rỗng ≠ sạch)

| tstats count min(_time) as first_seen max(_time) as last_seen where index={{INDEX_WEB}} earliest={{EARLIEST}} latest={{LATEST}} by sourcetype | head 20
```